מבוא לאבטחת סייבר · 689156 · המכללה למינהל

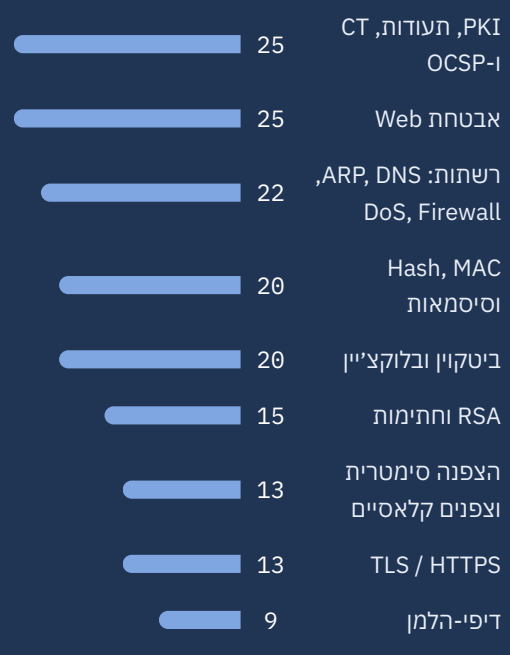
חוברת סיכום לקראת המבחן

כל חומר הקורס בתמצית, מוצלב עם 162 השאלות שהופיעו במבחנים בין 2022 ל-2026. הדגש על תיאוריה וחישובים. בסוף: מלכודות שחוזרות, נוסחאות, 17 דוגמאות חישוב פתורות ומילון מושגים.

איך לעבוד עם החוברת

1. קוראים פרק, ואז פותרים את שאלות הנושא שלו באפליקציית התרגול. מתקשים בנושא? יש סרטון הסבר בספריית הסרטונים.
2. כל טעות: חוזרים לתיבות **במבחן** ו-**מלכודת** של הנושא.
3. יום לפני המבחן: פרק "טענות שחוזרות", דף הנוסחאות, הדוגמאות הפתורות והמילון.

משקל הנושאים במאגר השאלות



מבנה המבחן: 10 שאלות אמריקאיות, שעה ועשרים דקות (8 דקות לשאלה), בלי חומר עזר. אין קוד: השאלות הן תיאוריה וחישובים מתמטיים. שאלות קוד ממבחני עבר מופיעות כאן כשאלות תיאוריה.

תוכן העניינים

1	מבוא, מודלי תוקף ותקשורת ברשת	3
2	הצפנה סימטרית וצפנים קלאסיים	7
3	פונקציות Hash, סיסמאות ו-MAC	13
4	קריפטוגרפיה אסימטרית: דיפי-הלמן, RSA וחתימות	18
5	PKI ותעודות דיגיטליות	23
6	תקשורת מאובטחת: TLS ו-HTTPS	28
7	ביטקוין ובלוקצ'יין	31
8	אבטחת Web	35
9	אבטחת רשת: ARP, DNS, DoS וחומות אש	39
10	טענות שחוזרות במבחנים	43
11	דף נוסחאות ודוגמאות פתורות	44
12	מילון מושגים	48

הסילבוס מול החוברת

נושא בסילבוס	מה כולל	פרק
מבוא	עקרונות, דוגמאות מהתקפות עבר, מודלי תוקף (off-path, MitM)	1
קריפטוגרפיה	צפנים היסטוריים, הצפנה סימטרית וא-סימטרית, אימות וחתימות, hash	2, 3, 4
Cryptocurrency and blockchains	ביטקוין: יסודות, טכנולוגיה, קריפטוגרפיה; בלוקצ'יין	7
תקשורת מאובטחת	PKI, מנגנוני החלפת מפתחות, TLS	4, 5, 6
הגנה ב-web	עוגיות, SOP, SQL injection, XSS, XSRF, clickjacking	8
הגנה בשכבות אחרות	סריקות, sniffing, spoofing, הרעלת ARP ו-Kali, DNS, TCP, DoS/DDoS, Botnets	1, 9

1 מבוא, מודלי תוקף ותקשורת ברשת

הקורס עוסק בשאלה אחת שחוזרת בכל פרק: **שני צדדים רוצים לתקשר דרך רשת שהם לא שולטים בה, ומישהו באמצע מנסה להזיק.** כדי לענות על שאלות במבחן צריך לדעת שלושה דברים: מה התוקף **יכול** לעשות (מודל התוקף), מה אנחנו רוצים **להבטיח** (מטרות האבטחה), ואיזה **כלי** מבטיח כל מטרה. הפרק הזה בונה את התשתית הזאת.

מה אנחנו רוצים להבטיח

כמעט כל שאלה במבחן מתרגמת בסוף לשאלה "איזו תכונה נפגעה" או "איזה מנגנון נותן את התכונה הזאת". אלה חמש התכונות שחוזרות לאורך כל הקורס:

תכונה	מה היא אומרת	דוגמה לפגיעה	מי מספק אותה
סודיות (Confidentiality)	רק מי שמורשה יכול לקרוא את התוכן	מאזין קורא סיסמה שנשלחה בגלוי	הצפנה (פרקים 2, 4)
שלמות / אמינות (Integrity)	ההודעה הגיעה בדיוק כפי שנשלחה	תוקף משנה את סכום ההעברה	Hash, MAC, חתימה (פרקים 3, 4)
אותנטיות (Authenticity)	ההודעה באמת הגיעה ממי שטוען ששלח אותה	תוקף שולח הודעה "בשם" אליס	MAC, חתימה, תעודות (פרקים 3–5)
אי-הכחשה (Non-repudiation)	השולח לא יכול לטעון אחר כך שלא שלח	אליס מכחישה שאישרה עסקה	חתימה דיגיטלית בלבד (פרק 4)
זמינות (Availability)	השירות עובד כשצריך אותו	הצפת שרת בבקשות (DoS)	הגנות רשת (פרק 9)

הרעיון החשוב: הצפנה לבד לא נותנת שלמות

הרבה סטודנטים מניחים ש"אם זה מוצפן, אי אפשר לשנות את זה". זה לא נכון. תוקף יכול לשנות ביטים בהודעה מוצפנת גם בלי להבין אותה, והמקבל יפענח זבל או, גרוע מזה, הודעה ששונתה בצורה מכוונת. לכן לסודיות יש כלי אחד (הצפנה) ולשלמות כלי אחר (MAC או חתימה), וב-TLS משתמשים בשניהם יחד.

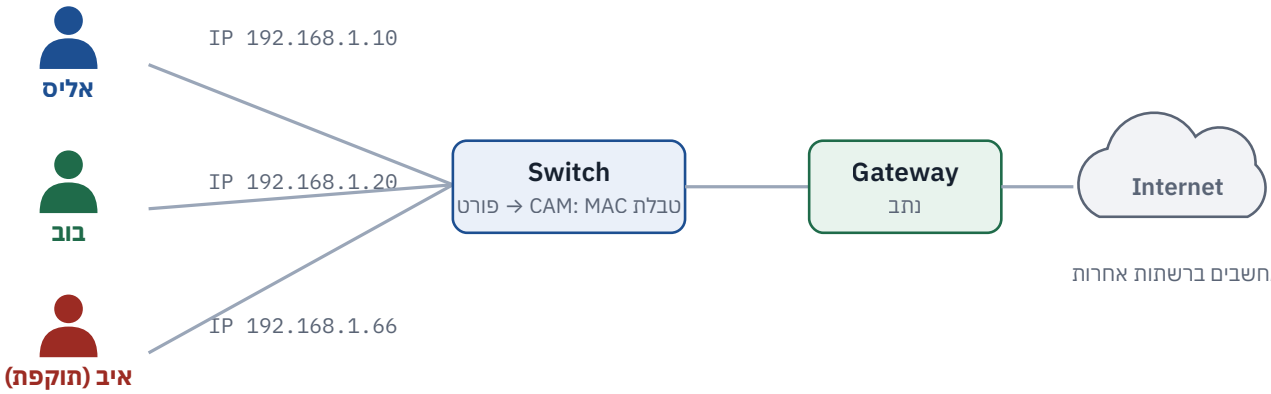
למה אבטחת סייבר קשה

תוקפים מונעים בעיקר מכסף: כופרה (ransomware) כמו WannaCry (2017, תולעת שהתפשטה ברשתות והצפינה קבצים) ו-CryptoLocker, גניבת מידע, סחיטה ופרסום מידע מביך. יש גם מניעים מדינתיים, כמו פריצה לחברת החשמל, ורצון להביך (למשל הפריצה לטכניון). בצד המגן המצב קשה: אין תמריץ כלכלי חזק לכתוב קוד מאובטח, אבטחה יקרה ואיטית, המשתמשים טועים, ופרויקטים נשענים על ספריות חיצוניות שעלולות להכיל חולשות (דוגמאות מההרצאה: BlueBorne בבלוטות', Meltdown ו-Spectre במעבדים). מצד שני יש "האקרים בכובע לבן" שמדווחים על חולשות ומקבלים תגמול (bug bounty).

איך הודעה עוברת ברשת

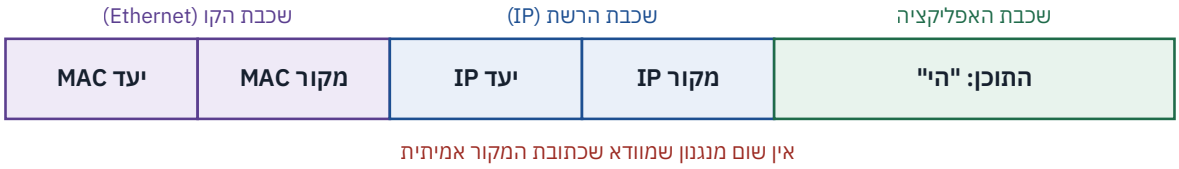
נניח שאליס רוצה לשלוח לבוב את ההודעה "הי". לכל אחד מהם יש כרטיס רשת, ולכל כרטיס שתי כתובות:

- **כתובת MAC** (כתובת פיזית): 48 ביט, כלומר 12 ספרות הקסדצימליות כמו `AA-1B-9C-...`. נצרבת בכרטיס בזמן הייצור ולא תלויה במיקום. בפועל אפשר לשנות אותה בתוכנה, ולכן אסור לסמוך עליה כהוכחת זהות. בין כתובות MAC של מחשבים באותה רשת אין שום קשר.
- **כתובת IP** (כתובת לוגית): תלויה ברשת שאליה מתחברים ומשתנה כשעוברים רשת. למחשבים באותה רשת יש תחילית משותפת, למשל `192.168.1.x`.



רשת מקומית טיפוסית. כל מחשב מחובר רק ל-Switch. כדי לצאת לאינטרנט התעבורה עוברת דרך ה-Gateway.

כל מחשב מחובר רק לרכיב התקשורת (Hub או Switch), לא ישירות למחשבים אחרים. כשאליס רוצה לדבר עם מחשב **מחוץ** לרשת המקומית, היא שולחת את ההודעה ל-**Gateway** (הנתב), והוא מעביר אותה לאינטרנט ומחזיר את התשובה. ההודעה עצמה לא נשלחת לבד: לפניה מוצמדות כתובות המקור והיעד, לפי **פרוטוקול**, כלומר אוסף כללים שמגדיר את מבנה ההודעה ואיך מבקשים ומקבלים מידע.



מה עוד נשלח עם כל הודעה. ה-Switch מסתכל רק על כתובות ה-MAC. ה-IP משמש לניתוב בין רשתות. תוקף יכול לכתוב בשדות המקור כל ערך שירצה (spoofing).

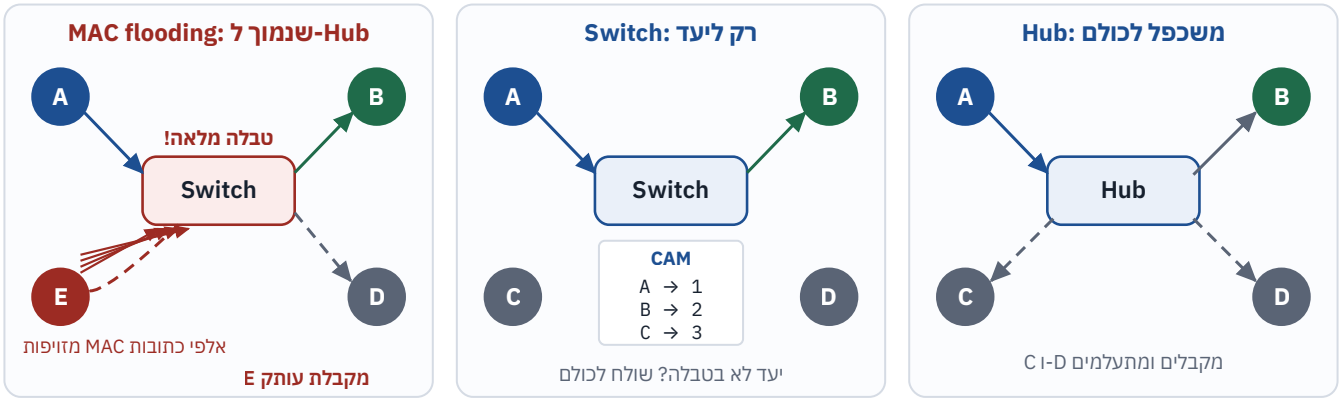
שני פרוטוקולים עוזרים למחשב למצוא את הכתובות האלה, ושניהם יחזרו כמטרות להתקפה בפרק 9:

- **DNS** מתרגם שם (www.amazon.com) לכתובת IP.
- **ARP** מתרגם כתובת IP לכתובת MAC ברשת המקומית: המחשב שולח לכולם (broadcast) "למי יש את ה-IP הזה?", ושומר את התשובה בטבלת ARP (מעין זיכרון מטמון). אם התשובה כבר בטבלה, לא שואלים שוב.

Hub, Switch והתקפת שנמוך

Hub הוא הרכיב הפשוט: כל הודעה שמגיעה אליו משוכפלת לכל הפורטים (broadcast). המחשבים שההודעה לא מיועדת אליהם אמורים להתעלם ממנה, אבל כל מי שמחובר ל-Hub יכול פשוט לא להתעלם, ולקרוא הכל. כלומר ב-Hub כל מחובר הוא מאזין פוטנציאלי.

Switch חכם יותר. הוא מחזיק טבלת **CAM** (Content Addressable Memory) שממפה כל כתובת MAC לפורט הפיזי שמאחוריו היא נמצאת, ושולח כל הודעה רק לפורט של היעד. הוא לומד את הטבלה מכתובות המקור של ההודעות שעוברות דרכו. אם היעד לא מופיע בטבלה, הוא מתנהג כמו Hub ושולח לכולם. כשהטבלה מלאה, הוא מוחק את הרשומה שעודכנה הכי מזמן.



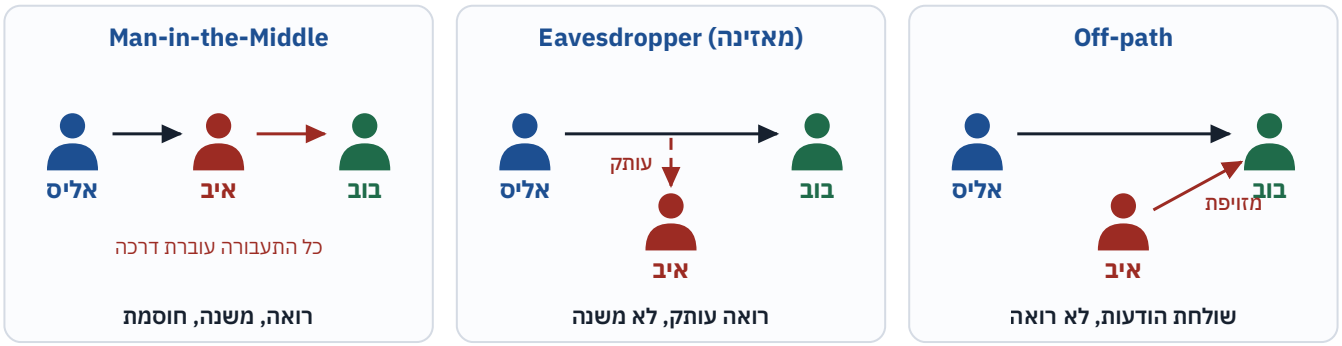
Hub, Switch והצפת MAC. ה-Switch שולח רק ליעד לפי טבלת CAM. כשהטבלה מתמלאת בכתובות מזויפות, הוא חוזר להתנהג כמו Hub, והתוקפת מקבלת עותק של הכל.

MAC flooding: התוקפת מציפה את ה-Switch בהודעות, כל אחת עם כתובת MAC מזויפת אחרת בשדה המקור. ה-Switch מוסיף את כולן לטבלה (כולן ממופות לפורט של התוקפת), הזיכרון מתמלא והרשומות האמיתיות נדחקות החוצה. מעכשיו היעדים "לא בטבלה", וה-Switch שולח לכולם, כמו Hub. זו **התקפת שנמוך** (downgrade): התקפה שמאלצת מערכת לחזור לגרסה ישנה ופחות מאובטחת של עצמה. אותו רעיון יחזור ב-TLS (פרק 6).

ההגנה: port security, הגבלה של מספר כתובות ה-MAC שמותר לראות בכל פורט. לכן הטענה "אי אפשר למנוע שנמוך של Switch" שגויה. חשוב גם להבין ש-Switch לבד **לא** מונע מתוקף להאזין, לחסום או לשנות: הוא רק מקשה.

שלושת מודלי התוקף

כשהמבחן שואל אם התקפה אפשרית, הוא תמיד שואל "מול איזה תוקף". שלושת המודלים מסודרים לפי כוח, וכל אחד כולל את היכולות של הקודם:



שלושת מודלי התוקף, מהחלש לחזק. כל מודל יכול את כל מה שהקודם יכול, ועוד.

מודל	מה יכול	מה לא יכול	איך מגיעים למצב הזה
Off-path	לשלוח הודעות משלו ולכתוב בהן כל כתובת מקור ויעד (IP/MAC spoofing). בוב לא יודע להבחין אם ההודעה באמת מאליס	לראות את התעבורה בין הצדדים, לשנות או לחסום אותה	כל מחשב שמחובר לאינטרנט
Eavesdropper (מאזין פסיבי)	כל מה ש-off-path יכול, ובנוסף מקבל עותק של כל הודעה. אם היא לא מוצפנת, קורא אותה	לשנות, לחסום או לעכב הודעות	מתחבר ל-Hub, רשת Wi-Fi פתוחה, MAC flooding
MitM (Man-in-the-Middle)	הכל: לקרוא, לשנות, לחסום, לעכב, להזריק, ולהתחזות לכל צד מול השני. הצדדים חושבים שהם מדברים ישירות	לשבור קריפטוגרפיה טובה	הרעלת ARP (פרק 9), שליטה בנתב

> **במבחן**
> כמעט כל שאלה על התקפה שואלת "איזה תוקף נדרש". כלל אצבע: ככל שהתוקף צריך **לראות** את התשובה האמיתית או **לחסום** אותה, הוא צריך יכולת גבוהה יותר. הרעלת ARP אפשרית כבר ל-off-path (מספיק לשלוח הודעה מזויפת), והיא **הופכת** אותו ל-MitM. הרעלת DNS לא מחייבת MitM: off-path שמנחש נכון מספיק.

איך מזהים תוקף ברשת

תוכנת **sniffing** כמו Wireshark מציגה את כל התעבורה שנכנסת ויוצאת דרך כרטיס הרשת. סימנים חשודים: הרבה כתובות IP שממופות לאותה כתובת MAC (סימן להרעלת ARP), או הצפה של כתובות MAC חדשות. בנוסף אפשר להתקין כלים ייעודיים על ה-Switch שמזהים וחוסמים תוקפים, וכלים על המחשבים עצמם.

עקרון קרקהוף (Kerckhoffs)

מערכת קריפטוגרפית צריכה להישאר בטוחה גם כשכל הפרטים שלה גלויים, **חוץ מהמפתח**. הסיבה מעשית: אלגוריתם קשה להחליף, ואם הבטיחות תלויה בסודיות שלו, ברגע שהוא דולף (וזה תמיד קורה) הכל נשבר. מפתח מחליפים בקלות. לכן עדיף להשתמש באלגוריתם ידוע שמומחים ניסו לשבור במשך שנים, ולא להמציא צופן "סודי" (security by obscurity), שכמעט בטוח שבור בלי שנדע.

סיכום הפרק

- חמש תכונות: סודיות (הצפנה), שלמות (Hash/MAC/חתימה), אותנטיות (MAC/חתימה), אי-הכחשה (רק חתימה), זמינות.
- MAC = כתובת פיזית, ניתנת לזיוף. IP = כתובת לוגית לפי רשת. DNS. IP→MAC: ARP: שם→IP.
- Hub שולח לכולם; Switch שולח לפי טבלת CAM; MAC flooding משנמך Switch ל-Hub; הגנה: port security.
- Off-path שולח בלבד; מאזין גם רואה; MitM גם משנה וחוסם.
- קרקהוף: רק המפתח סודי.

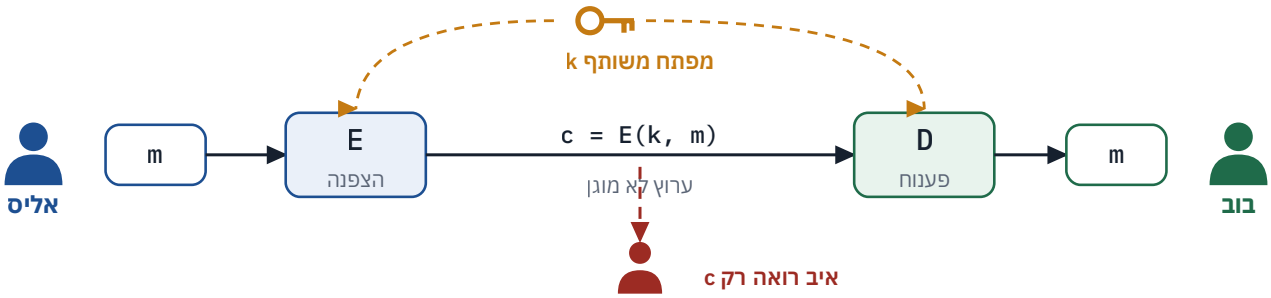
2 הצפנה סימטרית וצפנים קלאסיים

הצפנה פותרת בעיה אחת: **סודיות**. אליס רוצה לשלוח לבוב הודעה כך שמאזין שרואה את כל מה שעובר בערוץ לא ילמד ממנה כלום. בפרק הזה נלמד את ההצפנה הסימטרית, שבה לשני הצדדים יש אותו מפתח, מהצפנים ההיסטוריים ועד AES, ונבין למה כל צופן ישן נשבר.

המושגים הבסיסיים

- **טקסט גלוי** (plaintext) `m`: ההודעה המקורית.
- **טקסט מוצפן** (ciphertext) `c`: מה שעובר בערוץ. הוא לא אמור לגלות שום דבר על m.
- **מפתח** `k` מתוך **מרחב המפתחות** `K`, כלומר כל המפתחות האפשריים.
- **פונקציית הצפנה** `E: K × M → C` ו**פונקציית פענוח** `D: K × C → M`. יחד הן נקראות **סכמת הצפנה**.
- **דרישת הנכונות**: $D_k(E_k(m)) = m$. כלומר, מי שיש לו את המפתח תמיד מקבל בחזרה את ההודעה המקורית.

ההצפנה נקראת **סימטרית** כי **אותו מפתח** משמש להצפנה ולפענוח. שמות נוספים: shared-key encryption (המפתח משותף לשולח ולמקבל) ו-private-key encryption (רק מי שצריך יודע אותו).



הצפנה סימטרית. אותו מפתח k משמש להצפנה ולפענוח. איב רואה רק את c. השאלה הפתוחה: איך אליס ובוב הסכימו על k בלי שאיב ראתה אותו?

מרחב מפתחות קטן = צופן לא בטוח. אם אפשר לעבור על כל המפתחות בזמן סביר, התוקף פשוט מנסה את כולם עד שהפענוח נראה הגיוני. זו התקפת **brute force**, שנקראת גם **חיפוש ממצה** (exhaustive search). לכן הדרישה הראשונה מכל צופן היא מרחב מפתחות עצום. אבל, כמו שנראה מיד, זה תנאי הכרחי ולא מספיק.

צופן הזזה וצופן קיסר

מספרים את האותיות `A=0, B=1, …, Z=25`, ומזיזים כל אות במספר קבוע של מקומות. בסוף האלפבית חוזרים להתחלה, ולכן החישוב הוא מודולו 26:

$$E_k(m)[i] = (m[i] + k) \bmod 26 \;;\; D_k(c)[i] = (c[i] - k) \bmod 26$$

צופן קיסר הוא המקרה הפרטי `k = 3`. ההודעה `hello` מוצפנת ל-`khoor`.

m	A	B	C	D	**E**	F	G	**H**	I	J	K	**L**	M	N	**O**	P	Q	R	S	T	U	V	W	X	Y	Z
c	D	E	F	G	**H**	I	J	**K**	L	M	N	**O**	P	Q	**R**	S	T	U	V	W	X	Y	Z	A	B	C

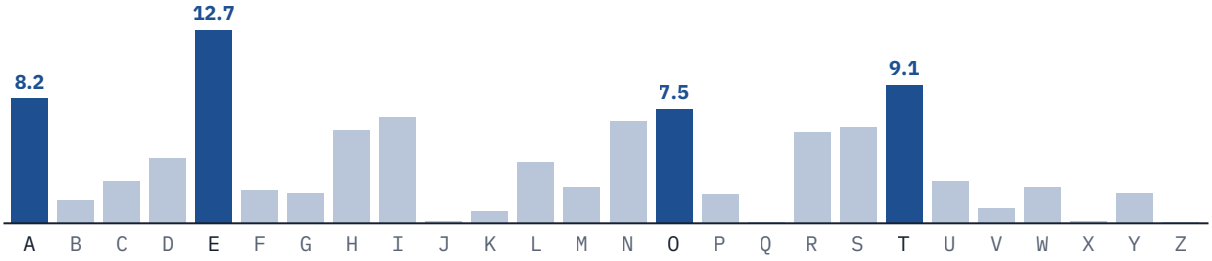
`X→A, Y→B, Z→C`

צופן קיסר: הזזה ב-3. כל אות מוחלפת באות שנמצאת 3 מקומות אחריה. בסוף האלפבית חוזרים להתחלה (mod 26). HELLO הופך ל-KHOOR.

בקיסר "המפתח" הוא השיטה עצמה: מי שיודע שזה קיסר, פענח. זה בדיוק מה שעקרון קרקהוף אוסר (פרק 1). בצופן הזזה כללי יש **25 מפתחות** שימושיים (הזזה ב-0 לא מצפינה), ולכן brute force מסיים אחרי 25 ניסיונות לכל היותר.

צופן החלפה (Monoalphabetic Substitution)

במקום הזזה קבועה, כל אות ממופה לאות אחרת לפי טבלה שרירותית (תמורה של האלפבית). המפתח הוא הטבלה. מספר הטבלות האפשריות הוא $26! \approx 4 \cdot 10^{26}$, הרבה מעבר ל-brute force. ובכל זאת הצופן נשבר בקלות, כי הוא שומר על **מבנה השפה**: כל E בטקסט הופכת תמיד לאותה אות.



שכיחות האותיות בטקסט אנגלי רגיל (באחוזים, בקירוב). בצופן החלפה כל E הופכת תמיד לאותה אות מוצפנת, ולכן האות השכיחה ביותר ב-ciphertext היא כנראה E, השנייה כנראה T, וכן הלאה. כך שוברים מרחב של $26!$ מפתחות בלי לנסות אותם.

ניתוח שכיחויות: סופרים כמה פעמים מופיעה כל אות ב-ciphertext. אם Q היא השכיחה ביותר, כנראה Q מחליפה את E. ממשיכים עם T, A, O ועם צירופים שכיחים (th, he), ותוך דקות מפענחים. המסקנה: **מרחב מפתחות גדול לא מספיק** אם הצופן מדליף דפוסים.

צופן Vigenère

הרעיון: לשבור את השכיחויות. בוחרים מפתח קצר (מילה), משרשרים אותו לעצמו עד שהוא באורך הטקסט, ומזיזים כל אות לפי אות המפתח שמעליה: `c[i] = (m[i] + k[i mod |k|]) mod 26`. כך אותה אות בטקסט יוצאת שונה במקומות שונים.

טקסט גלוי m	A	T	T	A	C	K	A	T	D	A	W	N
מפתח k	L	E	M	O	N	L	E	M	O	N	L	E
מוצפן c	L	X	F	O	P	V	E	F	R	N	H	R

אותיות 1, 6, 11 מוזזות כולן ב-11 (=L): קבוצה אחת לניתוח שכיחויות

A+L=L, T+E=X

Vigenère: מצפינים ATTACKATDAWN עם המפתח LEMON. משכפלים את המפתח לאורך הטקסט ומחברים מודולו 26 (A=0). אותה אות (A או T) יוצאת שונה בכל פעם, ולכן ניתוח שכיחויות רגיל לא עובד. אבל כל חמישית אות מוזזת באותה הזזה, ושם אפשר לתקוף.

איך שוברים בכל זאת, בשני שלבים:

1. **מוצאים את אורך המפתח.** כשאותו רצף אותיות בטקסט נופל על אותו מקום במפתח, הוא מוצפן לאותו רצף ב-ciphertext. לכן מחפשים רצפים שחוזרים ב-ciphertext (בהרצאה: elq ,fvr) ומודדים את המרחקים ביניהם. המרחקים הם כפולות של אורך המפתח, והמחלק המשותף שלהם הוא המועמד לאורך.
2. **מוצאים כל אות במפתח בנפרד.** אם האורך הוא 5, האותיות במקומות 1, 6, 11, ... מוזזות כולן באותה הזזה. כל קבוצה כזו היא בעצם צופן קיסר, ושם ניתוח שכיחויות עובד: האות השכיחה בקבוצה היא כנראה E, וההפרש בינה לבין E הוא אות המפתח.

מרחב המפתחות של Vigenère הוא $26^{|k|}$: עם מפתח באורך 5 יש כ-11.9 מיליון מפתחות.

במבחן

- קיסר על QUIZ: כל אות +3, כלומר Q→T, U→X, I→L, Z→C, ולכן TXLC. לא לשכוח את הגלישה Z→C.
- צופן החלפה על אלפבית של 28 תווים, ו-ciphertext של 7 תווים זהים: מיפוי הוא חד-חד-ערכי, ולכן כל ה-plaintext חייב להיות אותה אות 7 פעמים. יש **28** אפשרויות.
- בצופן החלפה התוקף יודע ש-X מחליף את Y: נשארו 25 אותיות לסדר, כלומר **$25!$** מפתחות.
- Vigenère עם מפתח CODE (אורך 4) ו-plaintext באורך 5: תווים 1 ו-5 מוצפנים עם אותה אות מפתח, לכן הם זהים ב-ciphertext רק אם הם זהים ב-plaintext. ההסתברות לכך: **1/26**.

(OTP) One-Time Pad

עכשיו עוברים מאותיות לביטים. הפעולה המרכזית היא **XOR** (⊕): התוצאה 1 כשהביטים שונים ו-0 כשהם שווים.

a	b	a ⊕ b
0	0	0
0	1	1
1	0	1
1	1	0

שלוש תכונות שמשמשות בכל הוכחה:

$$K \oplus K = 0 \;;\; A \oplus 0 = A \;;\; A \oplus B \oplus B = A$$

XOR גם קומוטטיבי ואסוציאטיבי, כך שאפשר לסדר את האיברים בכל סדר.

ההגדרה: $E_k(m) = m \oplus k$, $D_k(c) = c \oplus k$. ההוכחה שהפענוח עובד:

$$D_k(E_k(m)) = (m \oplus k) \oplus k = m \oplus (k \oplus k) = m \oplus 0 = m$$

הצפנה ופענוח עם XOR

m	1	0	1	1	0	1	1	0
k	0	1	1	0	1	1	0	1
$c = m \oplus k$	1	1	0	1	1	0	1	1
$c \oplus k$	1	0	1	1	0	1	1	0

כי $k \oplus k = 0$ — $= m$ ✓

Two-Time Pad: פעמיים k אותו

$$c_1 = m_1 \oplus k$$
$$c_2 = m_2 \oplus k$$
$$c_1 \oplus c_2 = m_1 \oplus m_2$$

המפתח נעלם. מנחשים מילה שכיחה כמו the ב-m_1 ומקבלים את החלק המתאים ב-m_2

One-Time Pad ומה קורה כשמשתמשים במפתח פעמיים. מימין: XOR של ההודעה עם מפתח אקראי, ופעם נוספת עם אותו מפתח מחזיר את ההודעה. משמאל: עם אותו k פעמיים, ה-XOR של שני המוצפנים מבטל את המפתח.

שני תנאים: המפתח אקראי ובאורך ההודעה לפחות ($|k| \geq |m|$), ומשתמשים בו **פעם אחת בלבד**. בתנאים האלה ל-OTP יש **סודיות מושלמת** (perfect secrecy): כל plaintext באותו אורך אפשרי באותה מידה, כי לכל m יש מפתח שמוביל ל-c הנתון. אין דפוס ואין שכיחויות לנצל, והבטיחות **לא** תלויה בתוכן הטקסט.

החיסרון המעשי: העברת המפתח. כדי לשלוח סרטון של 20GB צריך קודם להעביר לבוב מפתח אקראי של 20GB בדרך מאובטחת. ואם יש דרך מאובטחת כזו, למה לא לשלוח בה את הסרטון עצמו? לכן הטענה "בעיה גדולה ב-OTP היא העברת המפתחות" **נכונה**.

Two-Time Pad: אם משתמשים באותו מפתח פעמיים, ה-XOR של שני המוצפנים מבטל את המפתח: $c_1 \oplus c_2 = m_1 \oplus m_2$. התוקף מנחש מילה שכיחה (the) במקום כלשהו ב-m_1, עושה XOR עם $c_1 \oplus c_2$ ובודק אם יצא משהו הגיוני ב-m_2 (בהרצאה יצא and). כך הוא משחזר את שני הטקסטים חלק אחר חלק.

מלכודת

פרוטוקול "שלוש העברות עם XOR": אליס שולחת $K_1 = K \oplus K_A$, בוב מחזיר $K_2 = K_1 \oplus K_B$, אליס שולחת $K_3 = K_2 \oplus K_A$. נראה בטוח (כל ערך "מוצפן" במפתח אקראי), אבל מאזין שרואה את שלושתם מחשב $K = K_1 \oplus K_2 \oplus K_3$, כי כל K_A ו-K_B מופיעים פעמיים ומתבטלים. הפרוטוקול **לא מוגן אפילו מפני מאזין**.

צפני זרם: OTP עם מפתח קצר

PRG (Pseudo-Random Generator) הוא אלגוריתם שמקבל מחרוזת אקראית קצרה (seed) ומייצר ממנה רצף ארוך שנראה אקראי. הרצף לא באמת אקראי, אבל אם ה-PRG טוב, אי אפשר להבחין בהבדל. **Stream cipher** משתמש בפלט ה-PRG במקום מפתח ה-OTP:

$$E_k(m) = m \oplus PRG(k) \;;\; D_k(c) = c \oplus PRG(k)$$



צופן זרם. מפתח קצר נכנס ל-PRG, שמייצר רצף ארוך שנראה אקראי (keystream). ה-keystream משמש כמו מפתח OTP. אותו רעיון, אבל המפתח שצריך להעביר קצר.

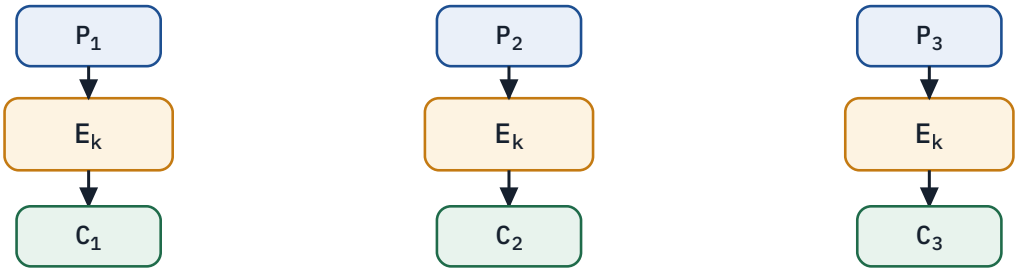
היתרון: מהיר מאוד (דוגמה: RC4), ומפתח של 128 ביט מספיק להודעה בכל אורך. האזהרה מ-Two-Time Pad נשארת: אסור להצפין שתי הודעות עם אותו keystream. והשאלה "איך מעבירים את המפתח" עדיין פתוחה. התשובה בפרק 4.

צפני בלוקים ומצבי הפעלה

Block cipher מצפין בלוק בגודל קבוע בבת אחת: DES עובד על 64 ביט (8 בתים), AES על 128 ביט. בפנים זה לא XOR פשוט אלא סבבים רבים של החלפות ותמורות. כדי להצפין הודעה ארוכה חותכים אותה לבלוקים, ואם הבלוק האחרון קצר מדי, משלימים אותו ב**ריפוד** (padding) שמוסר בפענוח. הדרך שבה "מחברים" את הבלוקים נקראת **mode of operation**. לכן גם כשמשתמשים ב-AES צריך להגדיר mode.

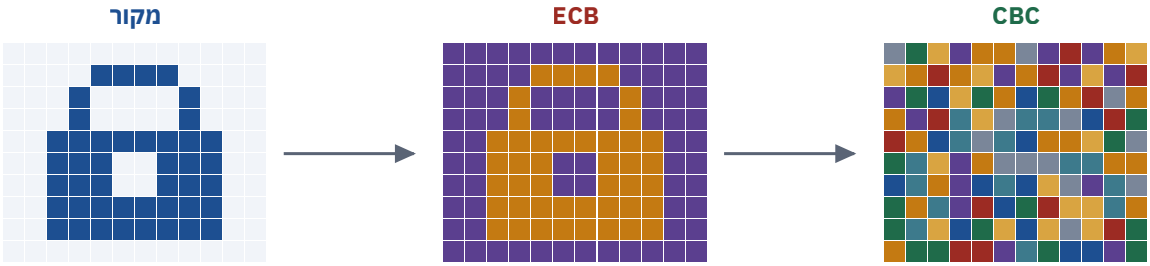
(Electronic Code Book) ECB

הדרך הנאיבית: כל בלוק מוצפן לבד, עם אותו מפתח, כאילו אין בלוקים אחרים.



ECB: כל בלוק מוצפן לבד, עם אותו מפתח. $C_i = E_k(P_i)$. אם $P_1 = P_3$ אז גם $C_1 = C_3$, ותוקף רואה שהבלוקים חוזרים.

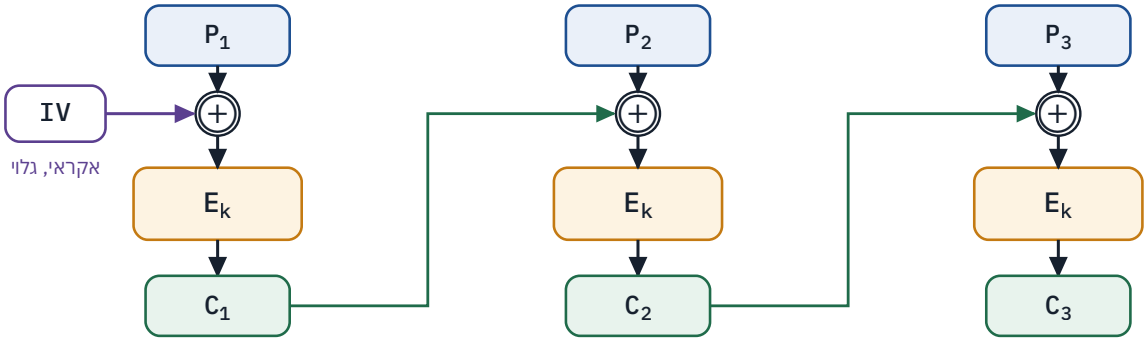
הבעיה: הצפנה דטרמיניסטית. שני בלוקים זהים ב-plaintext נותנים שני בלוקים זהים ב-ciphertext. בהרצאה, 16 תווים זהים יצרו שני חצאים זהים במוצפן. כל דפוס כזה עוזר לתוקף להתחיל לנחש את התוכן. בתמונה זה בולט במיוחד:



למה ECB מסוכן: הדפוס עובר להצפנה. כל משבצת מייצגת בלוק. ב-ECB כל בלוק "לבן" מוצפן לאותו ערך, כך שהצורה נשארת גלויה (רק הצבעים מתחלפים). ב-CBC כל בלוק יוצא שונה, והתוצאה נראית כמו רעש.

(Cipher Block Chaining) CBC

לפני שמצפינים בלוק, עושים לו XOR עם **הבלוק המוצפן הקודם**. לבלוק הראשון אין בלוק קודם, ולכן מגרילים **IV** (Initialization Vector) בגודל בלוק. ה-IV **אקראי אבל לא סודי**: הוא נשלח יחד עם ההודעה.



CBC: לפני ההצפנה עושים XOR עם הבלוק המוצפן הקודם. $C_i = E_k(C_{i-1} \oplus P_i)$, ולבלוק הראשון משתמשים ב-IV. גם בלוקים זהים יוצאים שונים.

עכשיו גם אם $P_1 = P_2$, הקלט לפונקציית ההצפנה שונה (כל אחד עבר XOR עם משהו אחר), ולכן $C_1 \neq C_2$. וגם הצפנה של אותה הודעה פעמיים, עם IV אחר, נותנת תוצאה שונה. שימו לב: CBC נותן **סודיות**, לא שלמות. תוקף עדיין יכול לשנות ביטים במוצפן, ולשם כך צריך MAC (פרק 3).

Mode	נוסחה	בלוקים זהים	צריך IV?
ECB	$C_i = E_k(P_i)$	יוצאים זהים: דליפת דפוסים	לא
CBC	$C_i = E_k(C_{i-1} \oplus P_i),\ C_0 = IV$	יוצאים שונים	כן, אקראי וגלוי

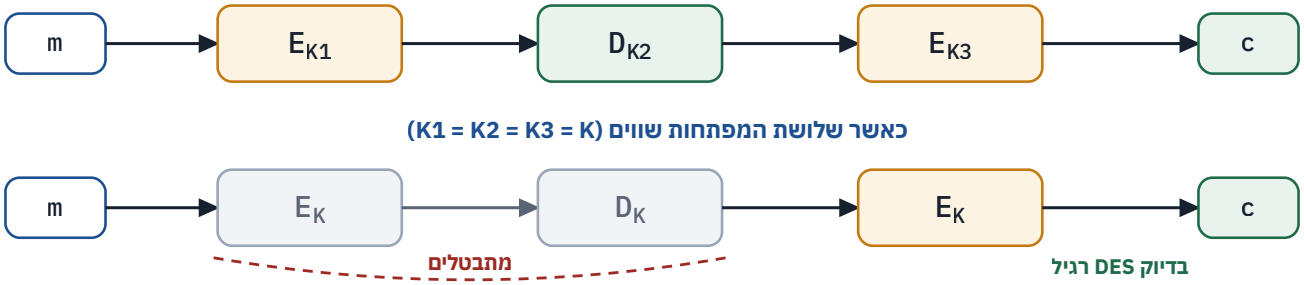
במבחן

הצפנת ABCDEFGHABCDEFGH ב-DES-ECB: 16 תווים = שני בלוקים של 8 בתים, ושניהם ABCDEFGH. לכן ה-ciphertext הוא אותו חצי פעמיים: `437d7e60707604f3 437d7e60707604f3`. בלי לחשב כלום: מחפשים בתשובות את המחרוזת שבה החצי הראשון שווה לשני. ב-CBC התשובה הייתה **שונה** בין החצאים.

DES, 3DES ו-AES

DES (Data Encryption Standard) הוא צופן הבלוקים הסימטרי שתוקנן בארה"ב בשנות ה-70: בלוק של 64 ביט ומפתח של **56 ביט בלבד**. "סטנדרט" פירושו הגדרה מדויקת של איך ההצפנה עובדת, כך שכל מימוש יכול לדבר עם כל מימוש אחר. היום 56 ביט חלשים מדי: $2^{56} \approx 7 \cdot 10^{16}$ מפתחות נבדקים בימים.

3DES מחזק את DES בלי להחליף אותו: מריצים DES שלוש פעמים עם שלושה מפתחות. הבעיה הייתה תאימות לאחור, כי אי אפשר להגיד לכל העולם להפסיק להשתמש ב-DES ביום אחד. הפתרון: מבנה **EDE** (הצפנה, פענוח, הצפנה):



3DES במבנה EDE. הצפנה, פענוח, הצפנה. כששלושת המפתחות שווים, הפענוח האמצעי מבטל את ההצפנה הראשונה ונשארת הצפנת DES אחת. כך מחשב 3DES יכול לדבר עם מחשב DES ישן.

- מחשב 3DES שרוצה לדבר עם מחשב DES משתמש באותו מפתח שלוש פעמים. הפענוח השני מבטל את ההצפנה הראשונה, ונשארת הצפנת DES רגילה. בפענוח: פענוח, הצפנה, פענוח, וההצפנה האמצעית מבטלת את הפענוח הראשון.
- עם מפתחות שונים, EEE ו-EDE נותנים פלט **שונה**. הטענה "אין הבדל" שגויה.

AES הוא הסטנדרט המודרני: בלוק של 128 ביט ומפתח של 128, 192 או 256 ביט. ב-2^{128} מפתחות, brute force לא אפשרי בשום חומרה קיימת.

מי סימטרי ומי לא

סימטריים: DES, 3DES, AES, RC4. אסימטריים: RSA, Diffie-Hellman (פרק 4). SHA-256 הוא **hash**, לא הצפנה: אין לו מפתח ואין פענוח.

סיכום הפרק

- סימטרי = אותו k להצפנה ולפענוח. נכונות: $D_k(E_k(m)) = m$.
- הזזה: 25 מפתחות, brute force. החלפה: $26!$ מפתחות, נשבר בשכיחויות. Vigenère: אורך מפתח לפי מרחקים בין חזרות, ואז שכיחויות לכל עמודה.
- OTP: $m \oplus k$, סודיות מושלמת, מפתח באורך ההודעה ופעם אחת. Two-time pad: $c_1 \oplus c_2 = m_1 \oplus m_2$.
- Stream cipher: $m \oplus PRG(k)$. Block cipher צריך mode: ECB מדליף דפוסים, CBC משרשר עם IV אקראי וגלוי.
- DES: 64/56 ביט. 3DES EDE: מפתחות שווים = 128 ביט. AES.

3 פונקציות Hash, סיסמאות ו-MAC

הצפנה מסתירה תוכן, אבל לא מגלה אם מישהו שינה אותו. הפרק הזה עוסק ב**שלמות** ו**אותנטיות**: פונקציית Hash נותנת "טביעת אצבע" קצרה לכל קובץ, שמירת סיסמאות נשענת עליה, ו-MAC מוסיף מפתח סודי כדי שרק מי שמחזיק אותו יוכל לייצר טביעת אצבע תקינה.

מה זו פונקציית Hash

פונקציית גיבוב (hash function) h מקבלת קלט **בכל אורך** (מערך בתים, קובץ שלם) ומחזירה פלט **באורך קבוע**, שנקרא digest. אין לה מפתח, וכל אחד יכול לחשב אותה. SHA-256 (ממשפחת Secure Hash Algorithm) מחזירה תמיד 256 ביט. שתי תכונות שרואים מיד:

- **דטרמיניסטית**: אותו קלט תמיד נותן אותו פלט.
- **אפקט המפולת**: שינוי של ביט אחד בקלט משנה בערך חצי מהביטים בפלט, בלי שום קשר נראה לעין.

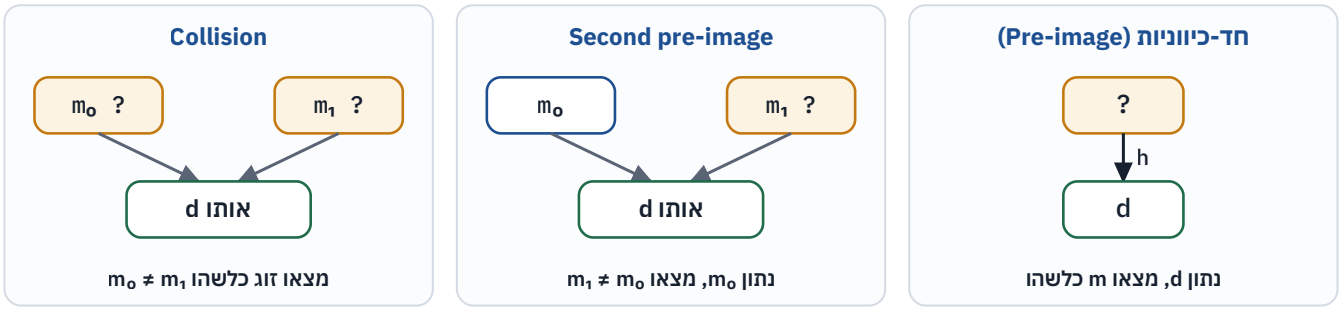


פונקציית Hash. קלט בכל אורך נכנס, ויוצא digest באורך קבוע (ב-SHA-256: 256 ביט = 64 ספרות הקסדצימליות). שינוי של אות אחת (H גדולה מול h קטנה) משנה את כל הפלט. אלה ערכי SHA-256 אמיתיים (מקוצרים).

דוגמה מההרצאה: מורידים קובץ ISO של Ubuntu. מריצים עליו SHA-256 ומשווים לערך שמפורסם באתר. אם תוקף MitM החליף את הקובץ בגרסה זדונית, ה-hash יהיה שונה לגמרי. שימו לב לתנאי: ה-hash שמשווים אליו צריך להגיע ממקור אמין. אם התוקף מחליף גם את הקובץ וגם את ה-hash המפורסם, הבדיקה חסרת ערך. Hash לבד לא נותן אותנטיות, וזו הסיבה שצריך MAC או חתימה.

שלוש התכונות הקריפטוגרפיות

מספר הקלטים האפשריים אינסופי ומספר הפלטים סופי, ולכן **בהכרח** יש התנגשויות (שני קלטים עם אותו פלט). הדרישה היא שיהיה **קשה עד בלתי אפשרי למצוא** אותן:



שלוש התכונות של Hash. מה נתון ומה התוקף צריך למצוא. בצהוב: מה שהתוקף בוחר בעצמו.

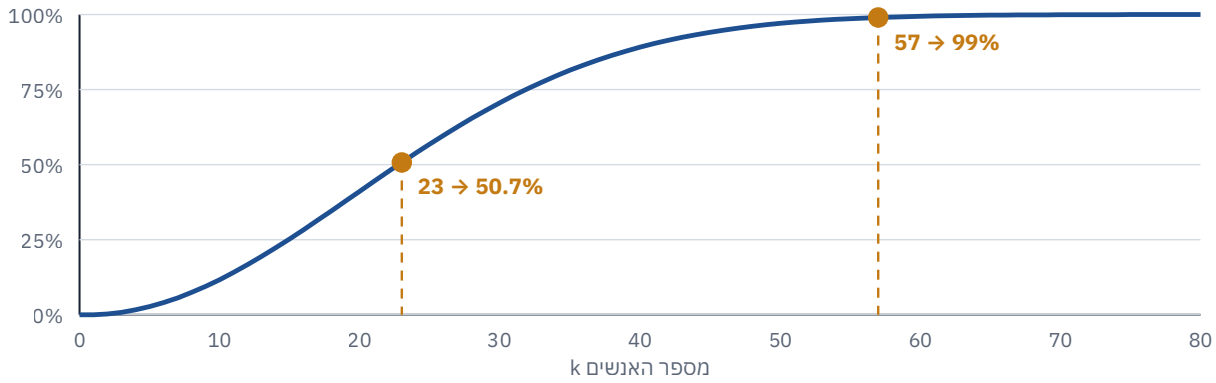
תכונה	הגדרה (כך היא מופיעה במבחן)	למה צריך אותה
Pre-image resistance (חד-כיווניות)	בהינתן digest d של הודעה אקראית, קשה למצוא m כך ש-$h(m) = d$	שמירת סיסמאות: מה-hash אי אפשר לחזור לסיסמה
Second pre-image resistance	בהינתן הודעה אקראית m_0, קשה למצוא $m_1 \neq m_0$ כך ש-$h(m_0) = h(m_1)$	בדיקת קובץ שהורד: תוקף לא יכול לייצר קובץ אחר עם אותו hash
Collision resistance	קשה למצוא **שתי** הודעות כלשהן $m_0 \neq m_1$ כך ש-$h(m_0) = h(m_1)$	חתימות: תוקף לא יכול להכין מראש שני חוזים עם אותו hash, להחתים על התמים ולהציג את השני

מלכודת

ההבדל בין השתיים האחרונות: ב-second pre-image ההודעה הראשונה **נתונה** (התוקף לא בחר אותה), וב-collision התוקף בוחר את **שתיהן**. לכן collision היא המשימה הקלה יותר לתוקף, ועמידות להתנגשויות היא הדרישה החזקה ביותר. תשובה עם "$h(m_0) \neq h(m_1)$" או "קשה למצוא 'd digest שונה מ-d" היא תמיד מסיח שגוי.

פרדוקס יום ההולדת

כמה אנשים צריך בחדר כדי שבסיכוי של 50% לשניים מהם יהיה אותו יום הולדת? האינטואיציה אומרת כ-183 (חצי שנה). התשובה: **23**. הסיבה היא שלא מחפשים מישהו עם יום ההולדת **שלי**, אלא **כל זוג**. עם 23 אנשים יש $23 \cdot 22/2 = 253$ זוגות, וכל אחד מהם הוא הזדמנות להתנגשות.



פרדוקס יום ההולדת (N = 365). ההסתברות שלפחות לשניים מתוך k אנשים יש אותו יום הולדת. עם 23 אנשים כבר עוברים 50%, ועם 57 כמעט ודאי. הסיבה: מספר הזוגות גדל כמו k^2, לא כמו k.

באופן כללי, אם יש N ערכים אפשריים ובוחרים k ערכים אקראיים:

$$P(\text{collision}) \approx 1 - e^{-k^2/2N} \quad ; \quad k \approx \sqrt{N} \Rightarrow \text{about } 40\% \quad ; \quad k \approx 1.18 \cdot \sqrt{N} \Rightarrow 50\%$$

כלל האצבע של הקורס: **כ-$\sqrt{N}$ ניסיונות** נותנים סיכוי של כ-50% להתנגשות. המשמעות ל-hash: אם הפלט באורך n ביט, יש $N = 2^n$ ערכים, ומציאת התנגשות עולה רק כ-$2^{n/2} = \sqrt{2^n}$ ניסיונות. לכן SHA-256 נותן "רק" 128 ביט של עמידות להתנגשויות, ולכן hash של 128 ביט נחשב חלש. זה עיקרון כללי, ולא ייחודי להתקפת קמינסקי על DNS (משמש גם שם, פרק 9).

שמירת סיסמאות

אתר צריך לבדוק שהסיסמה שהמשתמש הקליד נכונה, אבל **לא** צריך לדעת אותה. לכן שומרים $h(\text{password})$ במקום הסיסמה. בכניסה מחשבים hash על מה שהוקלד ומשווים. אם ה-DB נגנב, חד-כיווניות מונעת מהתוקף לחשב את הסיסמאות ישירות.

אבל זה לא מספיק. משתמשים בוחרים סיסמאות נפוצות (123456, qwerty). תוקף שגנב את הטבלה:

- רואה מיד אילו משתמשים חולקים אותה סיסמה (אותו hash).
- מחשב **פעם אחת** hash למיליון הסיסמאות הנפוצות ושומר טבלת חיפוש password → hash. עכשיו כל שורה בטבלה הגנובה היא חיפוש אחד. זו התקפת **מילון** (סוג של brute force).

סיסמה גלויה

user	password
alice	123456
bob	123456
carol	qwerty

בלבד hash

user	h(password)
alice	8d969eef6eca…
bob	8d969eef6eca…
carol	65e84be33532…

אליס ובוב: אותו hash

salt + hash

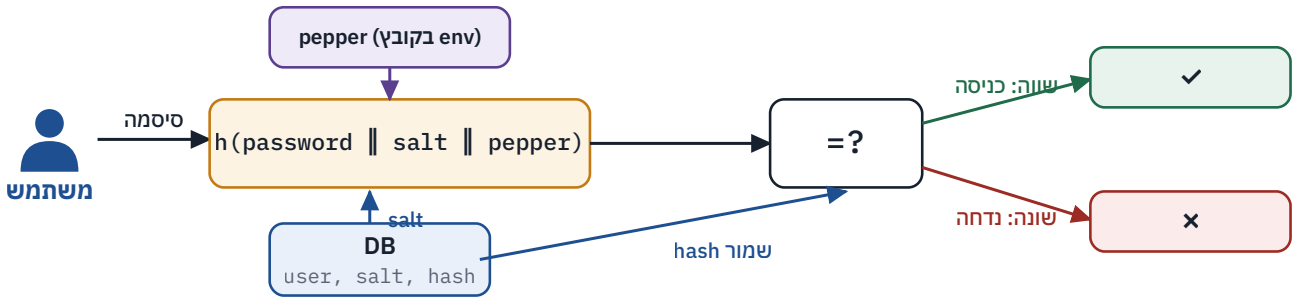
user	salt	h(password‖salt)
alice	x7Qp	088a85e3a994…
bob	Lm2a	005618c01608…
carol	Zr8u	d3457f88f541…

אותה סיסמה, hash שונה

שלוש דרכים לשמור סיסמאות. בטבלה האמצעית רואים מיד שלאליס ולבוב אותה סיסמה, ומילון מוכן של hash-ים חושף אותה. עם salt אקראי לכל משתמש, ה-hash-ים שונים ומילון מוכן מראש כבר לא עוזר.

שלוש תוספות שמקשות על התוקף

- **Salt**: מחרוזת אקראית וארוכה, **שונה לכל משתמש**, שנשמרת **גלויה** ב-DB ליד המשתמש. שומרים `username, salt, h(password || salt)`. שני משתמשים עם אותה סיסמה מקבלים hash שונה. וחשוב יותר: טבלת מילון מוכנה מראש כבר לא עובדת, כי היא הייתה צריכה להיבנות לכל salt אפשרי. התוקף נאלץ לעשות brute force **נפרד לכל משתמש**. salt **לא** מאט ניחוש של סיסמה בודדת.
- **Pepper**: מחרוזת סודית שמוסיפים ל-hash ו**לא נשמרת ב-DB**: `h(password || salt || pepper)`. שתי אפשרויות מההרצאה: (1) pepper אחד (או כמה) לכל המשתמשים, שנשמר בקובץ סביבה (env) או בזיכרון אחר בשרת. תוקף שגנב רק את ה-DB חסר רכיב. (2) pepper קצר לכל משתמש, שלא נשמר בכלל. בכניסה השרת מנסה את כל הערכים האפשריים, ולכן הוא חייב להיות קצר, אבל מספיק כדי להכפיל את עבודת התוקף.
- **Key stretching**: מפעילים את ה-hash `i` פעמים ברצף, כל פעם על הפלט הקודם. למשתמש לגיטימי זה עוד מילישניות בכניסה; לתוקף זה מכפיל פי i את עלות **כל** ניחוש. זו התוספת החזקה ביותר מבין האפשרויות שבמבחן.



בדיקת סיסמה בכניסה, עם salt ו-pepper. ה-salt נשמר גלוי ב-DB ליד המשתמש. ה-pepper סודי ונשמר מחוץ ל-DB, כך שתוקף שגנב רק את ה-DB חסר רכיב.

במבחן

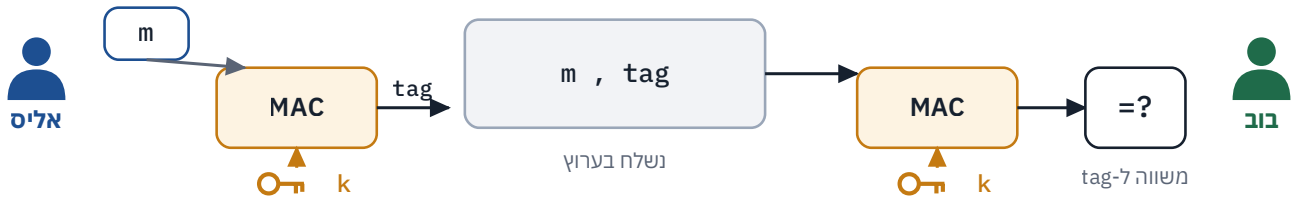
- תוקף גנב קובץ hash-ים של סיסמאות: הוא יבצע **brute force** (מילון).
- `Username, hash(salt), hash(password || salt)`: **לא פרקטי**, כי ה-salt עצמו לא נשמר (רק ה-hash שלו), והשרת לא יכול לחשב את ה-hash בכניסה.
- `username, salt, h(password) || h(salt || pepper)`: **שימוש שגוי**. החלק של הסיסמה נשאר `h(password)` בלי salt, והחלק השני קבוע. מילון עובד.
- משתמש אחר נכנס עם סיסמה שונה משל אליס: **hash collision**.
- לקוח ששולח `Hash(Username || Password)` על TCP רגיל: פגיע ל**מאזין**, שפשוט ישדר את אותו ערך שוב (replay). ה-hash הופך בפועל לסיסמה.

MAC (Message Authentication Code)

Hash רגיל אפשר לחשב בלי סוד, ולכן תוקף שמשנה הודעה פשוט מחשב hash חדש. **MAC** הוא "hash עם מפתח": פונקציה שמקבלת הודעה `m` ומפתח סימטרי `k` ומחזירה tag. בלי k אי אפשר לייצר tag תקין.

1. לאליס ולבוב יש מפתח משותף k.
2. אליס מחשבת $tag = MAC_k(m)$ ושולחת את `m` ואת ה-tag.

3 בוב מחשב בעצמו $\mathrm{MAC}_k(m)$ על ההודעה שקיבל ומשווה ל-tag. שווה: ההודעה לא שונתה והגיעה ממי שמחזיק את k.

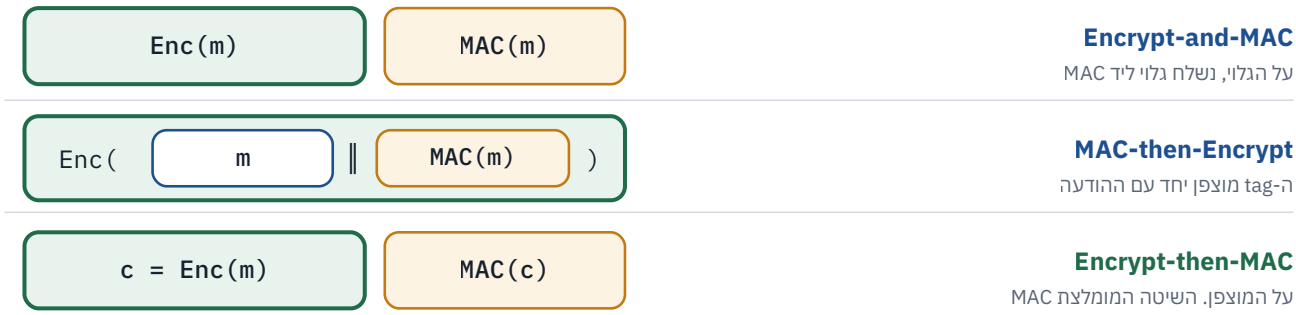


MAC. אליס מחשבת tag מההודעה ומהמפתח המשותף. בוב מחשב בעצמו ומשווה. מי שאין לו k לא יכול לייצר tag תקין להודעה ששינה.

MAC נותן **שלמות** (ההודעה לא שונתה) ו**אותנטיות** (נשלחה ע"י מי שמחזיק את k). הוא **לא** נותן **אי-הכחשה**. הדוגמה מההרצאה: בוב הולך לבית משפט עם הודעה ו-tag ואומר "רק אליס יכלה לייצר את זה". אליס עונה "גם לבוב יש את k, אולי הוא ייצר את זה בעצמו". אין דרך להכריע. אי-הכחשה דורשת מפתח ש**רק לאדם אחד** יש, כלומר חתימה דיגיטלית (פרק 4). (בפועל MAC נפוץ שנבנה מפונקציית hash נקרא HMAC.)

שילוב הצפנה ו-MAC

רוצים גם סודיות (אף אחד לא קורא) וגם אותנטיות (אף אחד לא משנה או מתחזה). יש שלוש דרכים לשלב, וההבדל הוא **על מה** מחושב ה-MAC ו**האם הוא מוצפן**:



שלוש דרכים לשלב הצפנה ו-MAC. מה בדיוק עובר בערוץ. המסגרת הירוקה מסמנת מה מוצפן.

שיטה	מה נשלח	מה המקבל עושה, ובאיזה סדר
Encrypt-and-MAC	$\mathrm{Enc}(m)$, $\mathrm{MAC}(m)$	מפענח, ואז מחשב MAC על ה-**plaintext** ומשווה ל-tag שהגיע **גלוי** ליד ה-ciphertext
MAC-then-Encrypt	$\mathrm{Enc}(m \,\|\, \mathrm{MAC}(m))$	קודם **מפענח**, מפריד בין ההודעה ל-tag (ה-tag היה **בתוך** ההצפנה), ואז בודק MAC על ה-plaintext
Encrypt-then-MAC	$c = \mathrm{Enc}(m)$, $\mathrm{MAC}(c)$	קודם **בודק MAC על ה-ciphertext**, ורק אם תקין מפענח. השיטה המומלצת

למה Encrypt-then-MAC עדיף: המקבל דוחה הודעה מזויפת **לפני** שהוא מפענח אותה, כך שהתוקף לא יכול "לשחק" עם מנגנון הפענוח ולהסיק מידע מהתגובות שלו. ובנוסף, ב-Encrypt-and-MAC ה-tag מחושב על ה-plaintext ונשלח גלוי, ו-MAC לא מתחייב להסתיר את ההודעה, כך שהוא עלול להדליף מידע (למשל: אותה הודעה תמיד נותנת אותו tag).

במבחן: איך מזהים את השיטה מתיאור

שואלים שתי שאלות: (1) על מה חושב ה-MAC, על ה-plaintext או על ה-ciphertext? (2) האם ה-tag מוצפן יחד עם ההודעה? MAC על ה-ciphertext הוא תמיד Encrypt-then-MAC. MAC על ה-plaintext כשה-tag מוצפן בפנים הוא MAC-then-Encrypt. ה-plaintext כשה-tag נשלח גלוי הוא Encrypt-and-MAC.

מלכודת

אליס שולחת לבוב ולקרול $M,\ MAC_K(M),\ Enc_{PK_B}(K),\ Sign_{SK_A}(hash(K))$. החתימה היא רק על K, לא על ההודעה. בוב שקיבל את K יכול להצפין את אותו K לקרול, לצרף את אותה חתימה, ולכתוב MAC על הודעה שאליס לא שלחה. כלומר **אין אותנטיות**. הלקח: מפתח MAC שמשותף ליותר משני צדדים לא מוכיח מי מהם כתב.

סיכום הפרק

- Hash: קלט בכל אורך, פלט קבוע, אין מפתח. Pre-image (מ-d ל-m), second pre-image (m_0 נתון), collision (שניהם לבחירת התוקף).
- יום הולדת: התנגשות אחרי כ-$\sqrt{N}$ ניסיונות; hash של n ביט נותן $2^{n/2}$.
- סיסמאות: hash + salt גלוי לכל משתמש (מבטל מילון מוכן) + pepper סודי מחוץ ל-DB + stretching (מאט כל ניחוש).
- MAC: שלמות + אותנטיות, בלי אי-הכחשה. Encrypt-then-MAC: MAC על המוצפן, בודקים לפני פענוח.

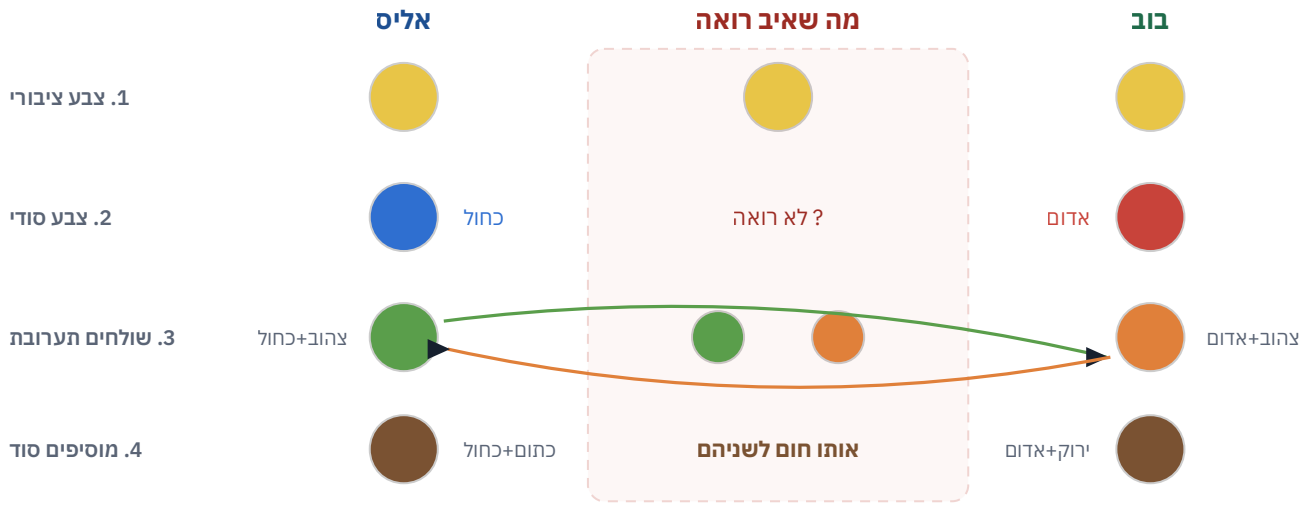
4 קריפטוגרפיה אסימטרית: דיפי-הלמן, RSA וחתימות

בפרקים 2 ו-3 כל הכלים הניחו שלאליס ולבוב **כבר יש** מפתח משותף. אבל איך מסכימים על מפתח סודי עם שרת שמעולם לא פגשנו, כשכל מה שנשלח עובר דרך מאזין? הפרק הזה פותר את הבעיה בשתי דרכים: **דיפי-הלמן** (שני הצדדים "מייצרים" מפתח משותף בגלוי) ו-**RSA** (לכל צד זוג מפתחות, פומבי ופרטי). אותו RSA נותן גם **חתימה דיגיטלית**.

הרעיון: זוג מפתחות

בהצפנה אסימטרית לכל משתתף יש **שני** מפתחות שקשורים מתמטית: **מפתח פומבי** (public key, pk) שמפרסמים לכל העולם, ו**מפתח פרטי** (secret/private key, sk) שרק הבעלים יודע. מה שמוצפן באחד נפתח רק בשני. כך אין צורך להעביר סוד מראש: כל אחד יכול להצפין לבוב עם המפתח הפומבי שלו, ורק בוב יכול לפענח. המחיר: החישובים כבדים, ולכן ההצפנה הזו **איטית בהרבה** מהצפנה סימטרית.

ההרצאה מסבירה את הרעיון בעזרת צבעים: קל לערבב שני צבעים, וכמעט בלתי אפשרי להפריד תערובת בחזרה לרכיביה.



דיפי-הלמן בצבעים (האנלוגיה מההרצאה). קל לערבב צבעים וקשה מאוד "להפריד" תערובת. איב רואה את הצהוב, הירוק והכתום, אבל לא את הכחול או האדום, ולכן לא יכולה להרכיב את החום.

אליס ובוב מסכימים בגלוי על צבע בסיס (צהוב). כל אחד מוסיף צבע סודי (כחול, אדום) ושולח את התערובת. כל אחד מוסיף לתערובת שקיבל את הסוד **שלו**, ושניהם מגיעים לאותו צבע, כי שניהם ערבבו צהוב+כחול+אדום, רק בסדר אחר. בשפה של מפתחות: הכחול והאדום הם המפתחות הפרטיים, הירוק והכתום הם הפומביים. עכשיו מחליפים צבעים במספרים.

רקע מתמטי: חשבון מודולרי וחבורות

$a \bmod n$ הוא השארית של חלוקת a ב-n. למשל $17 \bmod 5 = 2$, ו-$-3 \bmod 7 = 4$. כותבים $a \equiv b \pmod n$ כש-a ו-b נותנים אותה שארית. הכלל שחוסך את כל החישובים: **מותר לצמצם מודולו n בכל שלב**, גם באמצע כפל. למשל $(13 \cdot 13) \bmod 17 = 169 \bmod 17 = 16$, ואפשר גם לכתוב $13 \equiv -4$ ואז $(-4) \cdot (-4) = 16$.

- $\mathbb{Z}_p = \{0, 1, \ldots, p-1\}$ עבור p ראשוני, ו-$\mathbb{Z}_p^* = \{1, \ldots, p-1\}$ הם האיברים שיש להם הופכי (כולם חוץ מ-0).
- $\mathbb{Z}_p^*$ עם כפל מודולו p היא **חבורה**: (1) **סגירות**, מכפלה של שני איברים היא איבר; (2) **אסוציאטיביות**, $(a \cdot b) \cdot c = a \cdot (b \cdot c)$; (3) **איבר יחידה** 1; (4) לכל איבר יש **הופכי**: $x \cdot x^{-1} \equiv 1 \pmod 5$. למשל $3 \cdot 2 = 6 \equiv 1 \pmod 5$, ולכן $3^{-1} \equiv 2 \pmod 5$.
- **חבורה ציקלית** (מעגלית): יש בה **יוצר** g שהחזקות שלו $g, g^2, g^3, \ldots$ עוברות על **כל** איברי החבורה.

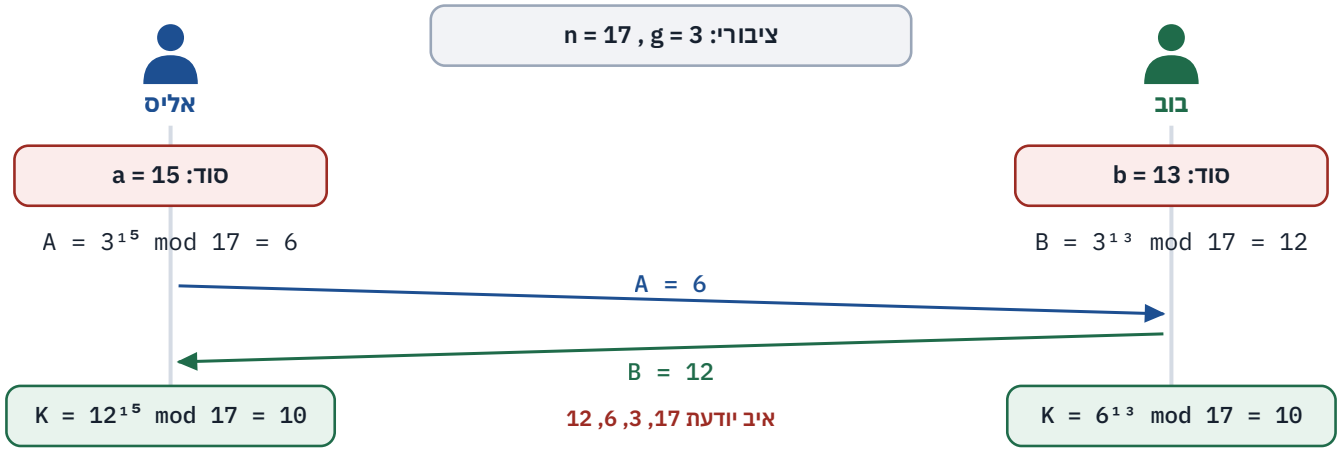


חבורה ציקלית ויוצר. בחבורה $Z_7^* = \{1,\dots,6\}$ עם כפל מודולו 7: החזקות של 3 עוברות על **כל** האיברים, ולכן 3 הוא יוצר. החזקות של 2 חוזרות אחרי 3 צעדים, ולכן 2 אינו יוצר. לחשב $3^5 \bmod 7 = 5$ קל; לגלות מ-5 שהמעריך היה 5 קשה כשהמספרים ענקיים.

בעיית הלוגריתם הדיסקרטי: בהינתן p ראשוני גדול, יוצר g, ואיבר $a = g^x \bmod p$, קשה למצוא את x. בכיוון אחד החישוב קל (חזקה מודולרית, בעזרת ריבועים חוזרים, פרק 11). בכיוון ההפוך, כשהמספרים בני מאות ספרות, אין שום שיטה יעילה ידועה: ה"חזקות" קופצות על פני המעגל בלי שום סדר נראה. זו בדיוק "הפרדת התערובת" של אנלוגיית הצבעים.

החלפת מפתחות דיפי-הלמן

1. מסכימים **בגלוי** על ראשוני גדול n ועל יוצר g.
2. אליס בוחרת בסוד מספר אקראי a ושולחת $A = g^a \bmod n$. בוב בוחר בסוד b ושולח $B = g^b \bmod n$.
3. אליס מחשבת $B^a \bmod n$, ובוב מחשב $A^b \bmod n$. שניהם מקבלים את אותו מפתח: $K = (g^b)^a = (g^a)^b = g^{ab} \bmod n$.



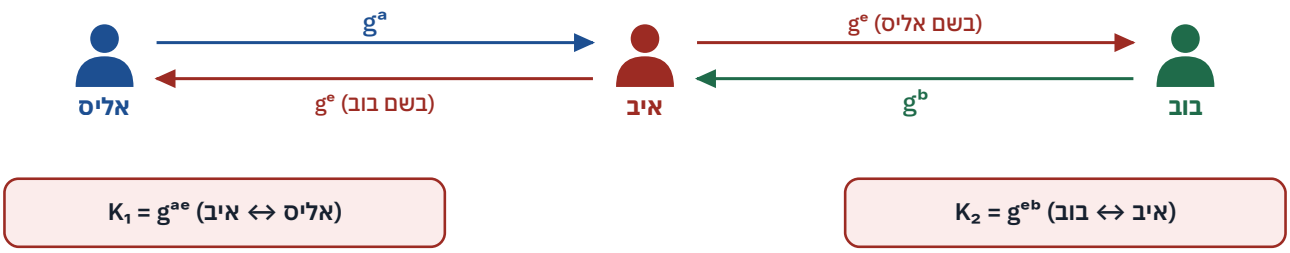
דיפי-הלמן במספרים (הדוגמה מההרצאה). $n = 17,\ g = 3$. אליס בוחרת בסוד 15, בוב בוחר 13. ברשת עוברים רק 6 ו-12. שניהם מגיעים ל-$K = 10$; איב, כדי לחשב אותו, צריכה לפתור לוגריתם דיסקרטי.

מאזינה רואה $n,\ g,\ A,\ B$. כדי לחשב את K היא צריכה את a או את b, כלומר לפתור לוגריתם דיסקרטי. לכן **מול מאזין הפרוטוקול בטוח**. את K משתמשים אחר כך כמפתח להצפנה סימטרית.

> **מלכודת**
> הסדר בחזקה: מעלים את **g בחזקת המספר הסודי** ($g^x \bmod n$), ולא את הסוד בחזקת g. זו התשובה השגויה הקבועה בשאלות "בחרו את הסעיף שאינו נכון".

החולשה: אדם-בתווך

הפרוטוקול לא בודק **ממי** הגיע הערך. איב, אם היא MitM, מבצעת דיפי-הלמן נפרד מול כל צד:



פענוח ב-K_1, קריאה ושינוי, הצפנה ב-K_2

מתקפת אדם-בתווך על דיפי-הלמן. איב מבצעת החלפה נפרדת מול כל צד. אליס חושבת ש-E הגיע מבוב; בוב חושב ש-E הגיע מאליס. כל הודעה מפוענחת אצל איב ומוצפנת מחדש. הבעיה: אין שום אימות של מי שלח את הערך.

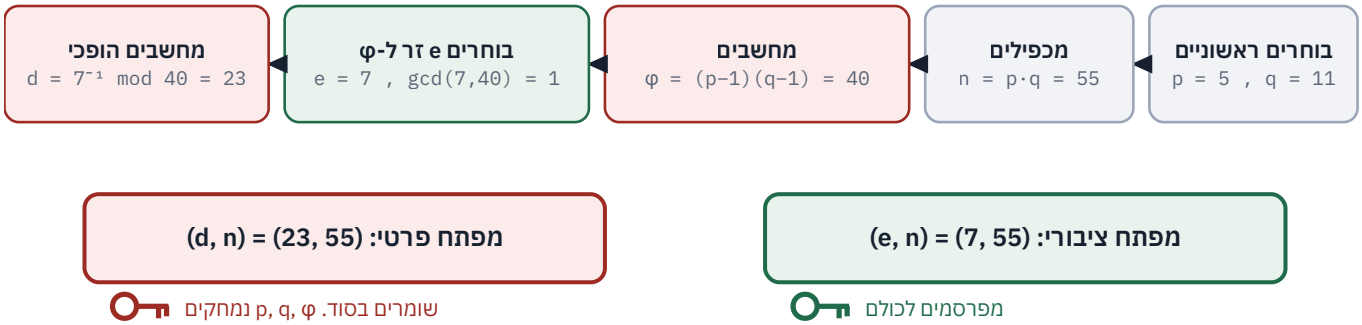
הפתרון הוא **לאמת** את ערכי ההחלפה: אליס מצפינה את A במפתח הפומבי של בוב (או חותמת עליו במפתח הפרטי שלה), ובוב עושה אותו דבר. לאיב אין את המפתחות הפרטיים, ולכן היא לא יכולה לפענח או לזייף, ולא יכולה להשלים את החישוב. בפועל, האימות נעשה בעזרת תעודות (פרק 5).

במבחן: "איזו שיטה לא תעבוד מול מאזין?"

- × כל צד שולח בערוץ מפתח סימטרי שבחר: המאזין רואה אותו.
- × כל צד שולח את המפתח הפומבי שלו, ואז **מצפין במפתח הפומבי של עצמו**: רק הוא יכול לפענח. צריך להצפין במפתח הפומבי של **הצד השני**.
- ✓ דיפי-הלמן, תעודות של CA ודיפי-הלמן, או החלפת מפתחות פומביים והצפנה במפתח של הצד השני. כל אלה עובדים מול מאזין.

Textbook RSA

RSA נותן לכל משתתף זוג מפתחות קבוע. הבטיחות שלו נשענת על בעיה קשה אחרת: **פירוק לגורמים**. קל להכפיל שני ראשוניים ענקיים, וקשה מאוד, בהינתן המכפלה, למצוא אותם.



יצירת מפתחות RSA, עם המספרים הקטנים של דוגמה 10 (פרק 11). רק מי שיודע את p ו-q יכול לחשב את φ, ובלי φ אי אפשר לחשב את d. הבטיחות נשענת על כך שקשה לפרק את n לגורמים.

יצירת מפתחות: ראשוניים p, q; $n = p \cdot q$; $\varphi(n) = (p-1)(q-1)$; בוחרים e כך ש-$\gcd(e, \varphi) = 1$; מחשבים d כך ש-$e \cdot d \equiv 1 \pmod{\varphi(n)}$.

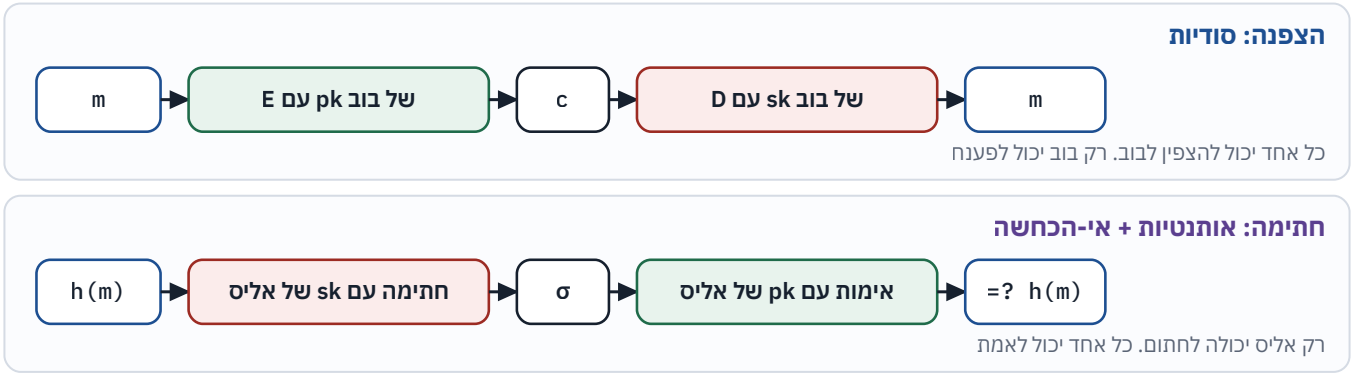
מפתח פומבי (e, n) · **מפתח פרטי** (d, n)

הצפנה $c = m^e \bmod n$ · **פענוח** $m = c^d \bmod n$

חתימה $\sigma = m^d \bmod n$ · **אימות** $\sigma^e \bmod n \stackrel{?}{=} m$

למה זה עובד: $(m^e \bmod n)^d \bmod n = m^{ed} \bmod n = m$, כי $ed \equiv 1 \pmod{\varphi(n)}$ (משפט אוילר). התנאי סימטרי ב-e ו-d, ולכן אפשר גם "להצפין" בפרטי ו"לפענח" בפומבי. זה הבסיס לחתימה.

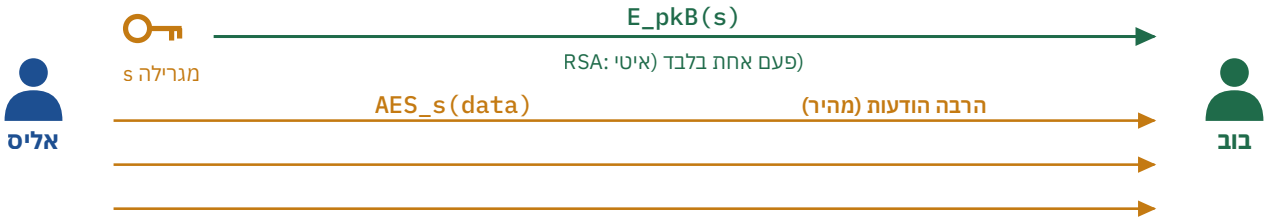
למה תוקף לא יכול לחשב את d: d נגזר מ-φ, ו-φ נגזר מ-p ו-q. מי שיודע רק את (e, n) צריך לפרק את n לגורמים, וזה לא אפשרי כשהוא בן 2048 ביט. בדוגמאות קטנות ($n = 55$) רואים מיד ש-$55 = 5 \cdot 11$, ולכן במבחן אפשר לחשב הכל ביד. דוגמאות מלאות: פרק 11.



מי משתמש באיזה מפתח. בהצפנה מטרת המפתח הפומבי של **המקבל** היא שכולם יוכלו להצפין ורק הוא יוכל לפענח. בחתימה ההפך: רק **השולח** יכול לחתום, וכולם יכולים לאמת.

הצפנה היברידית

RSA איטי, וגם מוגבל: אפשר להצפין רק $m < n$. לכן אף אחד לא מצפין קובץ שלם ב-RSA. במקום זה משלבים את שני העולמות: מגרילים מפתח **סימטרי** אקראי, מעבירים אותו מוצפן ב**מפתח הפומבי** של הצד השני, ומשם ממשיכים בהצפנה סימטרית מהירה.



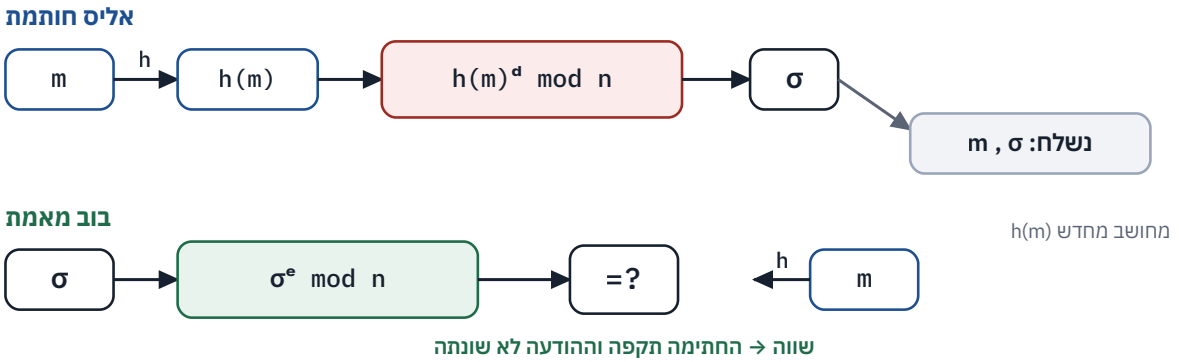
פתרנו את בעיית העברת המפתח (אסימטרי) ושמרנו על מהירות (סימטרי)

הצפנה היברידית. הצפנה אסימטרית איטית, ולכן משתמשים בה פעם אחת בלבד, כדי להעביר מפתח סימטרי אקראי. את כל הנתונים מצפינים בהצפנה סימטרית מהירה עם המפתח הזה. כך עובד TLS עם RSA.

אבל נשארה בעיה אחת: איך אליס יודעת שהמפתח הפומבי שקיבלה **באמת** של בוב, ולא של איב שהתחזתה אליו? זו השאלה של פרק 5.

חתימה דיגיטלית

חתימה היא ה"הפוך" של הצפנה: רק בעל המפתח הפרטי יכול לחתום, וכל מי שיש לו את המפתח הפומבי יכול לאמת. היא נותנת את שתי התכונות של MAC, **שלמות** (כל שינוי של ביט אחד יכשיל את האימות) ו**אותנטיות** (רק בעל sk יכול היה לחתום), ובנוסף **אי-הכחשה**: המפתח הפרטי שייך לאדם אחד בלבד, ולכן הוא לא יכול לטעון "אולי מישהו אחר יצר את זה", כמו שקרה ב-MAC.



חתימה על hash ואימות. החותמת מחשבת $h(m)$ ומעלה אותו בחזקת d. המאמת מעלה את σ בחזקת e, ומשווה ל-hash שהוא מחשב בעצמו על ההודעה שקיבל.

החתימה הנכונה (כך מנוסחת התשובה הנכונה)

מחשבים $h(m)$, "מצפינים" אותו עם **המפתח הפרטי של השולח** ושולחים יחד עם ההודעה. המקבל "מפענח" עם **המפתח הפומבי של השולח** ומשווה ל-hash שהוא מחשב על ההודעה. כל וריאציה עם "פומבי של השולח לחתימה" או "פרטי של השולח לאימות" שגויה.

למה חותמים על $h(m)$ ולא על m עצמו? שתי סיבות:

1. **זיוף כפלי.** ב-textbook RSA, אם יש לתוקף חתימות על m_1 ועל m_2, הוא כופל אותן ומקבל חתימה חוקית על $m_1 \cdot m_2$, הודעה שהחותם מעולם לא ראה:

$$\sigma_1 \cdot \sigma_2 = m_1^d \cdot m_2^d = (m_1 \cdot m_2)^d \bmod n$$

עם hash זה לא עובד, כי $h(m_1 \cdot m_2) \neq h(m_1) \cdot h(m_2)$.

2. **אורך.** h(m) קצר וקבוע, אז חותמים בפעולה אחת גם על קובץ של גיגה.

במבחן: סודיות + אותנטיות + אמינות

אליס שולחת $E_{PKB}(M),\ S_{SKA}(H(M))$: הצפנה ב**פומבי של בוב** (רק בוב יפענח) וחתימה ב**פרטי של אליס** (רק אליס יכולה לחתום).
$E_{PKB}(s),\ Enc_s(M)$ נותן סודיות בלבד, בלי אימות שולח. הצפנה ב-PKA או חתימה ב-SKB שגויים מהיסוד.

במבחן: עוד ניסוחים שחוזרים

- אי-הכחשה (X לא יוכל להתכחש): **חתימות דיגיטליות** / public key cryptography. לא MAC.
- הצפנה נועדה ל**סודיות**; חתימה נועדה לאימות **מקור ושלמות**.
- שרת מפיץ מפתחות פומביים עם חתימה על (מפתח + תאריך ושעה): פגיע ל-MitM. החתימה לא כוללת **את שם בעל המפתח**, ולכן תוקף מבקש מהשרת את המפתח של עצמו, מקבל תשובה חתומה ועדכנית ומעביר אותה לאליס כאילו זה המפתח של בוב.

סיכום הפרק

- אסימטרי: pk לכולם, sk רק לבעלים. איטי, ולכן משמש להעברת מפתח סימטרי (היברידי).
- דיפי-הלמן: $A = g^a,\ B = g^b,\ K = g^{ab} \bmod n$. בטוח מול מאזין (לוגריתם דיסקרטי), פגיע ל-MitM בלי אימות.
- RSA: $n = pq,\ \varphi = (p-1)(q-1),\ ed \equiv 1 \pmod{\varphi}$. הצפנה m^e, פענוח c^d. בטיחות: פירוק לגורמים.
- הצפנה: פומבי של המקבל. חתימה: פרטי של השולח, על $h(m)$. חתימה = שלמות + אותנטיות + אי-הכחשה.

5 PKI ותעודות דיגיטליות

הצפנה אסימטרית פותרת את העברת המפתח, אבל פותחת בעיה חדשה: הדפדפן מקבל מפתח פומבי שטוען "אני של bank.com". איך הוא יודע שזה לא המפתח של איב? אם איב מצליחה להחליף את המפתח, היא MitM מושלמת, וכל ההצפנה חסרת ערך. **PKI** (Public Key Infrastructure) היא התשתית שעונה על השאלה הזו, בעזרת **תעודות** שחותמים עליהן גופים שכולם סומכים עליהם: **CA** (Certificate Authority).

מה זו תעודה

תעודה דיגיטלית היא מסמך שאומר "המפתח הפומבי הזה שייך לדומיין הזה", וה-CA חותם עליו במפתח הפרטי **שלו**. מי שיש לו את המפתח הפומבי של ה-CA יכול לאמת את החתימה, ולכן לסמוך על הקישור בין השם למפתח.

Certificate	
Subject (מי)	www.bank.com
Public Key	public key of bank.com
Issuer (מי הנפיק)	Example CA
Validity	01/2026 – 01/2027
Serial Number	0x3A7F…
Extensions	CRL/OCSP URLs, SCTs …
חתימת ה-CA על כל השדות	σ = Sign_skCA(…)

× **מפתח פרטי**
לא נמצא בתעודה

מה יש בתעודה דיגיטלית. התעודה קושרת שם (subject) למפתח פומבי, וה-CA מעיד על כך בחתימה. אין בתעודה שום דבר סודי: **אין בה מפתח פרטי**.

שימו לב: התעודה ציבורית לגמרי. השרת שולח אותה לכל מי שמתחבר. מה שמונע מאיב פשוט להעתיק אותה ולהציג אותה בעצמה הוא שבלי **המפתח הפרטי** שמתאים ל-pk שבתעודה, היא לא תוכל להשלים את לחיצת היד (פרק 6): היא לא תוכל לפענח את מה שהלקוח מצפין במפתח הפומבי.

הנפקת תעודה

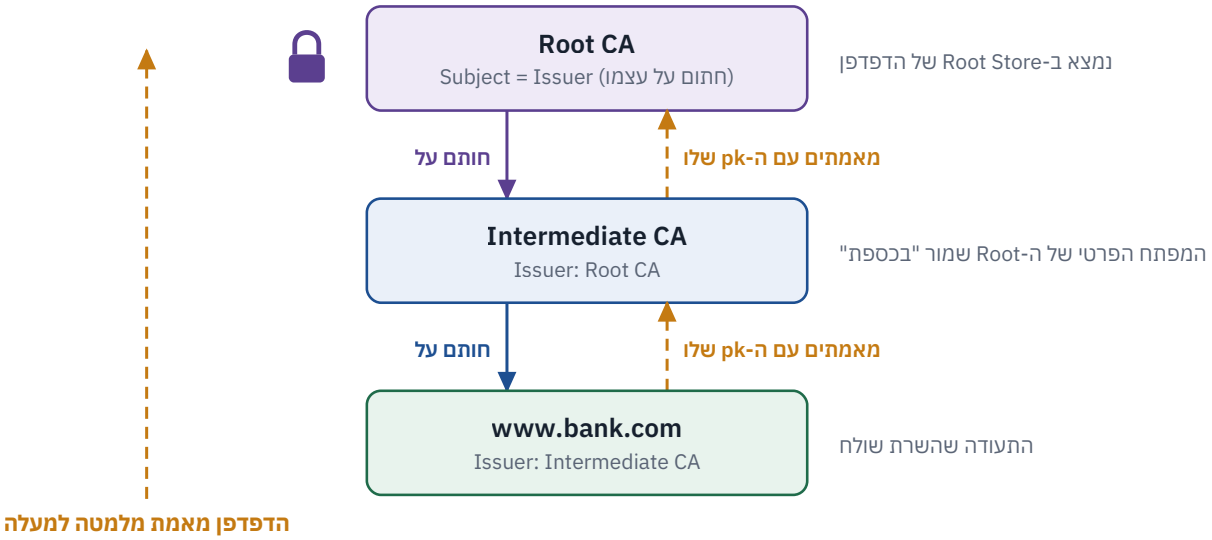


הנפקת תעודה. האתר מייצר את זוג המפתחות בעצמו, וה-CA בודק שהוא באמת שולט בדומיין (verify eligibility), למשל באתגר DNS שרק בעל הדומיין יכול לבצע.

1. האתר מייצר בעצמו זוג מפתחות pk, sk. **המפתח הפרטי לא יוצא מהשרת ולא מופיע בתעודה.**
2. האתר פונה ל-CA. ה-CA מוודא שהפונה באמת שולט בדומיין (verify eligibility). הדוגמה מההרצאה: ה-CA מבקש להוסיף לשרת ה-DNS של הדומיין רשומה עם טקסט שהוא בחר. רק מי ששולט בדומיין יכול לעשות את זה. יש גם בדיקות קפדניות יותר, עד בדיקה ידנית ע"י אדם.
3. ה-CA חותם עם **המפתח הפרטי שלו** על התעודה, שמכילה: subject (הדומיין), pk של האתר, issuer (מי הנפיק), תוקף (from–to), חתימת ה-CA וextensions.

שרשרת אמון (Chain of Trust)

כדי לאמת את חתימת ה-CA צריך את המפתח הפומבי שלו. אבל איך יודעים שהמפתח הזה באמת של ה-CA? הוא מגיע בתעודה שחתם עליה CA אחר, וכן הלאה, עד שמגיעים ל-**Root CA**. תעודת ה-Root חתומה על ידי עצמה (self-signed: ה-subject וה-issuer זהים), ולכן אי אפשר לאמת אותה בחתימה. במקום זה, היא **כבר נמצאת** אצלנו: מערכת ההפעלה והדפדפן מגיעים עם רשימה של תעודות Root שהם סומכים עליהן, ה-**Root Store** (למשל DigiCert).



שרשרת אמון. כל תעודה חתומה במפתח הפרטי של הרמה שמעליה. הדפדפן מאמת מלמטה למעלה, עד תעודת Root שכבר נמצאת אצלו ב-Root Store. ה-Root חותם על עצמו, ולכן הוא חייב להגיע מראש עם מערכת ההפעלה או הדפדפן.

למה צריך Intermediate: אם המפתח הפרטי של Root דולף, צריך לבטל אותו, כלומר לעדכן את ה-Root Store בכל מחשב בעולם. לכן שומרים אותו "בכספת" (לרוב לא מחובר לרשת) ומשתמשים בו רק לחתימה על תעודות של Intermediate CA. ה-Intermediate הם שחותמים על תעודות האתרים, ואם אחד מהם נפרץ, מבטלים רק אותו.

דוגמה מההרצאה: Fiddler

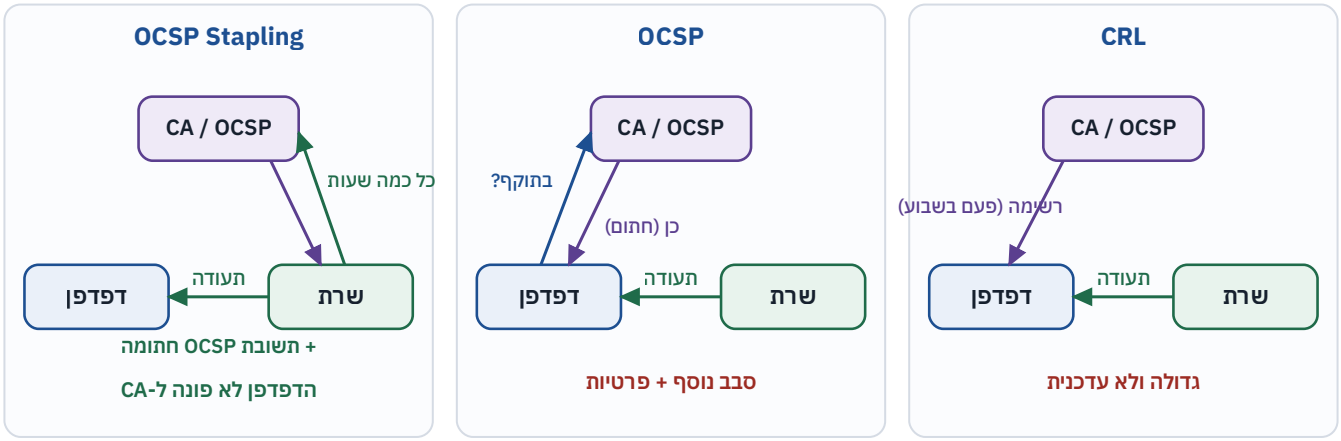
תוכנה לניתוח תעבורה שיודעת "לראות" בתוך HTTPS. איך? היא מתקינה **תעודת Root משלה** ב-Root Store של המחשב, ואז מייצרת תעודות "מזויפות" לכל אתר, חתומות ע"י ה-Root שלה. הדפדפן מקבל אותן, כי ה-Root נמצא ברשימה. זו בדיוק מתקפת MitM, והיא עובדת רק כי מי שמחזיק את המחשב הכניס את ה-Root. הלקח: מי שיכול להוסיף Root ל-Root Store שולט בכל ה-HTTPS של המחשב.

במבחן

- ב-TLS **משתמשים** ב-chain of trust, והיא מה שמאפשר ללקוח לוודא את התעודה של foo.com. הלקוח בודק **חתימות תקינות לאורך השרשרת עד Root שהוא סומך עליו**.
- ה-CA שהנפיק לשרת את התעודה **לא חייב** להיות ב-Root Store (מספיק שיש שרשרת). Root CA חותם על המפתח הפומבי של intermediate CA, ולא ישירות על כל שרת.
- התעודה של foo.com חתומה ע"י ה-CA, לא ע"י foo.com. שרת אינטרנט **לא** מייצר תעודה לכל חיבור. אסור שיופיע בתעודה: **מפתח פרטי**.
- CA שהנפיק תעודה לאתר **לא** יכול לפענח את התעבורה שלו, כי אין לו את המפתח הפרטי של האתר.

ביטול תעודות (Revocation)

לתעודה יש תאריך תפוגה, אבל לפעמים צריך לבטל אותה **לפני** כן: המפתח הפרטי של האתר נגנב, או שה-CA הנפיק תעודה בטעות. המקרה המפורסם: **DigiNotar** (2011). ה-CA ההולנדי נפרץ, הונפקו בשמו תעודות מזויפות (בין השאר ל-Google), הדפדפנים הוציאו אותו מרשימות האמון, כל התעודות שהנפיק בוטלו, והחברה נסגרה. ביטול **Root** קשה במיוחד, כי צריך לעדכן את כל מערכות ההפעלה. איך הדפדפן יודע שתעודה בוטלה?



שלוש דרכים לבדוק שתעודה לא בוטלה. ב-CRL הדפדפן מוריד רשימה. ב-OCSP הוא שואל בזמן אמת (וה-CA לומד לאן גולשים). ב-Stapling השרת מביא מראש תשובה חתומה ומצרף אותה, והדפדפן לא פונה ל-CA בכלל.

- **CRL** (Certificate Revocation List): ה-CA מפרסם רשימה של המספרים הסידוריים של כל התעודות שבוטלו, עם תאריך וסיבה. כתובת הרשימה מופיעה בתעודה (CRL Distribution Points), ולרוב מחלקים אותה לכמה רשימות קטנות. החסרונות: הרשימות גדולות, והן מתעדכנות רק מדי פעם (בדוגמה מההרצאה, פעם בשבוע). בחלון הזה תעודה מבוטלת עדיין נראית תקפה.
- **OCSP** (Online Certificate Status Protocol): הדפדפן שואל את שרת ה-OCSP של ה-CA, בזמן אמת, "האם התעודה הזו תקפה?", ומקבל תשובה **חתומה ע"י ה-CA**. החסרונות: סבב תקשורת נוסף לפני שהדף נטען (extra round-trip), עומס על השרת של ה-CA, ו**פגיעה בפרטיות**: ה-CA לומד לאילו אתרים כל משתמש גולש.
- **OCSP Stapling**: **השרת** פונה ל-OCSP בעצמו, אחת לכמה דקות עד שעות, ומקבל תשובה חתומה ועדכנית. הוא "מצמיד" (staple) אותה לתעודה בזמן ה-handshake. הלקוח מאמת את חתימת ה-CA על התשובה, בלי לפנות ל-CA. זה פותר את בעיות הפרטיות, ההשהיה והעומס. השרת לא יכול לזייף את התשובה, כי היא חתומה ע"י ה-CA.
- **CRLSets**: רשימת ביטולים מצומצמת שהדפדפן (Chrome) מוריד ומעדכן בעצמו. מכסה רק חלק מהתעודות.

Soft fail: למה ביטול לא באמת עוצר תוקף

מה עושה דפדפן ששאל את OCSP ולא קיבל תשובה (שרת לא זמין, תקלה רגעית)? רוב הדפדפנים **מקבלים את התעודה כתקפה**, כדי לא לשבור את הגלישה בגלל תקלה אצל ה-CA ("חף מפשע עד שהוכח אחרת"). זה נקרא **soft fail**. אבל תוקף MitM שולט ברשת, אז הוא פשוט חוסם את בקשת ה-OCSP:



Soft fail מנוצל ע"י MitM. התוקפת מציגה תעודה גנובה שכבר בוטלה, וחוסמת את בקשת ה-OCSP. הדפדפן לא מקבל תשובה, מניח שהכל בסדר, וממשיך.

לכן soft fail **לא** מסייע לעצור תוקף, וזה מסביר למה תעודה גנובה עדיין מתקבלת שעות אחרי הביטול. ב-Stapling יש לזה מענה חלקי: תעודה עם הרחבת Must-Staple אומרת לדפדפן "תדחה אותי אם אין staple".

מלכודות

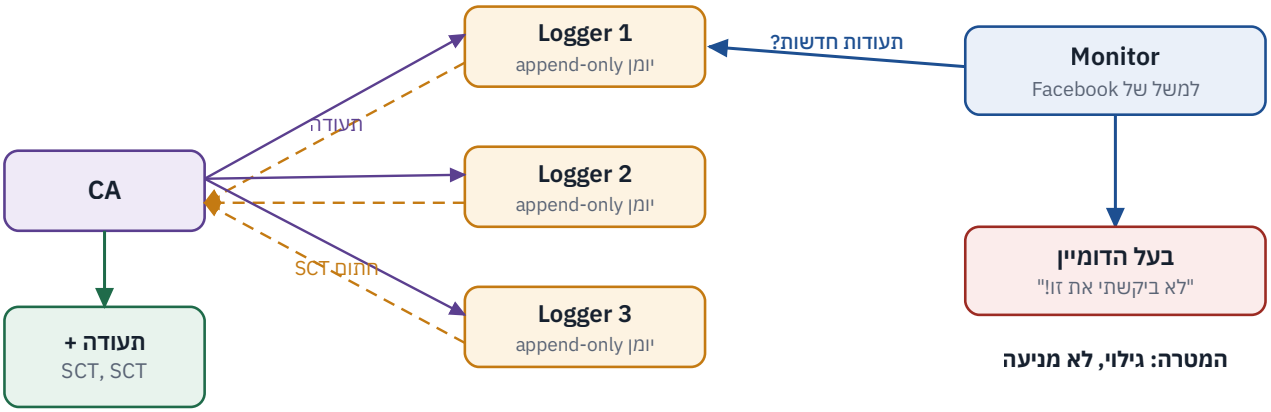
- "OCSP stapling יעיל בזכות עץ מרקל": **שגוי**. עץ מרקל שייך ל-CT ולביטקוין.
- "OCSP משמש miners לבדיקת בלוק": **שגוי**. אין קשר בין OCSP לביטקוין.
- "OCSP stapling הוא מנגנון שונה לחלוטין מ-OCSP": התשובה זהה (אותה תשובה חתומה), רק מי שמביא אותה שונה.

מפתח פרטי של שרת נגנב: מה התוקף יכול

✓ יכול	× לא יכול
להתחזות לשרת מול לקוחות, אם הוא MitM; לחתום על מידע בשם השרת; לפענח תעבורה **ישנה** שהקליט, אם החיבור השתמש בהחלפת מפתחות RSA (אין forward secrecy, פרק 6)	לחתום על CRL או לזייף תשובות OCSP בשם ה-CA, כי אלה חתומים במפתח **של ה-CA**. האתר מבקש מה-CA לבטל, וזו טענה נכונה

Certificate Transparency (CT)

ביטול עוזר רק אחרי שמישהו **יודע** שיש תעודה בעייתית. אבל מה אם CA נפרץ ומנפיק "בשקט" תעודה ל-google.com, ותוקף MitM משתמש בה מול קורבנות? הקורבנות רואים מנעול תקין, ואף אחד לא יודע שהתעודה קיימת. CT, שפותח ע"י Google, דורש ש**כל תעודה תירשם ביומנים ציבוריים** לפני שהדפדפן יקבל אותה. המטרה היא **גילוי** מהיר, לא מניעה.



Certificate Transparency. ה-CA חייב לדווח על כל תעודה ללוגרים, ומקבל מכל אחד SCT חתום שמוטמע בתעודה. הדפדפן דוחה תעודה בלי מספיק SCT-ים. מוניטורים שואלים את הלוגרים על תעודות חדשות, וכך בעל הדומיין מגלה תעודה שהונפקה בשמו בלי ידיעתו.

- **Logger**: שירות שמתחזק יומן ציבורי של כל התעודות שדווחו לו. היומן **append-only** (אפשר להוסיף, אי אפשר למחוק), והוא בנוי כ**עץ מרקל** שבעליו ה-hash-ים של התעודות (עצי מרקל מוסברים בפרק 7). כך ה-Logger יכול להוכיח שתעודה נמצאת ביומן בהוכחה קצרה, בגובה העץ.
- **SCT** (Signed Certificate Timestamp): כשה-CA מדווח, ה-Logger לוקח את התעודה (המספר הסידורי), מוסיף את התאריך והשעה, וחותם במפתח הפרטי שלו. זו ההוכחה שה-CA דיווח. ה-SCT-ים מוטמעים בתעודה (Embedded SCTs).
- **אכיפה בדפדפן**: הדפדפן דורש כמה SCT-ים מ**לוגרים שהוא סומך עליהם**. בקורס: Chrome דורש לוגר של Google ולוגר שאינו של Google. כיום (מאז 2022) Chrome דורש 2 SCT-ים לתעודה של עד 180 יום ו-3 לתעודה ארוכה יותר, משני מפעילים שונים לפחות. הרעיון: גם אם תוקף השתלט על CA, ואפילו על Logger אחד, קשה לו להשתלט על כולם.
- **Monitor**: שירות שפונה ללוגרים אחת לכמה זמן ושואל "ראיתם תעודות חדשות?" (למשל שירות של Facebook). בעל דומיין שמפעיל מוניטור מגלה מיד תעודה שהונפקה בשמו בלי שביקש, ומבקש מה-CA לבטל. הלוגרים מחויבים לפרסם כל תעודה תוך MMD (Maximum Merge Delay).
- **Split-world**: Logger שנשלט ע"י תוקף מציג "עולם" אחד לדפדפן (עץ שכולל את התעודה המזויפת, כדי שתתקבל) ועולם אחר למוניטורים (עץ בלי התעודה, כדי שלא תתגלה). עצי מרקל ובדיקות עקביות בין מי שצופה ביומן (gossip) נועדו לחשוף זאת. אותו רעיון שקיפות משמש את WhatsApp: שם תוקף יכול לטעון "בוב החליף טלפון, זה המפתח החדש שלו", ויומן שקוף של מפתחות מאפשר לזהות החלפה חשודה.

מלכודות

- המוניטורים מנטרים את **הלוגרים**; הלוגרים מתחזקים יומן של תעודות שהנפיקו CA-ים. לא "המוניטורים מנטרים CA-ים".
- ה-CA חייב לרשום **אצל Logger**, לא אצל Monitor.
- "לא כל רשומה ב-log חייבת להיות גלויה לציבור": **שגוי**. כל היומן ציבורי.
- "SCT נועד לפתור בעיית פרטיות של OCSP": **שגוי**. את זה פותר OCSP stapling. CT ו-CRL/OCSP פותרים בעיות **שונות** (גילוי הנפקה זדונית לעומת ביטול).
- "ב-CT משתמשים ב-cookie למציאת התעודה הבאה": **שגוי**. ב-CT משתמשים בעץ מרקל.

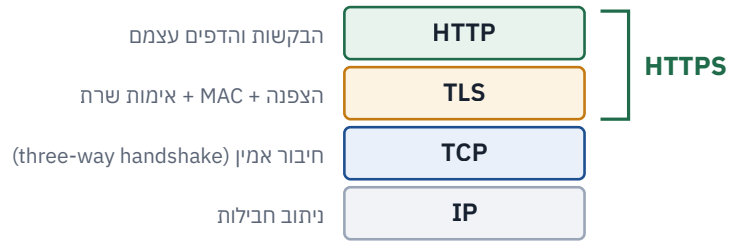
סיכום הפרק

- תעודה = issuer + pk + subject + תוקף + חתימת CA. אין בה מפתח פרטי.
- הנפקה: האתר מייצר מפתחות, ה-CA בודק בעלות על הדומיין וחותם.
- שרשרת: אתר ← Root ← Intermediate (self-signed, ב-Root Store). מאמתים מלמטה למעלה.
- ביטול: CRL (רשימה, לא עדכנית), OCSP (בזמן אמת, פרטיות וסבב נוסף), Stapling (השרת מצרף תשובה חתומה). Soft fail = MitM חוסם ומנצח.
- CT: Logger (יומן append-only בעץ מרקל) חותם SCT; Monitor שואל את הלוגרים. גילוי, לא מניעה.

6 תקשורת מאובטחת: TLS ו-HTTPS

TLS (בעבר SSL; היום גרסאות 1.2 ו-1.3) הוא המקום שבו כל מה שלמדנו מתחבר: תעודות (פרק 5) כדי לדעת שמדברים עם השרת הנכון, הצפנה אסימטרית (פרק 4) כדי להעביר סוד, הצפנה סימטרית (פרק 2) כדי להצפין את הנתונים מהר, ו-MAC (פרק 3) כדי שאף אחד לא ישנה אותם. HTTPS = HTTP over TLS. בקורס לומדים את גרסת **RSA key exchange**.

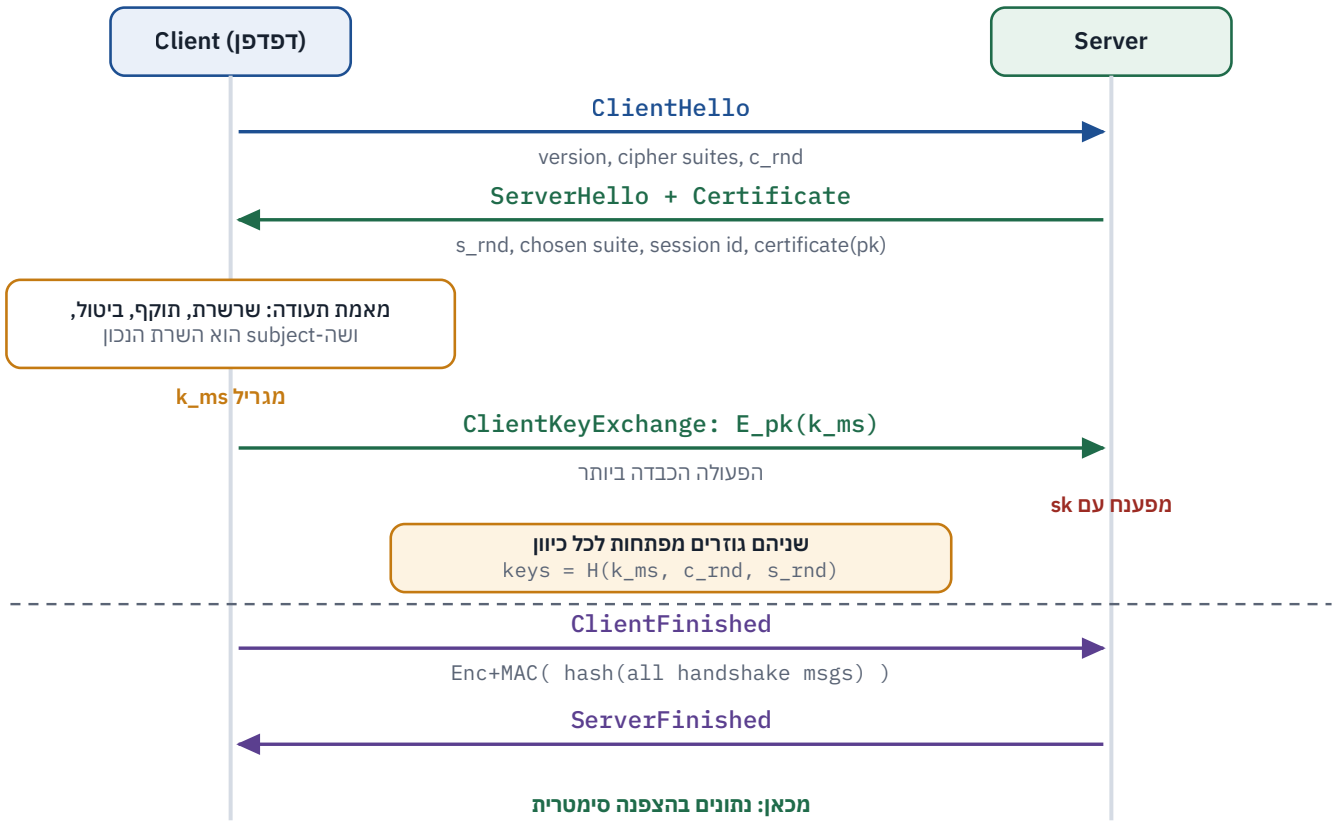
איפה TLS יושב ומה הוא מבטיח



איפה TLS יושב. קודם מקימים חיבור TCP, ואז TLS מקים מעליו ערוץ מוצפן. HTTPS הוא פשוט HTTP שעובר בתוך הערוץ הזה.

כשדפדפן רוצה לדבר עם שרת, הוא קודם מקים חיבור TCP. מיד אחרי ההקמה, ולפני שעובר שום מידע, מקימים מעליו TLS. מהרגע שהוא מוקם, TLS מבטיח שלוש תכונות: **סודיות** (רק הלקוח והשרת קוראים), **שלמות** (שינוי של ביט אחד מתגלה) ו**אימות השרת** (הלקוח בטוח שהוא מדבר עם בעל התעודה). שימו לב: TLS מאמת את **השרת**. הוא **לא** מחייב לאמת את הלקוח (זה נעשה בדרך כלל בסיסמה, אחרי שהערוץ מאובטח).

ה-Handshake המלא (כפי שנלמד בקורס)



לחיצת היד של TLS בגרסת RSA (כפי שנלמדה בקורס). כל מה שמעל הקו המקווקו נשלח לפני שיש מפתח משותף. הודעות ה-Finished הן הראשונות שמוגנות במפתחות החדשים, והן מכילות hash של כל ההודעות הקודמות.

1 **ClientHello**: הלקוח שולח את גרסת הפרוטוקול, רשימת cipher suites שהוא תומך בהם (צירופים של אלגוריתם אסימטרי, סימטרי ו-hash) ומספר אקראי c_{rnd}.

2 **ServerHello**: השרת שולח מספר אקראי משלו s_{rnd}, את ה-cipher suite שבחר מתוך הרשימה, session id ואת **התעודה** שלו (subject + מפתח פומבי).

3 הלקוח **מאמת את התעודה**: שרשרת חתימות עד Root, תוקף, ביטול, ו-**שה-subject הוא השרת שאליו רצה להתחבר**. אחר כך הוא מגריל סוד, pre-master secret k_{ms}, ושולח אותו מוצפן ב**מפתח הפומבי של השרת**: **ClientKeyExchange**. זו הפעולה הכבדה ביותר בכל התהליך.

4 השרת מפענח עם המפתח הפרטי ומקבל את k_{ms}. עכשיו שני הצדדים מפעילים פונקציית hash על $(k_{ms}, c_{rnd}, s_{rnd})$ וגוזרים ממנה מפתחות. בגרסה המלאה: ארבעה מפתחות, k_{EC}, k_{ES} להצפנה ו-k_{MC}, k_{MS} ל-MAC, אחד לכל כיוון ולכל תפקיד (ואם משתמשים ב-AES-CBC, גם IV-ים). העיקרון: לא משתמשים באותו מפתח לשתי מטרות.

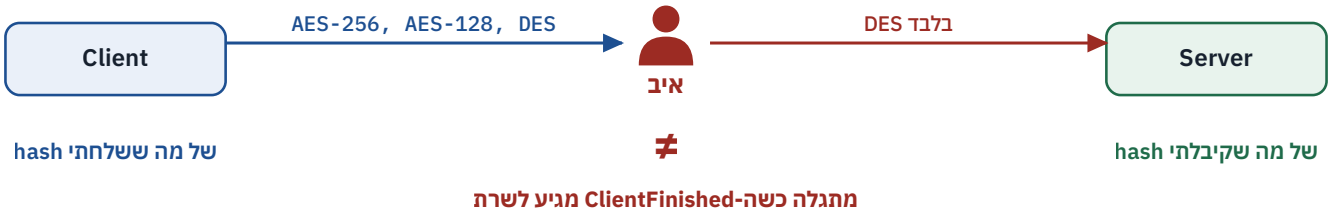
5 **ClientFinished**: הלקוח שולח, מוצפן ועם MAC במפתחות שלו, hash של **כל הודעות ה-handshake** עד כה. השרת מחשב בעצמו hash על מה ש**הוא** ראה ומשווה.

6 **ServerFinished**: אותו דבר בכיוון השני, במפתחות של השרת. הלקוח בודק. רק עכשיו ה-handshake הסתיים, ומתחילה העברת המידע בהצפנה סימטרית.

למה איב לא יכולה פשוט להציג את התעודה האמיתית של השרת? היא יכולה. אבל אז הלקוח יצפין את k_{ms} במפתח הפומבי של השרת, ובלי המפתח הפרטי איב לא תוכל לפענח אותו, לא תגזור את המפתחות, ולא תוכל לייצר ServerFinished תקין. ואם היא מציגה תעודה עם מפתח משלה, האימות בשלב 3 נכשל (אין חתימה של CA, או שה-subject לא מתאים).

שנמוך של cipher suites, ולמה Finished עוצר אותו

הלקוח מציע רשימה, והשרת בוחר. תוקף MitM יכול למחוק מה-ClientHello את כל האפשרויות החזקות ולהשאיר רק אלגוריתם חלש שהוא יודע לשבור. זו **התקפת שנמוך** (downgrade). ההגנה: hash של כל הודעות ה-handshake בהודעות ה-Finished.



למה Finished עוצר שנמוך. איב מוחקת את ההצפנות החזקות מה-ClientHello. השרת בוחר משהו חלש. אבל ב-ClientFinished הלקוח שולח hash של מה ש**הוא** שלח, והשרת משווה ל-hash של מה ש**הוא** קיבל. לא שווה: החיבור נסגר.

למה כל רכיב קיים

- **מספרים אקראיים** c_{rnd}, s_{rnd}: מבטיחים ש**המפתח הסימטרי יהיה שונה בכל חיבור** (גם ב-session resumption), ומונעים replay: תוקף שהקליט handshake ישן ומשדר אותו שוב לא יצליח, כי השרת בחר s_{rnd} חדש.
- **Finished עם hash של כל ההודעות**: מגן מפני **שנמוך**. MitM שמחק cipher suites חזקים ייתפס **מיד כשה-ClientFinished מגיע לשרת**.
- **מפתחות שונים לכל כיוון**: מונעים **התקפת reflection**, שבה התוקף מחזיר ללקוח הודעה שהלקוח עצמו שלח, כאילו השרת שלח אותה. עם מפתח לכל כיוון, ה-MAC לא יתאים.
- **תעודה**: מקשרת את המפתח הפומבי לשם השרת. בלעדיה כל MitM היה מציג מפתח משלו.

Session resumption

ה-handshake המלא יקר (הצפנה אסימטרית + סבבי תקשורת). כשלקוח חוזר לאותו שרת, אפשר לקצר:

1. הלקוח שולח ClientHello עם session id מהחיבור הקודם ו-c_{rnd} **חדש**.
2. אם השרת זוכר את ה-session, הוא מחזיר **אותו** id ו-s_{rnd} חדש. אם לא, הוא מחזיר id אחר, ומתבצע handshake מלא.
3. שני הצדדים זוכרים את k_{ms}, ולכן מדלגים על ClientKeyExchange (בלי הצפנה אסימטרית) וגוזרים מפתחות **חדשים** מ-k_{ms} ומה-randoms החדשים.
4. Finished כרגיל.

במבחן: מתי זה יתגלה?

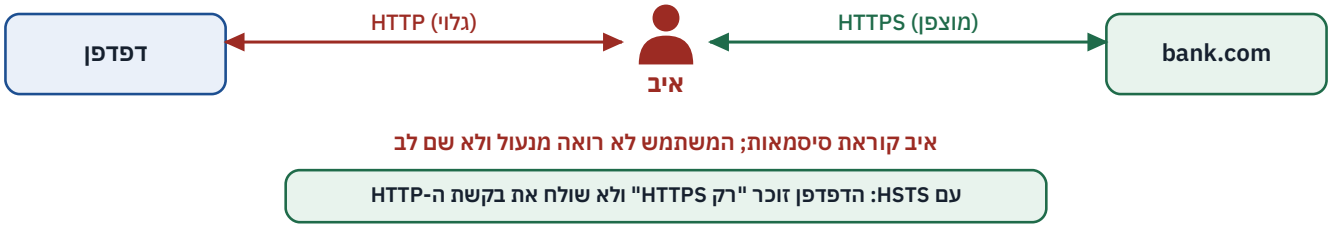
- MitM שינה תו ב-ClientHello: **מיד אחרי שה-ClientFinished מגיע לשרת**.
- MitM החליף את תעודת השרת בתעודה **חוקית של דומיין אחר** (עם שרשרת תקינה): **מיד אחרי שה-ServerHello מגיע ללקוח**, כי ה-subject לא מתאים.
- "ב-ClientHello השרת מעביר ללקוח את ה-cipher suite": שגוי. ClientHello נשלח **מהלקוח**.
- "אחרי ClientKeyExchange ה-handshake מסתיים": שגוי. צריך עוד את הודעות ה-Finished.
- אחרי ה-handshake ההודעות מוצפנות **סימטרית**, לא ב-public key של השרת. TLS משתמש בהצפנה סימטרית **וגם** בא-סימטרית.
- אם החומרה תבצע קריפטו "באפס זמן", HTTPS עדיין איטי מ-HTTP, כי ה-handshake מוסיף **סבבי תקשורת**.
- אפשר להקים TLS בלי דיפי-הלמן (RSA key exchange), והשרת **לא חייב** לאמת תעודת לקוח.

מעבר לקורס, בקצרה: forward secrecy

ב-RSA key exchange, מי שהקליט תעבורה וגונב **אחר כך** את המפתח הפרטי של השרת, מפענח את k_{ms} של כל החיבורים הישנים. ב-TLS 1.3 (RFC 8446) הוסר ה-RSA key exchange, וחובה להשתמש ב-(EC)DHE: דיפי-הלמן עם מספרים סודיים חדשים לכל חיבור, שהשרת חותם עליהם. המספרים נמחקים בסוף החיבור, כך שגניבת המפתח הפרטי לא חושפת את העבר. זו **forward secrecy**. ה-handshake גם מתקצר לסבב אחד (1-RTT).

HSTS (HTTP Strict Transport Security)

הבעיה: משתמש מקליד bank.com בלי `https://`. הדפדפן פונה ב-**HTTP**, והשרת מחזיר הפניה (redirect) ל-HTTPS. תוקף MitM מבצע **SSL stripping**: הוא לא מעביר את ההפניה. מול השרת הוא פותח חיבור HTTPS בשם המשתמש, ומול המשתמש הוא ממשיך ב-HTTP גלוי, ומעביר את הבקשות והתשובות ביניהם. המשתמש רואה את האתר האמיתי, אבל בלי מנעול, ואיב קוראת הכל. גם זו התקפת שנמוך: ממוצפן ללא מוצפן.



SSL stripping ו-HSTS. בלי HSTS, הבקשה הראשונה יוצאת ב-HTTP, ואיב "בולעת" את ההפניה ל-HTTPS: מול השרת היא מדברת HTTPS, ומול המשתמש HTTP גלוי. עם HSTS הדפדפן לא שולח HTTP בכלל.

הפתרון: השרת שולח header בשם `Strict-Transport-Security: max-age=...` (בדוגמה מההרצאה `max-age=10886400` שניות, כ-126 ימים). מאותו רגע ועד שה-max-age פג, הדפדפן פונה לדומיין **רק ב-HTTPS**, גם אם המשתמש מקליד `http://`, ולא שולח בכלל את בקשת ה-HTTP שאיב צריכה. החולשה: **TOFU** (Trust On First Use). הדפדפן לומד על HSTS רק אחרי הביקור הראשון, כך שהביקור הראשון עדיין חשוף.

HSTS Preload: בעל האתר מוסיף `preload` ל-header ורושם את הדומיין ב-hstspreload.org. הרשימה מובנית בתוך הדפדפנים עצמם, כך שגם דפדפן **שמעולם לא גלש לאתר** יפנה אליו רק ב-HTTPS, כבר מהפעם הראשונה.

סיכום הפרק

- TCP, ואז HTTP = TLS; HTTPS בתוך TLS. TLS נותן סודיות, שלמות ואימות שרת.
- ClientHello (suites, c_{rnd}) → ServerHello + תעודה (s_{rnd}) → אימות תעודה → ClientKeyExchange $E_{pk}(k_{ms})$ → גזירת מפתחות לכל כיוון → ServerFinished → ClientFinished.
- randoms: מפתח חדש בכל חיבור ומניעת replay. Finished: מניעת שנמוך. מפתח לכל כיוון: מניעת reflection.
- Resumption: אותו session id, randoms חדשים, בלי ClientKeyExchange.
- HSTS → SSL stripping (חולשה: TOFU) → HSTS Preload.

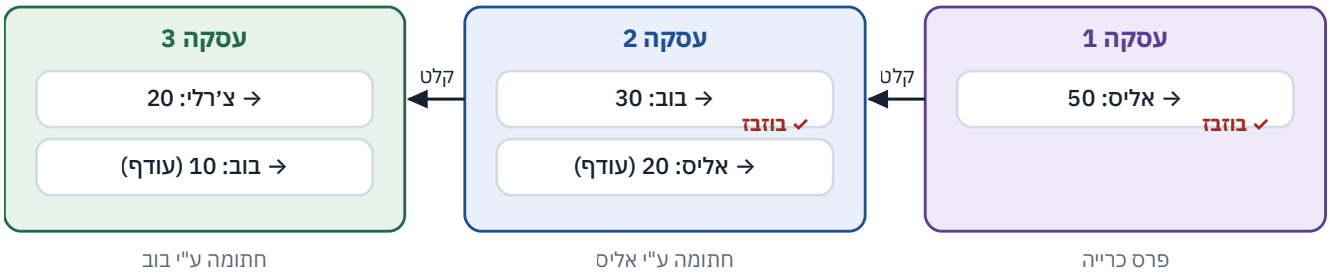
7 ביטקוין ובלוקצ'יין

ביטקוין (סאטושי נקמוטו, 2008) הוא מטבע **מבוזר**: אין בנק שמנהל את הספר, ואין מי שיקפיא חשבון או יבטל עסקה. הבעיה המרכזית של כסף דיגיטלי היא **double spending**, כלומר שימוש באותו מטבע פעמיים. ביטקוין פותר אותה בלי צד שלישי. 1 סאטושי = 10^{-8} ביטקוין.

ביטקוין (סאטושי נקמוטו, 2008) בונה מטבע בלי בנק מרכזי. כל הכלים מהפרקים הקודמים חוזרים כאן: חתימות (פרק 4) מוכיחות בעלות, hash (פרק 3) קושר את הבלוקים ומגן על שלמותם, ועץ מרקל מאפשר אימות יעיל. הפרק מראה איך הם מתחברים לספר חשבונות ציבורי שאי אפשר לזייף.

מפתחות, ארנקים ועסקאות

- **ארנק** = זוג מפתחות **א-סימטריים** (ECDSA). הכתובת נגזרת מ-hash של המפתח הפומבי; כדי לבזבז חותמים עם המפתח הפרטי. אין שימוש ב-OTP בארנקים. ארנק חם מחובר לרשת, ארנק קר לא.
- **עסקה** מבזבזת פלטים של עסקאות קודמות ויוצרת פלטים חדשים. מאחר שכל ההיסטוריה ציבורית, כל מי שמחזיק את הבלוקצ'יין (למשל miner) יכול לחשב כמה ביטקוין יש לכל כתובת. הבעלים של הכתובות לא מזוהים, אבל העסקאות גלויות.
- כדי לבצע עסקאות **לא** צריך לכרות. מי שמאמת עסקאות ומכניס אותן לבלוק הם **הכורים**.

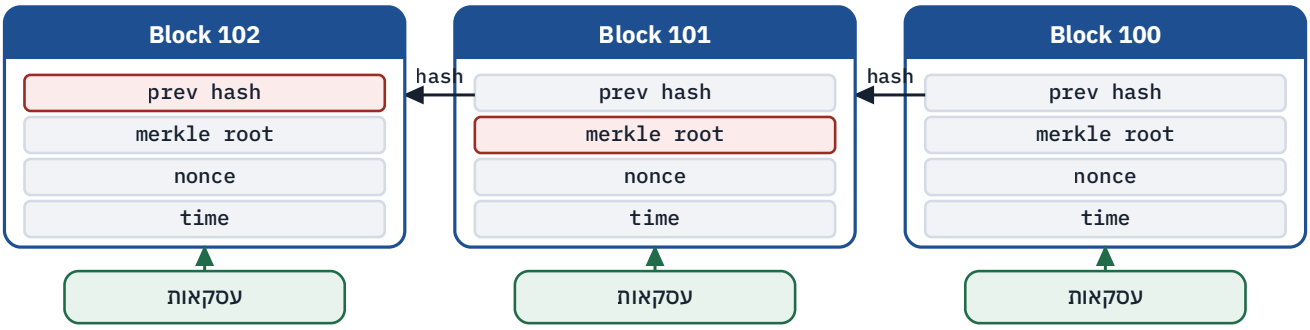


כמה יש לאליס? סוכמים את הפלטים שלה שעוד לא בוזבזו: 20

הספר (ledger) כרשימת עסקאות. אין "יתרות" שמורות בשום מקום: כל עסקה מבזבזת פלטים של עסקאות קודמות ויוצרת פלטים חדשים. פלט שכבר בוזבז מסומן, וניסיון לבזבז אותו שוב הוא double spending.

הבלוקצ'יין ועץ מרקל

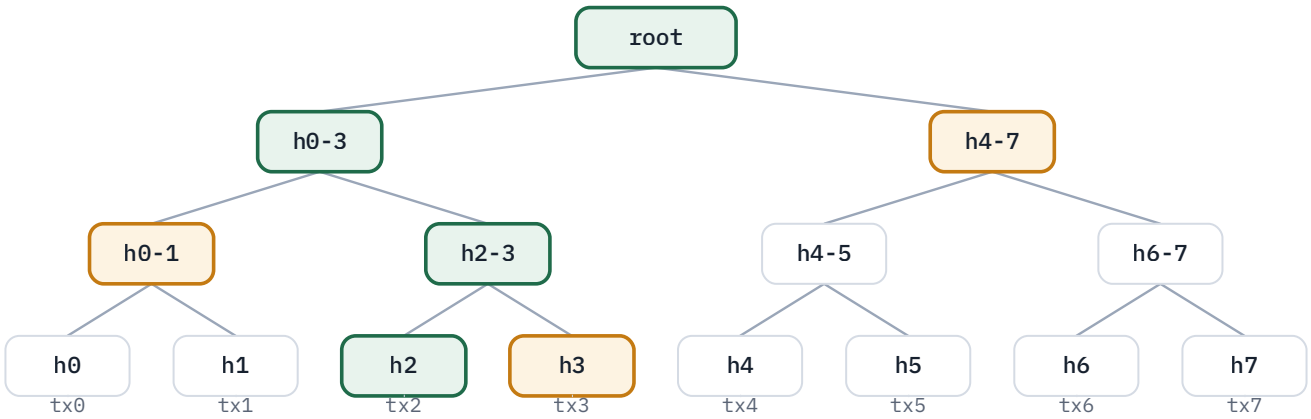
הספר (ledger) הוא שרשרת בלוקים. כל בלוק מכיל **prev hash** (hash של הבלוק הקודם), **שורש עץ מרקל** של העסקאות שלו, nonce וזמן. שינוי של בלוק ישן משנה את ה-hash שלו ושובר את כל השרשרת אחריו.



שינוי עסקה בבלוק 101 ← שורש מרקל חדש ← hash חדש ← prev hash של 102 לא מתאים

שרשרת בלוקים. כל כותרת מכילה את ה-hash של הכותרת הקודמת ואת שורש עץ המרקל של העסקאות שלה. תוקף ששינה עסקה בבלוק 101 משנה את שורש המרקל, ולכן את ה-hash של בלוק 101, ולכן את ה-prev hash שבבלוק 102, וכן הלאה עד הסוף.

עץ מרקל: עץ בינארי שהעלים שלו הם hash-ים של עסקאות, וכל צומת הוא $h(\text{שמאל} \,\|\, \text{ימין})$, עד השורש.

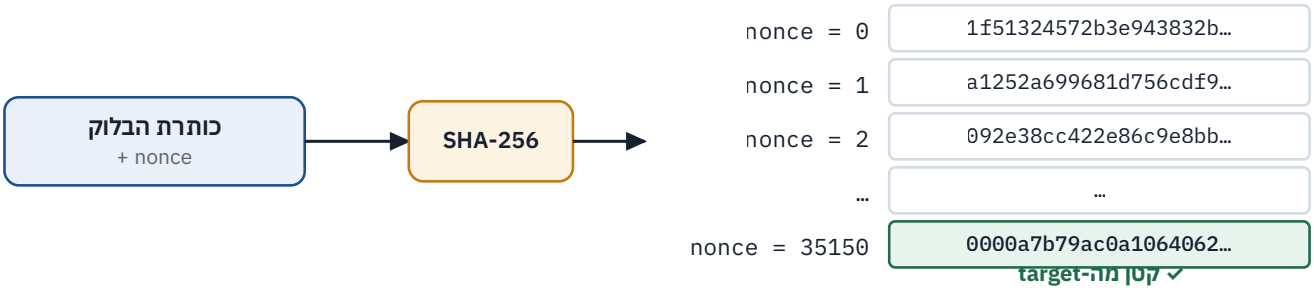


עץ מרקל והוכחת הכללה (הדוגמה מההרצאה). כדי להוכיח ש-h_2 בעץ מספיקים שלושה ערכים (בכתום): h_3, h_{0-1} ו-h_{4-7}. המאמת מחשב בעצמו את הצמתים הירוקים ומשווה לשורש. עם n עלים צריך $\log_2 n$ ערכים.

הוכחת הכללה (proof of inclusion): כדי להוכיח שעסקה בבלוק, מספיק לתת את ה-hash-ים של ה"אחים" לאורך המסלול לשורש, כלומר $\log_2(n)$ ערכים. לקוח קל (SPV) מאמת כך עסקה בלי להחזיק את כל העסקאות. עץ מרקל **לא** מספק פרטיות ולא הצפנה.

במבחן

1024 עסקאות בבלוק: לקוח קל צריך $\log_2 1024 = 10$ hash-ים. שינוי עלה אחד בעץ: משתנים **רק הצמתים במסלול מהעלה עד השורש (כולל)**.

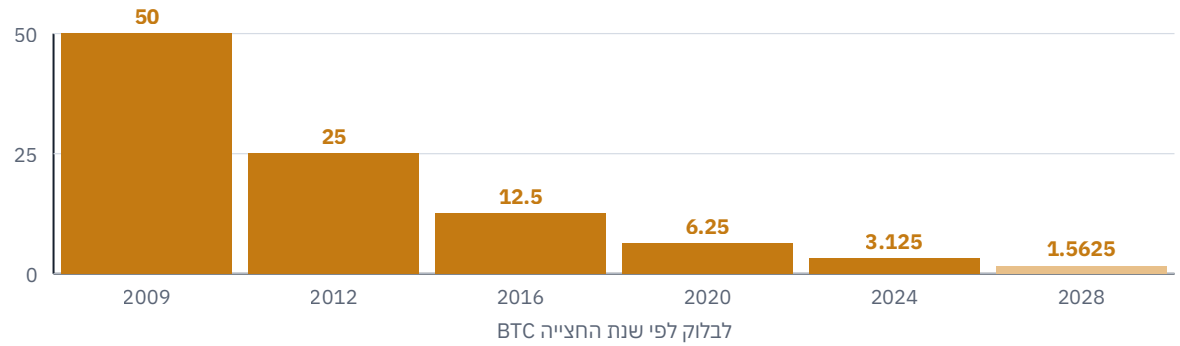


הוכחת עבודה בפועל (חישוב אמיתי). הכורה משנה את ה-nonce שוב ושוב ומחשב SHA-256 על כותרת הבלוק. במקרה הזה נדרשו 35,150 ניסיונות כדי לקבל hash שמתחיל בארבעה אפסים הקסדצימליים. כל מי שרוצה לבדוק מחשב hash אחד.

כרייה: Proof of Work

- כדי להוסיף בלוק, הכורה מחפש **nonce** שגורם ל-hash של כותרת הבלוק להתחיל במספיק אפסים (להיות קטן מ-target). הדרך היחידה היא ניסוי וטעייה. אימות הפתרון, לעומת זאת, דורש hash אחד.
- הקושי מתעדכן כל **2016 בלוקים** כדי לשמור על בלוק כל **~10 דקות**. הפרס נחצה כל **210,000 בלוקים** (כ-4 שנים). הכורה מקבל פרס בעסקה הראשונה בבלוק (coinbase) וגם את העמלות.
- **למה 10 דקות?** לוקח זמן עד שבלוק מתפשט ברשת (flooding via gossiping). אם בלוקים היו נמצאים כל 10 שניות כשההפצה לוקחת דקה, היו נוצרים המון fork-ים והכורים היו מבזבזים עבודה על שרשראות שיימחקו.
- **Mining pools**: היום כורים בעיקר בקבוצות שמחלקות ביניהן את מרחב החיפוש ואת הפרס. כורה בודד בבית ככל הנראה יפסיד כסף.

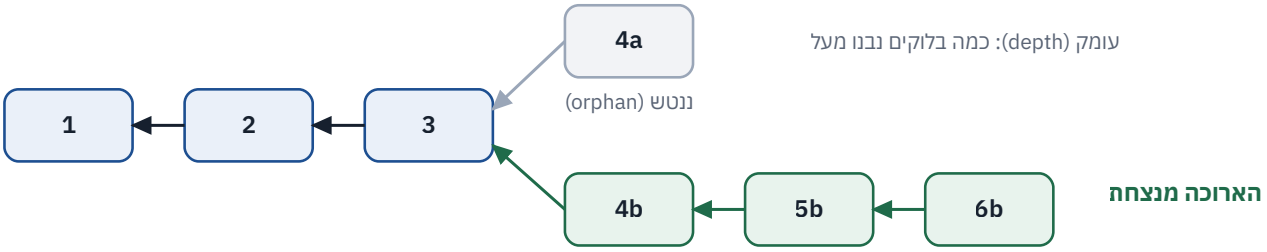
בדיקות שמחשב מבצע לפני שהוא מוסיף בלוק: ה-hash מתחיל במספיק אפסים; כל העסקאות חתומות כחוק; אין עסקאות סותרות בתוך הבלוק או מול השרשרת (double spending); ה-prev hash מצביע לסוף השרשרת. המחשב **לא** מוסיף חתימה משלו לבלוק.



פרס הבלוק נחצה כל 210,000 בלוקים (כ-4 שנים). סכום הסדרה `210,000 × 100 = 21,000,000` `210,000 × (50 + 25 + 12.5 + …)`, ולכן יהיו לכל היותר 21 מיליון ביטקוין. בסביבת 2140 הפרס יגיע לאפס, והכורים יחיו מעמלות.

קונצנזוס, Forks ו-Double Spending

- **כלל השרשרת הארוכה** (ליתר דיוק, זו עם הכי הרבה עבודה): כששני בלוקים נמצאים במקביל נוצר fork, וכל צומת ממשיך על השרשרת הארוכה ביותר שהוא מכיר. הענף הקצר ננטש.
- **מניעת double spending**: מחכים מספיק בלוקים (בדרך כלל 6) מעל העסקה (depth). כדי לשכתב היסטוריה, התוקף צריך לבנות שרשרת ארוכה יותר מכל הרשת, ואין לו מספיק כוח חישוב.
- אליס ששינתה בלוק בעותק המקומי שלה לא תצליח, **כל עוד השרשרת שלה לא ארוכה מהמקורית**. אין "בנקאי" שמאשר.
- **51%**: מי ששולט ברוב כוח החישוב יכול לקבוע אילו עסקאות ייכנסו, למנוע מעסקאות של משתמש מסוים להיכנס (exclusion) ולבצע double spending. הוא **לא** יכול לגנוב מטבעות של אחרים, כי אין לו את המפתחות שלהם.
- **Sybil attack**: יצירת המון זהויות מזויפות. PoW לא מושפע מזה, כי ההשפעה תלויה בכוח חישוב ולא במספר זהויות.
- לקוח "רזה" ששומר רק את העסקאות שלא בוזבזו ואת הבלוק האחרון **לא יכול להתמודד בעצמו עם fork-ים**, כי אין לו את ההיסטוריה כדי לחזור אחורה.



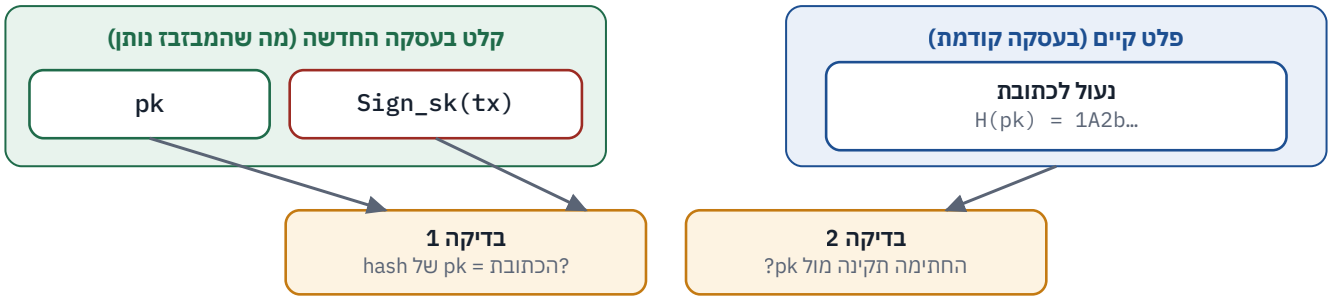
Fork וכלל השרשרת הארוכה. שני כורים מצאו בלוק 4 כמעט באותו זמן. כל צומת ממשיך על הענף שהגיע אליו ראשון, עד שענף אחד מתארך. הענף הקצר ננטש, והעסקאות שהיו רק בו חוזרות להמתנה. לכן מחכים כ-6 בלוקים לפני שסומכים על עסקה.

> **מלכודת**
> הצעה "לכרות רק כשיש x עסקאות חדשות כדי לחסוך חשמל, וזה לא מאפשר double spending": הטענה **שגויה**. בזמן שהכורים הישרים מחכים, תוקף ממשיך לכרות שרשרת סודית וצובר יתרון.

Soft fork לעומת Hard fork

Soft fork	Hard fork
הכללים **מוחמרים**: בלוקים חדשים עדיין חוקיים בעיני צמתים ישנים. דוגמה: הקטנת בלוק מ-2MB ל-1MB	הכללים **מורחבים או משתנים**: צמתים ישנים **דוחים** בלוקים חדשים, והרשת מתפצלת לשני מטבעות. דוגמאות: הגדלת בלוק (Bitcoin Cash, 2017), **החלפת פונקציית ה-hash של ה-PoW**

איך מוכיחים בעלות על מטבע (P2PKH)



איך מוכיחים בעלות על פלט. הפלט נעול ל-hash של מפתח פומבי (הכתובת). כדי לבזבז, מציגים את המפתח הפומבי ואת החתימה. הכורה בודק שתי התאמות.

פלט של עסקה "ננעל" ל-**hash של מפתח פומבי** (זו הכתובת). כדי לבזבז אותו, המבזבז חייב להציג שני דברים: **את המפתח הפומבי עצמו**, שה-hash שלו שווה לכתובת, ו**חתימה דיגיטלית על העסקה במפתח הפרטי** המתאים. הכורים בודקים קודם שה-hash של המפתח מתאים לכתובת, ואחר כך שהחתימה תקינה מול המפתח.

במבחן

מה המשתמש חייב לספק כדי לבזבז? **מפתח פומבי + חתימה במפתח הפרטי**. לא hash של המפתח (הוא כבר רשום בפלט), ולא "חתימה במפתח הפומבי" (אי אפשר לחתום בפומבי). בשאלה שמציגה סדרת פעולות, הסדר התקין: חתימה, מפתח פומבי, שכפול המפתח, חישוב hash שלו, השוואה ל-hash שבפלט, בדיקת החתימה.

עסקאות מתקדמות

- **Escrow / Multisig**: ארנק משותף לקונה, מוכר וצד שלישי מוסכם, שדורש **2 מתוך 3** חתימות. אם הכל תקין, קונה ומוכר חותמים **בלי** המתווך. בסכסוך, המתווך מכריע עם אחד הצדדים. לכן "המתווך חייב לאשר" ו"המוכר חייב לאשר" שגויים.



Escrow עם multisig של 2 מתוך 3. הכסף יושב בארנק משותף. כל שתי חתימות משחררות אותו. בעסקה תקינה הקונה והמוכר חותמים לבד; המתווך נכנס רק בסכסוך.

- **Micropayments**: אליס נועלת כסף בארנק משותף עם בוב, ומראש מקבלת עסקת החזר עם **LockTime** (עסקה שלא תיכנס לבלוק לפני זמן מסוים). כל תשלום קטן הוא עסקה חדשה שאליס חותמת. בוב מפרסם רק את האחרונה. אליס יודעת שבוב ישחרר את מה שמגיע לה, **כי אחרת, כשה-LockTime יגיע, כל הכסף יחזור אליה ובוב יפסיד**.
- **Proof of Stake**: הסיכוי לבחור את הבלוק הבא תלוי בכמות המטבעות שמחזיקים, לא בעבודה חישובית. חוסך חשמל.
- **Ethereum**: פלטפורמה לחוזים חכמים. **Namecoin**: בלוקצ'יין לניהול שמות דומיין במקום DNS. **BIP**: מסמך הצעת שיפור לביטקוין.

סיכום הפרק

- ארנק = זוג מפתחות; כתובת = hash של pk; מבזבזים בחתימה. הספר הוא שרשרת בלוקים עם prev hash ושורש מרקל.
- מרקל: הוכחת הכללה ב-$\log_2 n$ ערכים. PoW: מחפשים nonce, אימות ב-hash אחד; קושי כל 2016 בלוקים, 10 דקות, פרס נחצה כל 210,000.
- Fork: השרשרת הארוכה מנצחת, מחכים ~6 בלוקים. 51% שולט בקבלה אבל לא גונב מפתחות. Sybil לא עוזר מול PoW.
- P2PKH: מבזבזים עם pk + חתימה. Multisig 2-of-3 ל-escrow; LockTime למיקרו-תשלומים.

8 אבטחת Web

עד כה הגנו על ההעברה בין דפדפן לשרת. הפרק הזה עוסק בפרצות ב**אפליקציה** עצמה: הזרקת קוד למסד הנתונים (SQL injection), הרצת סקריפט של התוקף בדפדפן הקורבן (XSS), וניצול האמון שהדפדפן נותן לאתר (CSRF). בכל אחת, הרעיון החוזר הוא שקלט של התוקף מתפרש כ**קוד** במקום כ**נתון**.

SQL Injection

השרת בונה שאילתה למסד הנתונים על ידי **שרשור טקסט**: קטע קבוע, ואחריו מה שהמשתמש הקליד, בתוך גרשיים. אם הקלט עצמו מכיל **גרש**, הגרש סוגר את המחרוזת מוקדם, ומה שבא אחריו כבר לא נקרא כנתון אלא **כחלק מהלוגיקה של השאילתה**. כך התוקף משנה את מבנה השאילתה.

שרשור טקסט (פגיע)

```
WHERE user = '  x' OR 1=1 --  ' AND pass = '…'
```

הגרש של התוקף סוגר את המחרוזת מוקדם

-- הופך את בדיקת הסיסמה להערה

התנאי 1=1 OR תמיד אמת → ההתחברות עוברת בלי סיסמה

שאילתה עם פרמטרים (בטוח)

```
WHERE user = ?  AND pass = ?   ←  x' OR 1=1 --
```

נכנס כערך בלבד: מחפשים משתמש בשם הזה

הזרקת SQL: מה קורה לשאילתה. בשרשור, הקלט של המשתמש הופך לחלק מהטקסט של השאילתה. גרש בקלט סוגר את המחרוזת מוקדם, ומה שאחריו נקרא כקוד. בשאילתה עם פרמטרים המבנה קבוע מראש, והקלט תמיד נשאר נתון, גם אם יש בו גרשיים.

- **עקיפת התחברות:** סיסמה שמסתיימת בגרש ואחריו "או 1 שווה 1" הופכת את תנאי הבדיקה ל"שם משתמש וסיסמה תואמים, **או** 1=1". מאחר ש-AND מחושב לפני OR, התנאי תמיד אמת. דרך אחרת: גרש ואחריו סימן הערה (--), שמבטל את המשך השאילתה, כולל בדיקת הסיסמה.
- **מה לא עובד:** קלט בלי גרש נשאר בתוך המחרוזת ולא משנה את המבנה. קלט שמחבר תנאי **AND** שתמיד אמת לא עוקף כלום, כי הסיסמה עדיין נבדקת.
- **עוד יכולות:** הוספת שאילתה שנייה (למשל מחיקת טבלה), שליפה מטבלה אחרת (UNION), ועקיפת סינון שמבוסס על **רשימה שחורה** (קידוד התווים, או מילה שבנויה כך שאחרי מחיקה אחת של מילה אסורה נשארת המילה האסורה).

Blind SQL injection: התוקף לא רואה את תוצאת השאילתה, אבל שואל שאלות כן/לא ומסיק את התשובה בעקיפין: האם הדף מוצג או לא, או כמה זמן לוקח לשרת לענות. כך הוא מגלה סיסמה תו אחר תו.

הגנות: שאילתות עם פרמטרים (prepared statements, stored procedures): מבנה השאילתה מוגדר מראש, והקלט תמיד נשאר נתון ולעולם לא קוד. בנוסף: ORM, ולידציה **בצד השרת** (חובה; בצד הלקוח רק נוחות), escaping וקידוד, ואי-הצגת הודעות שגיאה למשתמש. **IDS לא מונע injection.** הודעות כניסה צריכות להיות כלליות ("שם משתמש **או** סיסמה שגויים").

עוגיות ו-Same-Origin Policy

עוגייה: מחרוזת שהשרת קובע והדפדפן מצרף **אוטומטית** לכל בקשה לאותו אתר, למשל session token.



הדפדפן מצרף את העוגייה גם לבקשה שאתר אחר יזם (זה הבסיס ל-CSRF)

עוגיית session. אחרי ההתחברות השרת שולח מזהה אקראי, והדפדפן מצרף אותו אוטומטית לכל בקשה לאותו אתר. מי שמחזיק את העוגייה הזו הוא, מבחינת השרת, המשתמש.

SOP: מנגנון **בדפדפן**. סקריפט מ-origin אחד לא יכול **לקרוא** מידע של origin אחר. **Origin = פרוטוקול + דומיין מלא + פורט.** הנתיב לא חלק מה-origin.



✓	אותו origin (רק נתיב שונה)	`https://www.foo.com/bob`
✗	פרוטוקול שונה	`http://www.foo.com`
✗	דומיין שונה (תת-דומיין)	`https://mail.foo.com`
✗	פורט שונה	`https://www.foo.com:8080`

Origin = פרוטוקול + דומיין מלא + פורט. הנתיב לא משנה. מספיק שאחד משלושת הרכיבים שונה, וזה origin אחר.

SOP חוסם	SOP לא חוסם
סקריפט מ-foo.com שקורא תוכן של תשובה מ-bar.com, או ניגש לתוכן של iframe מ-origin אחר	טעינת תמונה או סקריפט מדומיין אחר; שליחת טופס ל-bar.com (ולכן **לא מונע CSRF**); סקריפט שהוזרק ל-foo.com ושולח עוגיות ל-bar.com

מלכודת

`http://foo.com/~alice` ו-`http://foo.com/~bob` הם **אותו origin**. לכן iframe באתר של בוב **יכול** לקרוא את האתר של אליס, ו-SOP לא יחסום. הטענה שהוא "יגלה ש-SOP ימנע" שגויה.

XSS (Cross-Site Scripting)

התוקף גורם לקוד שלו לרוץ בדפדפן של הקורבן **כאילו הגיע מהאתר הפגיע**, וכך עוקף את SOP: גונב עוגיות או tokens, מציג תוכן מזויף ופועל בשם המשתמש. **הפרצה נמצאת בקוד האתר הפגיע**, לא בדפדפן.



Stored XSS. התוקף שומר תגובה שמכילה סקריפט. כל מי שצופה בדף מקבל אותו כחלק מתשובת השרת, ולכן הדפדפן מריץ אותו בתוך ה-origin של האתר, ומשם הוא קורא את העוגייה ושולח אותה החוצה.

סוג	מאיפה מגיע הקוד	איפה החולשה
Stored / Persistent	נשמר ב-DB (תגובה, פרופיל) ומוחזר לכל מי שגולש	שרת
Reflected	פרמטר בבקשה (קישור זדוני) שהשרת משקף לתוך הדף	שרת
DOM-based	קוד JavaScript של הדף קורא פרמטר (למשל מה-URL) בצורה לא בטוחה ומריץ אותו	**קוד שרץ בדפדפן** (צד לקוח)

ב-stored וב-reflected הקוד הזדוני חוזר כחלק מתשובת השרת. ניצול SQL injection כדי לשתול סקריפט ב-DB (ומשם XSS) הוא תרחיש הגיוני.

הגנות: חיטוי וקידוד פלט (sanitization, encoding; בצד השרת), עוגיות **HttpOnly** (JavaScript לא יכול לקרוא אותן, הן רק מצורפות לבקשות), ו-**CSP** (Content-Security-Policy: header שמגדיר לדפדפן מאילו origins מותר לטעון ולהריץ סקריפטים ומשאבים; כולל גם frame-ancestors).

במבחן
שם שמוצג כ- ' במקום גרש: האתר משתמש ב**קידוד (encoding)** כדי למנוע injection. הקידוד הוצג פעמיים או במקום הלא נכון.

CSRF / XSRF (Cross-Site Request Forgery)

אתר זדוני גורם לדפדפן של הקורבן לשלוח בקשה לאתר שבו הקורבן מחובר. הדפדפן **מצרף אוטומטית את העוגיות**, והשרת **לא יודע** שהבקשה לא באמת של הקורבן.



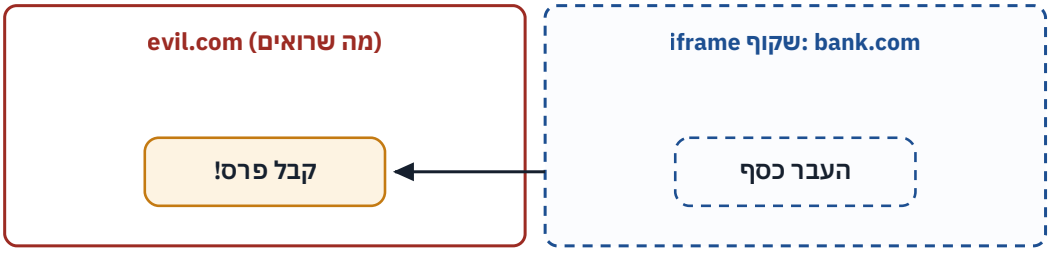
CSRF. הקורבן מחובר לבנק, ומבקר באתר של התוקף. האתר גורם לדפדפן לשלוח בקשה לבנק, והדפדפן מצרף אוטומטית את עוגיית ה-session. הבנק רואה בקשה שנראית תקינה. ה-token האקראי שהבנק מטמיע בטופס האמיתי חסר בבקשה המזויפת.

דוגמה קלאסית: שינוי מצב דרך **GET** (`/change-password?user=alice&password=1234`, או `foo.com/Bob?m=msg`). מספיק שהקורבן יבקר באתר של התוקף.

הגנות:

- **Anti-CSRF token**: ערך אקראי שהשרת מטמיע בטופס (שדה hidden) ובודק בכל בקשה. התוקף לא יכול לקרוא אותו בגלל SOP. **שקוף למשתמש**, שלא מקליד אותו ידנית.
- **cookies SameSite** (Strict/Lax): הדפדפן לא מצרף את העוגייה לבקשות שמקורן באתר אחר.
- בדיקת Referer/Origin header, custom headers, ו-CAPTCHA לפעולות רגישות.

מלכודות
- SOP **לא מונע** CSRF. ב-**stored XSS** צירוף העוגיות **לא** מהותי (זה מהותי ב-CSRF).
- XSS **שובר** הגנת CSRF: הסקריפט רץ מתוך ה-origin, קורא את ה-token מהטופס ושולח בקשה תקינה. "ה-token לא יכול להיגנב כי הוא ב-POST" שגוי.
- התקפת view-source ב-iframe, שבה המשתמש מתבקש להקליד טקסט מאזור ב-HTML: עוקפת **anti-CSRF token**.



שתי השכבות מונחות זו על זו; הלחיצה נוחתת על הכפתור האמיתי

Clickjacking. האתר האמיתי נטען ב-iframe שקוף, ממוקם כך שהכפתור שלו יושב בדיוק מעל כפתור תמים באתר של התוקף. המשתמש חושב שהוא לוחץ על "קבל פרס", ובפועל לוחץ על "העבר כסף" באתר האמיתי. הגנה: X-Frame-Options או CSP frame-ancestors.

Clickjacking

האתר הנתקף נטען ב-iframe שקוף מעל כפתור תמים, והמשתמש לוחץ בלי לדעת על כפתור באתר האמיתי. הגנה: `X-Frame-Options: SAMEORIGIN/DENY` או `CSP frame-ancestors`. X-Frame-Options **לא** נועד להגנה מ-SQL injection.

סיכום הפרק

- SQL injection: קלט בורח מהמחרוזת ונעשה קוד. הגנה: שאילתות עם פרמטרים (הקלט תמיד נתון).
- Origin = פרוטוקול + דומיין + פורט. SOP חוסם קריאה חוצת-origin, לא טעינה ולא שליחת טופס (ולכן לא מונע CSRF).
- XSS: סקריפט של התוקף רץ ב-origin של האתר. Stored/Reflected (שרת), DOM (לקוח). הגנה: קידוד, CSP, HttpOnly.
- CSRF: אתר זר שולח בקשה, הדפדפן מצרף עוגייה. הגנה: anti-CSRF token, SameSite. XSS שובר את ה-token.
- Clickjacking: iframe שקוף. הגנה: X-Frame-Options / CSP frame-ancestors.

9 אבטחת רשת: ARP, DNS, DoS וחומות אש

הפרק הזה חוזר לשכבות הנמוכות של הרשת (פרק 1) ומראה איך תוקף משתלט עליהן: הרעלת ARP שהופכת off-path ל-MitM, הרעלת DNS שמפנה גולשים לשרת התוקף, התקפות מניעת שירות, והכלים שמגינים: חומות אש, IDS ו-anonymizers.

הרעלת ARP (ARP poisoning / spoofing)

ARP ממפה כתובת IP לכתובת MAC ברשת המקומית: שואלים ב-broadcast "למי שייך 192.168.0.2?" ושומרים את התשובה בטבלת ARP (cache). שתי חולשות: ARP **מקבל תשובות שלא ביקש**, ו**התשובה האחרונה דורסת** את הקודמת. אין שום אימות.

ההתקפה: התוקף (MAC ...:66:66) שולח לאליס "ה-IP של בוב נמצא ב-...:66:66", ולבוב את אותו הדבר על אליס. מעכשיו אליס שולחת חבילות ל-**IP של בוב** (192.168.0.2) אבל ל-**MAC של התוקף** (66:66:66:66:66:66). ה-switch מנתב לפי MAC בלבד, והתוקף מעביר הלאה. כך תוקף **off-path** ברשת המקומית הופך ל-**MitM**.

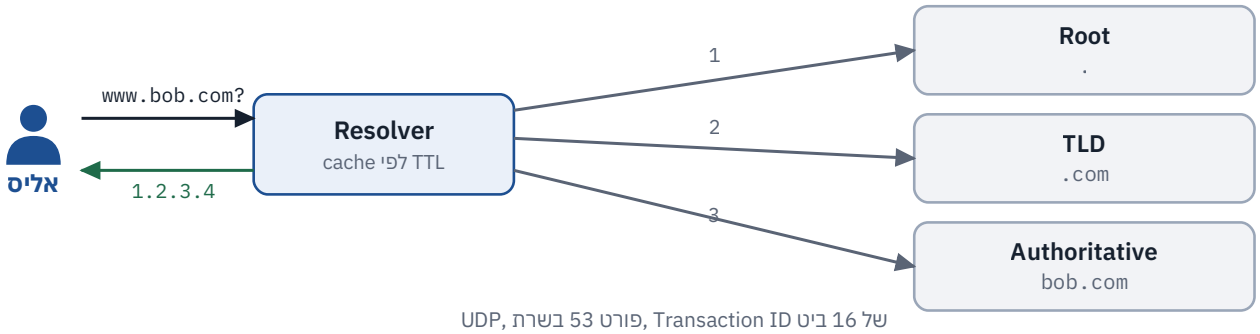
זיהוי והגנה: הרבה כתובות IP עם אותו MAC מעידות על התקפה; רשומות ARP **סטטיות** (מיפוי קבוע של IP ל-MAC); כלים על ה-switch (Dynamic ARP Inspection) ועל המחשב. מניעה **לא** אומרת להפסיק להשתמש בכתובות MAC.



הרעלת ARP. איב שולחת לאליס ולבוב תשובות ARP שלא ביקשו. ARP מקבל אותן (ותשובה אחרונה דורסת), ועכשיו כל אחד ממפה את ה-IP של השני ל-MAC של איב. ה-Switch מנתב לפי MAC, ולכן כל התעבורה עוברת דרך איב.

DNS והרעלת DNS

DNS מתרגם שם (www.bob.com) לכתובת IP. הלקוח שואל resolver, שפונה ל-root, משם ל-TLD ומשם לשרת המוסמך, ושומר תשובות ב-cache לפי TTL. DNS רץ מעל **UDP** (פורט 53 בשרת), ולכן קל לזייף כתובת מקור. כל שאילתה מזוהה ב-**Transaction ID** של 16 ביט, והתשובה חייבת להגיע לפורט שממנו נשלחה השאלה. מבנה התשובה: Query, Answer, Authority, Additional.



איך DNS עונה על שאלה. ה-resolver (בדרך כלל של ספק האינטרנט) שואל את ה-root, משם את שרת ה-.com, ומשם את השרת המוסמך של bob.com. התשובה נשמרת ב-cache למשך ה-TTL, ושאלות הבאות נענות ממנו בלי לשאול שוב.

המטרה: לשתול ב-cache של ה-resolver מיפוי שקרי (www.bob.com → 6.6.6.6), כך שכל לקוחותיו יגיעו לתוקף.

תוקף	מה צריך כדי להצליח
MitM (בין resolver ל-root)	לא צריך לנחש כלום: רואה את השאלה ועונה בעצמו (כשאין מידע ב-cache)
מאזין	רואה את ה-TXID והפורט, אבל לא יכול לחסום את התשובה האמיתית. **ב-DNS התשובה הראשונה קובעת** (עד שה-TTL פג), ולכן הוא צריך לנצח במרוץ, ו**לא תמיד יצליח**
Off-path	לנחש TXID (2^{16} : 1) ואת פורט היעד, ולענות לפני השרת האמיתי. לפי פרדוקס יום ההולדת, כ-$\sqrt{(2^{16})} = 256$ ניסיונות נותנים סיכוי של כ-50%

ARP מול DNS

ב-**ARP התשובה האחרונה דורסת**; ב-**DNS התשובה הראשונה קובעת** ולא נדרסת למשך ה-TTL. זו טענה שחוזרת שוב ושוב.

כלל Bailiwick: resolver מקבל רשומות רק לתחום שהשרת העונה מוסמך עליו. אם ns.evil.com מצרף בשדה Additional כתובת ל-google.com, הרשומה נזרקת.

התקפת קמינסקי: הבעיה של off-path היא שאחרי ניחוש שנכשל התשובה נשמרת ב-cache, וצריך לחכות TTL. קמינסקי שואל על תתי-דומיינים **שלא קיימים** (`1.foo.com, 2.foo.com`...). אין להם cache, ולכן כל שאלה פותחת הזדמנות חדשה ללא הגבלה. בתשובה המזויפת התוקף מצרף ב-Authority/Additional את **השרת המוסמך של כל foo.com** עם ה-IP שלו (מותר לפי bailiwick), ומשתלט על הדומיין כולו. זו התקפת הרעלה, **לא התקפת DoS**.

הגנות: פורט מקור אקראי ($2^{16} \cdot 2^{16} = 2^{32}$ אפשרויות); **0x20 encoding** (אותיות גדולות וקטנות אקראיות בשם: `wWw.GoOGle.cOm`, והתשובה חייבת להחזיר בדיוק); צמצום חלון ההזדמנויות; **DNSSEC** (תשובות חתומות). **IDS לא מונע** הרעלת DNS, ו**IP spoofing** היא הטכניקה שבה ההתקפה משתמשת.

מניעת שירות: DoS ו-DDoS

המטרה: למנוע מהשרת לשרת לקוחות לגיטימיים. ב-**DDoS** ההתקפה מגיעה ממחשבים רבים (botnet), ולכן קשה להבדיל בין תוקף ללקוח וחסימת IP לא עוזרת. **כוח ההמונים** הוא הכוח המניע. DoS **לא** מוגבל לשכבות התחתונות, וגם לא ראשי תיבות של Disk on Stack.

משפחה	דוגמאות
ניצול באג (abuse)	Ping of Death, Land: חבילה אחת או בודדות שמקריסות את המערכת
התשת משאבים	SYN flood (זיכרון), HTTP/UDP flood, **Slowloris**: פותח הרבה חיבורי HTTP ומחזיק אותם פתוחים כמה שיותר זמן עם בקשות חלקיות (low-rate)
התשת רוחב פס	**Reflection**: בקשות עם IP מקור מזויף של הקורבן, והתשובות הולכות אליו. **Amplification**: התשובה גדולה מהבקשה (DNS amplification דרך open resolver)

SYN flood: התוקף שולח המון SYN ולא משלים את ה-handshake. השרת שומר מצב (TCB) לכל חיבור חצי-פתוח עד שהזיכרון נגמר. המטרה היא זיכרון השרת, לא רוחב הפס. התוקף **מזייף כתובות מקור, ובעיקר כאלה שלא עונות**: כך אף אחד לא שולח **RST** על ה-SYN+ACK (שהיה משחרר את המשאב), וחסימת כתובות ספציפיות לא עוזרת.

SYN cookies: תחת עומס השרת **לא שומר מצב**. הוא מקודד את פרטי החיבור במספר הסידורי (sequence number) של ה-SYN+ACK באמצעות hash עם סוד, ומשחזר אותם רק כשמגיע ACK תקין. כך עדיין אפשר לשרת לקוחות לגיטימיים.

במבחן

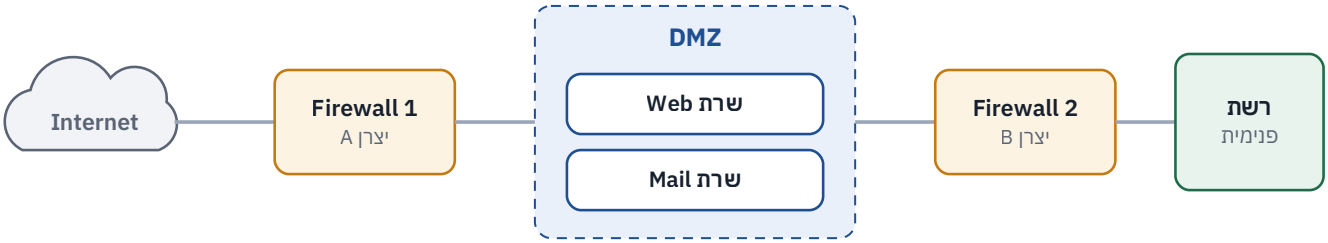
ב-**HTTP flood** מ-botnet החיבורים אמיתיים, ולכן **Source IP ו-Source Port שונים**, ו-**Destination IP ו-Destination Port זהים** (השרת, פורט 80/443).

Botnets, סריקות וכלים

- **Botnet**: Bot master שולט בשרתי **C&C** (Command & Control), והם שולטים בבוטים. כדי לנטרל, מפילים את ה-C&C.
  שימושים: DDoS, ספאם ופישינג, הונאת קליקים על פרסומות, קישורים מזויפים לקידום, כרייה. **Mirai**: botnet של מכשירי IoT שנפרצו בסיסמאות ברירת מחדל.

- **סריקות**: אנכית (כל הפורטים של כתובת אחת) או רוחבית (פורט מסוים בהרבה כתובות). **nmap** מגלה פורטים ושירותים פתוחים, ומסייע גם למגן לזהות חולשות. **Metasploit** הוא **כלי ניצול חולשות**, **לא** חומת אש. Hail Mary מנסה את כל ה-exploits בבת אחת. Kali Linux היא הפצה עם כלים אלה.
- **ערוץ צד של IP-ID**: אם מערכת ההפעלה של V מעלה מונה גלובלי ב-1 לכל חבילה, התוקף שולח ping בתחילת פרק זמן ובסופו. הפרש גדול מ-1 ב-IP-ID אומר ש-V שלח חבילות לאחרים בינתיים.

חומות אש (Firewalls)



חלוקת רשת ארגונית עם DMZ ושתי חומות אש. השרתים שפונים אליהם מבחוץ יושבים ב-DMZ. הרשת הפנימית מופרדת מה-DMZ בחומה נוספת, רצוי של יצרן אחר, כך שחולשה בחומה אחת לא פותחת את כל הרשת.

סוג	איך מחליט	מגבלה
Packet filter, stateless	כל חבילה בנפרד, לפי IP, פורט, פרוטוקול וכיוון	לא רואה שכבת אפליקציה (תוקף מפצל "ipt"+"scr"), לא זוכר הקשר
Stateful	זוכר חיבורים: מאשר תשובה רק לחיבור שנפתח מבפנים	עדיין לא מבין תוכן
Application layer / WAF	בודק תוכן (HTTP), למשל לפי רשימת OWASP Top 10	כבד ואיטי; לכן ממקמים אותו אחרי packet filter

Whitelist: ברירת המחדל **drop**, ועוברת רק חבילה שתואמת חוק Allow. זהיר יותר. **Blacklist**: ברירת המחדל לעבור, ונחסם רק מה שתואם חוק חסימה.

במבחן

- חומת אש שלא עובדת בשכבת האפליקציה **כן** מגינה מפני **spoofing**: חבילה מבחוץ עם כתובת מקור פנימית נזרקת. היא **לא** מגינה מפני אימייל עם קוד זדוני, חולשה במימוש של החומה עצמה או תוקף שכבר בפנים.
- חומה שחוסמת אוטומטית כתובות עם הרבה תעבורה **עלולה לחסום מחשבים לגיטימיים**. מחשב זדוני בפנים **יכול** לזייף כתובות מקור ולגרום לחסימת אחרים.

מלכודת חוקי ה-SSH

החוקים מאפשרים: (1) נכנס TCP לפורט 22 פנימי; (2) יוצא **מפורט מקור 22**; (3) יוצא לפורט 22 חיצוני; (4) נכנס **מפורט מקור 22** לכל פורט פנימי; (5) Drop. חוק 4 נועד לתשובות של שרתי SSH חיצוניים, אבל תוקף מבחוץ ששולח **מפורט מקור 22** לפורט 8080 עובר אותו ו**יכול** להקים חיבור לשרת הפרוקסי. הוספת עמודת **flags** (לאפשר בחוק 4 רק חבילות עם ACK) הייתה משפרת. הטבלה היא whitelist, ואכן מאפשרת SSH בשני הכיוונים.

- **DMZ**: תת-רשת נפרדת לשרתים שפונים אליהם מבחוץ (web, mail), מופרדת מהרשת הפנימית. נהוג להשתמש בשתי חומות אש, רצוי של יצרנים שונים. DMZ **לא** נועד למצב "שבו אי אפשר להשתמש בחומת אש".
- **IDS** (Intrusion Detection System): מערכת **פסיבית** שמזהה תבניות ואנומליות ו**מתריעה**. יכולה לעבוד offline על עותק של התעבורה, ולכן **לא מונעת**. **IPS**: מזהה **וגם** חוסם. **Honeypot**: מחשב שאף אחד לא אמור לפנות אליו, ולכן כל פנייה אליו חשודה. תוקפים מתחמקים בסריקות איטיות, obfuscation, פיצול חבילות ושינוי אותיות (`sCrIpT`).
- **SSH tunnel**: עוקף הגבלות גלישה דרך מחשב חיצוני שמשמש כ-proxy. **VPN**: חיבור מוצפן לשרת שגולש בשמך; ספק ה-VPN יודע לאן גלשת. **TOR**: ניתוב דרך 3 צמתים אקראיים עם הצפנה בשכבות ("בצל"); אף צומת לא רואה גם מקור וגם יעד. **Mix net**: מערבב הודעות ומשהה אותן כדי לשבור קשר תזמון. איטי, מתאים לדואר.

סיכום הפרק

- ARP: תשובה אחרונה דורסת ← off-path נעשה MitM. DNS: תשובה ראשונה קובעת (TTL); off-path מנחש TXID, קמינסקי פותח חלונות בתתי-דומיין. הגנות: פורט אקראי, 0x20, DNSSEC.
- DoS: ניצול באג / התשת משאבים (SYN flood, Slowloris) / רוחב פס (reflection+amplification). SYN cookies: לא שומרים מצב.
- Firewall: stateless (פר חבילה), stateful (זוכר חיבור), application/WAF (תוכן). whitelist בטוח מ-DMZ + IDS/blacklist. IPS + honeypot.
- אנונימיות: VPN (הספק יודע), TOR (3 צמתים, בצל), mix net (מערבב, איטי).

10 טענות שחוזרות במבחנים

אלה הטענות שמופיעות שוב ושוב, לפעמים כתשובה הנכונה ולפעמים כמסיח. מי שמזהה אותן מיד חוסך זמן ומונע טעויות.

✓ נכון

- MAC שמיושם נכון מספק אמינות מידע (integrity).
- בעיה גדולה ב-OTP היא העברת המפתחות.
- ב-TLS משתמשים בשרשרת אמון, וגם בהצפנה סימטרית וגם בא-סימטרית.
- ב-TLS משתמשים במפתחות שונים לכל כיוון (מונע reflection).
- CSP מגדיר מאילו כתובות מותר לטעון ולהריץ משאבים.
- SameSite=Strict מסייע נגד CSRF; SOP לא מונע CSRF.
- ב-DOM-based XSS הקוד הפגיע רץ בדפדפן.
- בביטקוין משתמשים במפתחות א-סימטריים, בעץ מרקל וב-Proof of Work.
- Root CA חותם על המפתח הפומבי של intermediate CA.
- CT מוסיף יומן ציבורי שבעזרתו מזהים הנפקה זדונית.
- OCSP stapling: השרת שולח סטטוס חתום ומעודכן, והלקוח לא פונה ל-CA.
- תוקף off-path מספיק כדי להרעיל ARP.
- הרעלת ARP מעניקה לתוקף יכולות MitM.
- ARP: האחרונה דורסת. DNS: הראשונה קובעת.
- פורט מקור אקראי ב-DNS מקשה על ניחוש.
- Slowloris מחזיק הרבה חיבורי HTTP פתוחים כמה שיותר זמן.
- Mining pools הן היום שיטת הכרייה הנפוצה.
- פרדוקס יום ההולדת הוא עיקרון כללי, לא ייחודי לקמינסקי.
- nmap יכול לסייע לזהות חולשות.

× לא נכון

- הגדרה נכונה של IDS מונעת הרעלת DNS.
- הרעלת DNS מחייבת יכולות MitM.
- Metasploit הוא חומת אש אפליקטיבית.
- x-frame-options מיועד להגנה מפני SQL injection.
- Anti-CSRF token דורש מהמשתמש להקליד קוד ידנית.
- ב-OTP הבטיחות תלויה בטקסט שמצפינים.
- בעיה גדולה ב-RSA / Diffie-Hellman / public key היא העברת המפתחות.
- OCSP stapling יעיל בזכות עץ מרקל.
- SCT נועד לפתור בעיית פרטיות של OCSP.
- CA חותם על מפתחות שה-CA יצר עבור השרת.
- שרת אינטרנט מייצר תעודה לכל חיבור TLS.
- הקמת TLS מחייבת תעודה רק בין שני שרתים.
- ב-escrow המתווך (או המוכר) חייב לאשר כל עסקה.
- צריך לכרות ביטקוין כדי לבצע עסקאות.
- בביטקוין משתמשים ב-OTP לייצור ארנקים.
- עץ מרקל מבטיח פרטיות של עסקאות.
- Syn cookies מכריחים את השרת לשמור TCB מה-SYN הראשון.
- אי אפשר למנוע התקפות שנמוך נגד switch.
- התקפת קמינסקי היא בעיקרה DoS.
- DoS נעשה רק בשכבות הפיזית, הלינק, התעבורה והרשת.
- מטרת SYN flood היא להעמיס על רוחב הפס.
- אפשר להשתמש בבוטים רק ל-DDoS.

אסטרטגיה לשאלת "איזו טענה אינה נכונה"

מחפשים את המילים המוחלטות ("תמיד", "אך ורק", "חייב", "לחלוטין") ואת ערבוב המושגים (OCSP עם miners, CT עם cookie, עץ מרקל עם OCSP). כמעט תמיד הן מסמנות את המסיח.

11 דף נוסחאות ודוגמאות פתורות

צפנים קלאסיים

הזזה (מספור a = 0 … z = 25):

$c = (m + k) \bmod 26 \;;\; m = (c - k) \bmod 26$

קיסר: $k = 3$

Vigenère: $c_i = (m_i + k_{i \bmod |k|}) \bmod 26$

מרחב מפתחות (הזזה · החלפה · החלפה כשמיפוי אחד ידוע):

```
shift 25 · subst 26! · one pair known
25!
```

XOR ו-OTP

$c = m \oplus k \cdot m = c \oplus k$

$x \oplus x = 0 \cdot x \oplus 0 = x \cdot A \oplus B \oplus B = A$

Two-time pad: $c_1 \oplus c_2 = m_1 \oplus m_2$

Stream: $c = m \oplus \mathrm{PRG}(k)$

Modes of operation

ECB: $C_i = E_k(P_i)$

CBC: $C_i = E_k(C_{i-1} \oplus P_i),\ C_0 = IV$

3DES (EDE): $C = E_{K3}(D_{K2}(E_{K1}(m)))$

DES: בלוק 64 ביט, מפתח 56 · AES: בלוק 128

Hash וסיסמאות

חד-כיווניות: נתון d, קשה למצוא m כך ש-$h(m) = d$

Second pre-image: נתון m_0, קשה למצוא $m_1 \neq m_0$ עם hash זהה

Collision: קשה למצוא זוג כלשהו $m_0 \neq m_1$ עם hash זהה

שמירת סיסמה:

```
user, salt, h(pass || salt || pepper)
```

דיפי-הלמן

$A = g^a \bmod n \cdot B = g^b \bmod n$

$K = B^a = A^b = g^{ab} \bmod n$

בטיחות: לוגריתם דיסקרטי. פגיע ל-MitM

RSA

$n = p \cdot q \cdot \varphi(n) = (p-1)(q-1)$

$(d,n)\ \mathrm{sk} \cdot (e,n)\ \mathrm{pk} \cdot e \cdot d \equiv 1 \pmod{\varphi(n)}$

הצפנה $c = m^e \bmod n$ · פענוח $m = c^d \bmod n$

חתימה $\sigma = m^d \bmod n$ · אימות $\sigma^e \bmod n = m$

הסתברות וגדלים

יום הולדת: כ-$\sqrt{N}$ ניסיונות ל-50% התנגשות (23 אנשים)

TXID ב-DNS: 2^{16}; עם פורט אקראי: 2^{32}; $\sqrt{2^{16}} = 256$

הוכחת מרקל: $\log_2 n$ hash-ים (מעגלים כלפי מעלה)

ביטקוין: מספרים

בלוק כל ~10 דקות · קושי מתעדכן כל 2016 בלוקים

חציית פרס כל 210,000 בלוקים (~4 שנים)

המתנה של ~6 בלוקים · 1 סאטושי = 10^{-8} BTC

טריק החישוב: חזקה מודולרית בלי מחשבון

לא מחשבים 20^7 במלואו. מצמצמים מודולו n אחרי כל כפל, ומשתמשים בריבועים חוזרים. מספר שלילי קטן קל יותר: $20 \equiv -13 \pmod{33},\ 26 \equiv -7,\ 30 \equiv -3$.

דוגמה 1 · חתימת RSA

נתון $n = 33,\ d = 7,\ m = 5$. מה החתימה $\sigma = 5^7 \bmod 33$?

$5^2 = 25$

$5^4 = 25^2 = 625 = 18 \cdot 33 + 31 \equiv 31 \equiv -2$

$5^7 = 5^4 \cdot 5^2 \cdot 5 \equiv (-2) \cdot 25 \cdot 5 = -250 \equiv -250 + 264 = \mathbf{14}$

דוגמה 2 · מחתימה להודעה

נתון $n = 33,\ d = 7,\ \sigma = 30$. מה ההודעה? קודם מוצאים את e:

$$\varphi(33) = 2\cdot 10 = 20\ ,\ 3\cdot 7 = 21 \equiv 1 \pmod{20} \Rightarrow e = 3$$

$$m = \sigma^e = 30^3 \equiv (-3)^3 = -27 \equiv \mathbf{6} \pmod{33}$$

דוגמה 3 · מציאת d

נתון $p = 5,\ q = 7,\ e = 11$:

$$n = 35\ ,\ \varphi = 4\cdot 6 = 24$$

$$11\cdot d \equiv 1 \pmod{24} \Rightarrow 11\cdot 11 = 121 = 5\cdot 24 + 1 \Rightarrow d = \mathbf{11}$$

המפתח הפרטי הוא $(11,\ 35)$. d יכול לצאת שווה ל-e. וריאציה: $p = 3,\ q = 17$ נותן $\varphi(51) = 2\cdot 16 = \mathbf{32}$.

דוגמה 4 · איזה זוג (e, d) מתאים?

נתון $n = 33$, והחתימה על $m = 20$ היא 26. $\varphi = 20$, ולכן בודקים קודם אם $e\cdot d \equiv 1 \pmod{20}$:

$$3\cdot 7 = 21 \equiv 1\ \checkmark\ ;\ 3\cdot 11 = 33 \equiv 13\ \times\ ;\ 7\cdot 11 = 77 \equiv 17\ \times\ ;\ 5\cdot 7 = 35 \equiv 15\ \times$$

אימות הזוג $(3,\ 7)$:

$$26^3 \equiv (-7)^3 = -343 \equiv -343 + 363 = 20 = m\ \checkmark$$

דוגמה 5 · RSA עם מספרים גדולים יותר

נתון $p = 13,\ q = 17$, כלומר $n = 221$, ו-$m = 7,\ d = 5$:

$$7^2 = 49\ ,\ 7^4 = 49^2 = 2401 = 10\cdot 221 + 191 \equiv 191 \equiv -30$$

$$\sigma = 7^5 \equiv -30\cdot 7 = -210 \equiv -210 + 221 = \mathbf{11}$$

דוגמה 6 · דיפי-הלמן כשהמספרים הפרטיים נתונים

נתון $n = 23,\ g = 5,\ a = 6,\ b = 15$. מחשבים את B, ואז את המפתח:

$$5^2 = 25 \equiv 2\ ,\ 5^4 \equiv 4\ ,\ 5^8 \equiv 16$$

$$B = 5^{15} = 5^8\cdot 5^4\cdot 5^2\cdot 5 \equiv 16\cdot 4\cdot 2\cdot 5 = 640 = 27\cdot 23 + 19 \equiv 19$$

$$K = B^a = 19^6 \equiv (-4)^6 = 4096 = 178\cdot 23 + 2 \equiv \mathbf{2}$$

דוגמה 7 · דיפי-הלמן כשרק הערכים שנשלחו נתונים

נתון $n = 19,\ g = 3$. אליס שלחה 15 ובוב שלח 5. כותבים את החזקות של 3 מודולו 19 עד שמוצאים את הערכים:

$$3^1{=}3\ ;\ 3^2{=}9\ ;\ 3^3{=}8\ ;\ 3^4{=}\mathbf{5}\ ;\ 3^5{=}\mathbf{15}\ ;\ \Rightarrow a = 5\ ,\ b = 4$$

$$K = B^a = 5^5 : 5^2 \equiv 6\ ,\ 5^4 \equiv 36 \equiv -2\ ,\ 5^5 \equiv -10 \equiv \mathbf{9}$$

$$A^b = 15^4 \equiv (-4)^4 = 256 = 13\cdot 19 + 9 \equiv 9\ \checkmark$$

באותה שיטה: $n = 23,\ g = 3$, נשלחו 12 ו-13, ולכן המפתח **18**. נשלחו 13 ו-4, ולכן המפתח **12**. בהרצאה: $n = 17,\ g = 3$ עם $x = 13,\ 15$ נותנים $y = 12,\ 6$, והמפתח 10.

עוד חישובים שעלולים להופיע

דוגמה 8 · חזקה מהירה (ריבועים חוזרים)

מחשבים $3^{13} \bmod 17$. כותבים את המעריך כסכום חזקות של 2: $13 = 8 + 4 + 1$. מחשבים ריבועים, ומצמצמים אחרי כל שלב:

$$3^1 = 3\ ,\ 3^2 = 9\ ,\ 3^4 = 81 \equiv 13\ ,\ 3^8 \equiv 13^2 = 169 \equiv 16$$

$$3^{13} = 3^8\cdot 3^4\cdot 3^1 \equiv 16\cdot 13\cdot 3 = 624 = 36\cdot 17 + 12 \equiv \mathbf{12}$$

זה בדיוק הערך שאליס שלחה בדוגמת דיפי-הלמן מההרצאה $(g = 3,\ n = 17,\ x = 13)$.

דוגמה 9 · מציאת d כשאין ניחוש קל (אוקלידס המורחב)

נתון $e = 7,\ \varphi = 40$. קודם בודקים ש-$\gcd(7, 40) = 1$, אחרת e לא חוקי (למשל $e = 5$ נפסל כי $\gcd(5, 40) = 5$). מחלקים עד שמגיעים לשארית 1:

$$40 = 5 \cdot 7 + 5 \ ; \ 7 = 1 \cdot 5 + 2 \ ; \ 5 = 2 \cdot 2 + 1$$

ומציבים חזרה מלמטה למעלה:

$$1 = 5 - 2 \cdot 2 = 5 - 2 \cdot (7 - 5) = 3 \cdot 5 - 2 \cdot 7 = 3 \cdot (40 - 5 \cdot 7) - 2 \cdot 7 = 3 \cdot 40 - 17 \cdot 7$$

$$d \equiv -17 \equiv \mathbf{23} \pmod{40} \ ; \ \text{check: } 7 \cdot 23 = 161 = 4 \cdot 40 + 1 \ \checkmark$$

במבחן אמריקאי לרוב מהיר יותר לבדוק כל תשובה: האם $e \cdot d \bmod \varphi = 1$.

דוגמה 10 · מחזור RSA מלא: הצפנה ופענוח

$p = 5,\ q = 11$, ולכן $\varphi = 40,\ n = 55$, ומדוגמה 9: $e = 7,\ d = 23$. מצפינים $m = 2$:

$$c = 2^7 = 128 = 2 \cdot 55 + 18 \equiv \mathbf{18}$$

מפענחים $18^{23} \bmod 55$, כאשר $23 = 16 + 4 + 2 + 1$:

$$18^2 = 324 \equiv 49 \equiv -6 \ , \ 18^4 \equiv 36 \ , \ 18^8 \equiv 36^2 = 1296 \equiv 31 \ , \ 18^{16} \equiv 31^2 = 961 \equiv 26$$

$$18^{23} \equiv 26 \cdot 36 \cdot (-6) \cdot 18 \ : \ 26 \cdot 36 = 936 \equiv 1 \Rightarrow 1 \cdot (-6) \cdot 18 = -108 \equiv -108 + 110 = 2 = m \ \checkmark$$

דוגמה 11 · צפנים קלאסיים

Vigenère: מצפינים ATTACK עם המפתח LEMON. משרשרים את המפתח לאורך הטקסט (LEMONL) ומחברים מספרי אותיות מודולו 26 (A=0):

$$A+L = 0+11 = L \ ; \ T+E = 19+4 = X \ ; \ T+M = 19+12 = 31 \equiv 5 = F \ ; \ A+O = O \ ; \ C+N = 2+13 = P \ ;$$
$$K+L = 10+11 = V \Rightarrow \mathbf{LXFOPV}$$

קיסר בכיוון ההפוך: KHOOR פחות 3 בכל אות נותן **HELLO**. בצופן הזזה כללי יש רק 25 מפתחות, ולכן brute force מסתיים אחרי 25 ניסיונות לכל היותר.

דוגמה 12 · XOR ו-OTP

$$m = 1011 \ , \ k = 0110 \Rightarrow c = m \oplus k = \mathbf{1101} \Rightarrow c \oplus k = 1011 = m$$

אם אותו k משמש פעמיים: $c_1 \oplus c_2 = m_1 \oplus m_2$, כלומר המפתח נעלם לגמרי מהביטוי.

דוגמה 13 · גודל מרחב מפתחות וזמן brute force

- **DES:** מפתח 56 ביט, כלומר $2^{56} \approx 7.2 \cdot 10^{16}$ מפתחות. בקצב של 10^9 ניסיונות בשנייה: כ-2.3 שנים במקרה הגרוע, ובממוצע חצי מזה (2^{55} ניסיונות). **AES-128:** 2^{128}, לא בר-ביצוע.
- **סיסמה:** 8 אותיות קטנות באנגלית: $26^8 \approx 2.1 \cdot 10^{11}$, כלומר כ-209 שניות ב-$10^9$ hash-ים בשנייה.
- **salt** לא מגדיל את מספר הניסיונות לסיסמה אחת, אבל מכריח לחזור על כל העבודה **לכל משתמש בנפרד** ומבטל טבלאות מוכנות. **hash חוזר i פעמים** מכפיל את עלות כל ניסיון פי i (עם $i = 1000$, 209 שניות הופכות לכ-58 שעות).

דוגמה 14 · פרדוקס יום ההולדת

$$P(\text{collision}) \approx 1 - e^{-k^2/2N} \ ; \ k = 23,\ N = 365 \Rightarrow \approx 50\%$$

- כלל האצבע של הקורס: כ-$\sqrt{N}$ ניסיונות לסיכוי של כ-50%. ל-TXID של 16 ביט: $\sqrt{2^{16}} = \mathbf{256}$. (חישוב מדויק: 256 ניסיונות נותנים כ-39%, ול-50% בדיוק צריך כ-$1.18 \cdot \sqrt{N} \approx 302$. במבחן עונים 256.)
- hash של n ביט: התנגשות אחרי כ-$2^{n/2}$ ניסיונות. ל-SHA-256 זה 2^{128}.

דוגמה 15 · הסתברויות בהרעלת DNS

- תשובה מזויפת אחת שמנחשת TXID: $1/2^{16} = 1/65,536$.
- k תשובות מזויפות לאותה שאילתה: בערך $k/2^{16}$. עם k = 100: $100/65,536 \approx 0.15\%$.
- עם פורט מקור אקראי צריך לנחש גם 16 ביט של פורט: $k/2^{32}$, כלומר קשה פי 65,536.

דוגמה 16 · עץ מרקל וביטקוין במספרים

- עץ עם n עלים (n חזקה של 2): $n - 1$ צמתים פנימיים, גובה $\log_2 n$. הוכחת הכללה ושינוי של עלה אחד: $\log_2 n$ צמתים (1024 עלים: 10; 1000 עלים: $10 \to \log_2 1000 \approx 9.97$).
- תקופת קושי: $2016 \times 10 \text{ min} = 20,160 \text{ min} = 14 \text{ days}$.
- פרס בלוק: $3.125 \to 6.25 \to 12.5 \to 25 \to 50$, חצייה כל 210,000 בלוקים. סך כל המטבעות: $210,000 \times 50 \times 2 = 21,000,000$.
- כורה עם 1% מכוח החישוב ימצא בממוצע בלוק אחד מכל 100, כלומר פעם ב-1000 דקות (כ-17 שעות).

דוגמה 17 · בלוקים וריפוד

הודעה של 20 בתים בצופן DES (בלוק של 8 בתים) מתחלקת ל-$20/8 = 2.5 \to 3$ בלוקים, עם 4 בתי ריפוד בבלוק האחרון. ב-ECB, שני בלוקים זהים ב-plaintext נותנים שני בלוקים זהים ב-ciphertext. ב-CBC לא.

סיכום · תשובות מהירות שחוזרות

קיסר על QUIZ	**TXLC**	עץ מרקל עם 1024 עלים	**10** hash-ים
ניחוש TXID בסיכוי 50%	**256**	7 תווים זהים, צופן החלפה על 28 אותיות	**28**
החלפה עם מיפוי אחד ידוע	**25!**	Vigenère, מפתח באורך 4, תווים במרחק 4	**1/26**

12 מילון מושגים

0x20 encoding
אותיות גדולות/קטנות אקראיות בשם הדומיין בשאילתת DNS, שהתשובה חייבת להחזיר בדיוק. מקשה על זיוף.

3DES
שלוש הפעלות DES במבנה E-D-E. עם שלושה מפתחות זהים שקול ל-DES (תמיכה לאחור).

attack 51%
שליטה ברוב כוח הכרייה, שמאפשרת לקבוע אילו עסקאות ייכנסו ולבצע double spending.

AES
צופן בלוקים סימטרי מודרני (בלוק 128 ביט). דורש mode of operation.

Amplification
התקפת DDoS שבה התשובה גדולה מהבקשה (למשל DNS דרך open resolver), כך שהתוקף מגביר את הנזק.

Anti-CSRF token
ערך אקראי בטופס שהשרת בודק. שקוף למשתמש. נשבר אם יש XSS.

ARP
פרוטוקול שממפה IP ל-MAC ברשת מקומית. אין בו אימות, והתשובה האחרונה דורסת.

ARP poisoning
שליחת תשובות ARP שקריות כדי שתעבורה תגיע לתוקף. מספיק off-path, ומשיג MitM.

Bailiwick rule
שרת DNS רשאי לספק רשומות רק לתחום שבסמכותו. רשומות זרות נזרקות.

Birthday paradox
כ-√N ניסיונות נותנים 50% להתנגשות מתוך N ערכים (23 אנשים ליום הולדת משותף).

Blacklist / Whitelist
רשימה שחורה: ברירת מחדל לאפשר וחוסמים את הרשום. רשימה לבנה: ברירת מחדל לחסום ומאפשרים רק את הרשום.

Blind SQL injection
הזרקה בלי לראות פלט, תוך הסקת אמת/שקר מהתנהגות הדף או מזמן התגובה.

Block cipher
צופן שמצפין בלוקים בגודל קבוע (DES, AES).

Botnet / C&C
רשת מחשבים שנפרצו, שנשלטת דרך שרתי פיקוד ושליטה. מפילים את ה-C&C כדי לנטרל.

Brute force
חיפוש ממצה של כל המפתחות או הסיסמאות האפשריים.

CA
רשות מאשרת שמוודאת בעלות על דומיין וחותמת על תעודות במפתח הפרטי שלה.

CBC
Mode שבו כל בלוק עובר XOR עם ה-ciphertext הקודם לפני ההצפנה. מתחיל ב-IV אקראי.

Certificate Transparency (CT)
רישום חובה של כל תעודה ביומנים ציבוריים (עץ מרקל), לגילוי הנפקה זדונית.

Chain of trust
שרשרת חתימות מתעודת האתר דרך intermediate CA עד Root CA שבמאגר האמון.

Clickjacking
iframe שקוף שגורם ללחיצה לא מודעת. הגנה: X-Frame-Options / frame-ancestors.

Cookie / HttpOnly
מחרוזת שהדפדפן מצרף לבקשות לאתר. HttpOnly חוסם גישה אליה מ-JavaScript.

CRL / CRLSets
רשימת תעודות שבוטלו שה-CA מפרסם / רשימה מצומצמת שהדפדפן מוריד.

CSP
Header שמגדיר מאילו מקורות מותר לדפדפן לטעון ולהריץ משאבים. הגנה מ-XSS.

CSRF / XSRF
זיוף בקשה מדפדפן הקורבן לאתר שבו הוא מחובר, תוך ניצול צירוף אוטומטי של עוגיות.

DDoS
מניעת שירות ממקורות רבים (botnet).

DES
צופן בלוקים סימטרי ישן: בלוק 64 ביט, מפתח 56 ביט.

Diffie-Hellman
הסכמה על מפתח משותף בערוץ גלוי, על בסיס בעיית הלוגריתם הדיסקרטי. פגיע ל-MitM.

Digital signature
חתימה בפרטי ואימות בפומבי. נותנת אמינות, אותנטיות ואי-הכחשה.

DMZ
תת-רשת מופרדת לשרתים שנגישים מבחוץ.

DNS poisoning
שתילת רשומת DNS שקרית ב-cache של resolver.

DNSSEC
חתימה דיגיטלית על רשומות DNS. מונעת זיוף תשובות.

Double spending
שימוש באותו מטבע דיגיטלי פעמיים.

Downgrade (שנמוך)
אילוץ שימוש בגרסה או אלגוריתם חלשים (switch ל-hub, TLS לצופן חלש, HTTPS ל-HTTP).

ECB
Mode שמצפין כל בלוק בנפרד. בלוקים זהים מוצפנים זהה.

Encrypt-then-MAC
מצפינים ואז מחשבים MAC על ה-ciphertext. המקבל בודק MAC לפני פענוח.

Escrow
עסקת נאמנות: ארנק 2-מתוך-3 של קונה, מוכר וצד שלישי.

Fork (soft / hard)
פיצול בשרשרת. Soft: כללים מוחמרים, תואם לאחור. Hard: שינוי שצמתים ישנים דוחים.

Forward secrecy
גניבת מפתח ארוך טווח לא חושפת תעבורה מהעבר. אין כזו ב-RSA key exchange.

Hash function
פונקציה חד-כיוונית לפלט קבוע (SHA-256). תכונות: pre-image, second pre-image, collision.

HSTS / Preload
הוראה לדפדפן לגשת לאתר רק ב-HTTPS / רשימה מובנית בדפדפן גם לביקור ראשון.

Hybrid encryption
מפתח סימטרי שמועבר מוצפן במפתח פומבי.

IDS / IPS
מערכת זיהוי חדירות (פסיבית, מתריעה) / מערכת מניעה (חוסמת).

IV
וקטור אתחול אקראי, לא סודי, לבלוק הראשון ב-CBC.

Kaminsky attack
הרעלת DNS דרך שאלות על תתי-דומיינים לא קיימים, שעוקפת את ה-cache.

Kerckhoffs
הבטיחות צריכה להישען רק על סודיות המפתח.

LockTime
שדה שמונע מעסקה להיכנס לבלוק לפני זמן מסוים (micropayments).

Logger / Monitor / SCT
יומן CT ציבורי / מי שמנטר את הלוגרים / חותמת זמן חתומה של הלוגר שמוטמעת בתעודה.

MAC (קוד)
Tag שמחושב עם מפתח משותף. אמינות ואותנטיות, בלי אי-הכחשה.

MAC flooding
הצפת טבלת ה-CAM של switch כך שיתנהג כמו hub. הגנה: port security.

Merkle tree
עץ hash-ים בינארי. הוכחת הכללה ב-log(n) ערכים.

Micropayments
תשלומים קטנים רבים, שרק האחרון מתפרסם, עם החזר מובטח דרך LockTime.

Mining pool
קבוצת כורים שמחלקת עבודה ופרס.

MitM
תוקף בתווך: קורא, משנה, חוסם ומזריק.

Mode of operation
שיטה להצפנת הודעה ארוכה בצופן בלוקים (ECB, CBC).

Multisig
כתובת שדורשת כמה חתימות כדי לבזבז.

Nonce (ביטקוין)
מספר שמשנים בחיפוש אחר hash של בלוק עם מספיק אפסים.

Non-repudiation
אי-הכחשה: השולח לא יכול להכחיש. ניתנת ע"י חתימות, לא ע"י MAC.

OCSP / Stapling
שאילתת סטטוס תעודה בזמן אמת / השרת מצרף תשובה חתומה מראש.

Off-path
תוקף שיכול רק לשלוח הודעות (כולל מזויפות). לא רואה ולא חוסם.

One-Time Pad
XOR עם מפתח אקראי באורך ההודעה, לשימוש חד-פעמי. בטוח לחלוטין.

Origin
פרוטוקול + דומיין + פורט.

Pepper
סוד שמשורשר לסיסמה ולא נשמר ב-DB.

PKI
תשתית שקושרת מפתחות פומביים לזהויות באמצעות תעודות ו-CA.

Port security
הגבלת כתובות MAC לכל פורט ב-switch.

Prepared statement
שאילתה עם פרמטרים, שבה הקלט לא משנה את המבנה. ההגנה העיקרית מ-SQLi.

Proof of Work / Stake
זכות יצירת בלוק לפי עבודה חישובית / לפי כמות מטבעות.

PRG
מחולל פסאודו-אקראי שמרחיב seed קצר לרצף ארוך.

Reflection (TLS)
החזרת הודעה של הלקוח אליו כאילו מהשרת. נמנעת עם מפתח נפרד לכל כיוון.

Root store
רשימת Root CA שמערכת ההפעלה או הדפדפן סומכים עליהם.

RSA
הצפנה וחתימה א-סימטריות, על בסיס קושי פירוק $n = p \cdot q$.

Salt
ערך אקראי לכל משתמש שמשורשר לסיסמה ונשמר גלוי. מונע מילונים מוכנים.

Same-Origin Policy
מנגנון בדפדפן שמונע מסקריפט לקרוא מידע של origin אחר.

SameSite cookie
עוגייה שלא נשלחת בבקשות שמקורן באתר אחר. הגנה מ-CSRF.

Session resumption
חידוש חיבור TLS עם session id קודם, בלי ClientKeyExchange.

Slowloris
DoS בקצב נמוך שמחזיק חיבורי HTTP פתוחים עם בקשות חלקיות.

Soft fail
קבלת תעודה כשאין תשובת ביטול. ניתן לניצול ע"י MitM.

Spoofing
זיוף כתובת מקור (IP, MAC) או זהות.

SQL injection
שינוי מבנה שאילתת SQL דרך קלט משתמש.

Stateful / Stateless filter
חומת אש שזוכרת חיבורים / שמחליטה על כל חבילה בנפרד.

Stream cipher
XOR עם רצף פסאודו-אקראי (RC4). מהיר.

SYN cookies
קידוד מצב החיבור במספר הסידורי, בלי זיכרון בשרת. הגנה מ-SYN flood.

SYN flood
הצפת SYN מכתובות מזויפות כדי למלא את זיכרון החיבורים החצי-פתוחים.

TLS
פרוטוקול להצפנת חיבור: handshake א-סימטרי ותעבורה סימטרית.

TOFU
Trust On First Use: הביקור הראשון חשוף (הבעיה של HSTS בלי preload).

TOR / Mix net
ניתוב בשכבות הצפנה דרך 3 צמתים / ערבוב והשהיית הודעות.

Transaction ID
מזהה של 16 ביט לשאילתת DNS.

VPN
מנהרה מוצפנת לשרת שגולש בשמך.

Vigenère
צופן הזזה עם מפתח חוזר. נשבר ע"י מציאת אורך המפתח וניתוח שכיחויות.

WAF
חומת אש לאפליקציות Web, שבודקת תוכן HTTP.

XSS
הזרקת סקריפט שרץ בדפדפן הקורבן מתוך האתר הפגיע. סוגים: stored, reflected, DOM-based.

מקורות

הסיכום מבוסס על סיכום ההרצאות של הקורס (הרצאות 1–12), על הסילבוס ועל 26 קבצי המבחנים שבמאגר (2022–2026). השלמות ואימותים מהאינטרנט:

- TLS 1.3 / RFC 8446, הסרת RSA key exchange ו-forward secrecy: noze.it/en/insights/tls-1-3-rfc-8446
- מדיניות Certificate Transparency של Chrome (מספר SCT-ים): groups.google.com/a/chromium.org/g/ct-policy
- OCSP Stapling, soft-fail ו-Must-Staple: ssl.com/article/ocsp-stapling-secure-and-efficient-certificate-validation
- חציית פרס וכוונן קושי בביטקוין: forklog.com/en/what-is-the-bitcoin-halving
- SameSite cookies ו-CSRF: simonwillison.net/2021/Aug/3/samesite